

IMPERIAL

FINAL REPORT

MSc INDIVIDUAL PROJECT

MSc HUMAN AND BIOLOGICAL ROBOTICS

IMPERIAL COLLEGE LONDON

DEPARTMENT OF BIOENGINEERING

Multimodal Teleoperation of a Wearable Dual-Arm Supernumerary Robotic System: Master Interface Design, Safety Architecture, and a Workspace Characterisation

Author:

Gaus Sayyad

CID: 06053822

Supervisor:

Dr. Dandan Zhang

Co-Supervisor:

Dr. Etienne Burdet

August 31, 2026

Submitted in partial fulfilment of the requirements for the degree of
Master of Science in Human and Biological Robotics
at Imperial College London

Word count: 38 210

Body text of [Chapter 1](#) to [Chapter 11](#) inclusive, counted from the typeset document. Excludes the title page, abstract, acknowledgements, contents, appendices and references, and includes table and figure captions.

Abstract

The base of a supernumerary robotic limb is a person. The aim of this project was to build a teleoperation system for two seven-degree-of-freedom arms carried on a worn backpack frame and commanded by a *second* person, give it a safety layer appropriate to a manipulator standing beside somebody who is not commanding it, and characterise what it can and cannot do.

A body-worn master supplies position from whichever sensor observes each component most directly, and classifies all fourteen of its channels so that a failing sensor degrades the system announcedly rather than silently. Sixteen simulations, run by one command, and four physical cameras measured the result.

Six of fourteen master channels met the health criterion, and the failing ones injected 54.5 mm of commanded motion nobody asked for. Vertical and fore-and-aft command track correctly; lateral does not, because directions intended to be 90° apart measure 137.7° apart, and no axis mapping makes a non-orthogonal triad orthogonal. The two arms share no reachable point, which blocks inter-arm handover for geometric reasons no control change addresses. The task set solves 13 927 inverse-kinematics calls with no failures and no breach of the 150 mm clearance floor around the wearer; replacing the motion generator removed 51.3 mm of unrequested excursion; and measuring object positions rather than declaring them takes the autonomy layer from confidently wrong on half of trials to never.

Nothing was validated on a loaded physical arm, and the two-person study the architecture was built for was designed in full but could not be run.

Acknowledgements

I thank my supervisor, Dr. Dandan Zhang, for the direction of this project and for consistently pushing the work toward what could be measured rather than what could be claimed. Much of what is useful in this thesis exists because a result was questioned before it was written down.

I thank my co-supervisor, Dr. Etienne Burdet, for the framing that changed the shape of the project. Treating the wearer and the operator as two people with different stakes, rather than as one role, is what turned a control problem into a question worth asking, and it is the reason the workspace and safety chapters exist in the form they do.

I am grateful to the Human Robotics Group for access to the Dr. Octopus platform, and to everyone who kept two Kinova arms and a backpack frame available and working.

This project spent a great deal of its time discovering that the instrument was wrong rather than the system. I am thankful to everyone who responded to a surprising number with scepticism instead of enthusiasm, which is the only reason several of them are not in this document as findings.

Finally, I thank my family and friends for their support over the course of the year.

Contents

1	Introduction	12
1.1	Two design premises	12
1.1.1	Commanding fourteen joints is a usability problem, not a plumbing problem	13
1.1.2	A body-worn master will lose channels, so losing them must be survivable	13
1.2	Objectives	13
1.3	Challenges	14
1.4	Contributions	14
1.5	Report structure	16
2	Background	17
2.1	Supernumerary robotic limbs	17
2.2	The operator and the wearer as different people	17
2.3	Master interfaces for dissimilar kinematics	18
2.4	Shared autonomy	19
2.5	Measuring embodiment, workload and trust	19
2.6	The gap	20
3	The master arm	22
3.1	Mechanical chain	22
3.2	Electronics	22
3.2.1	The A4 and A5 collision	25
3.3	The inertial measurement driver	26
3.3.1	Scale factors	26
3.4	Signal conditioning	27
3.4.1	Potentiometers	27
3.4.2	A start-up transient in the filter initialisation	27
3.4.3	A candidate explanation for the observed update rate	28
3.4.4	Force-sensitive resistors	28
3.4.5	Buttons	28
3.5	The serial frame	29

3.6	The CAD is not the kinematic source	29
3.7	Summary	30
4	System as built	31
4.1	Architecture	31
4.2	From master pose to arm command	33
4.2.1	Spherical decomposition	33
4.2.2	Anchoring, scaling and indexing	33
4.3	Graceful degradation	34
4.3.1	Channel health	34
4.3.2	Mode-scoped validation	34
4.3.3	The degradation ladder	35
4.3.4	Reduced-degree-of-freedom fallback	35
4.4	Control modes	36
4.4.1	The transport, and why it changed	36
4.4.2	The controller route is motion capture, not virtual reality	41
4.4.3	Why the voice mode uses a grammar and not a language model	41
4.4.4	Why not an end-to-end vision-language-action policy	41
4.5	Safety architecture	42
4.5.1	Collision avoidance in three stages	42
4.5.2	Blocking must be observable	43
4.5.3	Expiry is not permission	43
4.5.4	Emergency stop	43
4.6	The operating window	44
4.7	Verification methodology	45
4.8	Calibration	45
5	Perception, environment mapping and autonomous manipulation	48
5.1	The defect that motivated it	48
5.2	Detection	48
5.2.1	A reachability gate that could not fail	50
5.3	Measuring the working area	50
5.3.1	Why the sweep holds one wrist attitude	50
5.3.2	Stationary is not arrived	50
5.3.3	What the map contains	50
5.3.4	Two lifetimes, one command	52
5.4	Planning against the measurement	52
5.5	Commanding a task in natural language	54

5.6	Observing the wearer	55
5.7	The scene camera	56
6	Motion generation and simulation-to-hardware transfer	57
6.1	The step limiter was the entire motion generator	57
6.1.1	The reported metric had to change first	57
6.1.2	It is not a tuning parameter	58
6.1.3	Why a per-joint clamp cannot stay on the line	58
6.2	What replaced it	58
6.2.1	Five defects in the replacement, each caught by a check	59
6.2.2	One definition of which joints wind	60
6.3	The simulation-to-hardware gap, measured	60
6.3.1	A correction to this project's own reporting	61
6.3.2	The deadband was larger than the error it permitted	61
6.4	Where the end effector actually arrives	61
6.5	The reaction budget	62
7	Results	63
7.1	The simulation campaign	63
7.1.1	What ran, and what it needed	64
7.1.2	Computer vision, against a scene whose answer is known	64
7.1.3	Teleoperation: what the motion generator does to a command	67
7.1.4	Teleoperation: what the smoother does to the operator's hand	67
7.1.5	Shared autonomy: what the intent estimator is actually sensitive to	68
7.1.6	Full autonomy: seventy-five ways of asking for the same task	69
7.1.7	What the campaign could not run, and why that is stated here	70
7.1.8	The task set, inside the simulated platform	70
7.2	Channel health	72
7.2.1	Progressive degradation between two dates	73
7.3	What the degradation architecture costs	73
7.3.1	Which channels a repair should target	73
7.3.2	What the incoherent channels inject	75
7.3.3	The pipeline manufactures no motion of its own	75
7.4	Command mapping	75
7.4.1	Axis correctness	75
7.4.2	The lateral axis remains wrong, and the axis map cannot fix it	76
7.4.3	Indexing	76
7.5	Safety	77

7.5.1	Fault injection	77
7.5.2	Dead-man timing	77
7.5.3	An emergency stop that was four seconds late	77
7.6	Grasping	78
7.6.1	Top-down grasping has a lateral boundary	78
7.7	The wrist cannot be aligned under teleoperation	78
7.8	Task-set verification	79
7.9	Performance figures	79
7.10	Motion generation	81
7.11	The environment as measured rather than declared	81
7.12	Instruction handling	81
7.13	The recorded demonstration set	81
7.14	The first hardware session	82
7.15	Verification, in totals	82
7.16	What is not measured	82
8	The vision pipeline, measured on the real cameras	84
8.1	Method: one figure and one formula per stage	84
8.1.1	The geometry, written out	85
8.2	The orientation defect	85
8.2.1	What the defect did and did not corrupt	86
8.3	What the cameras measured	86
8.3.1	The two defects the first runs exposed	88
8.3.2	What did not work, and why that is an answer	88
8.3.3	A caption that could not be written	89
8.4	What this chapter changes	89
9	Workspace characterisation and its consequences	90
9.1	Continuous reachability from the home pose	90
9.2	The two arms share no reachable workspace	90
9.2.1	What this does and does not establish	93
9.3	The fix is buildable	93
9.4	One boundary, two independent constraints	93
9.5	The centre of the working space, asked four ways	94
9.6	What the applied mount change bought	95
9.7	Clearance along the task paths	95
9.8	Consequences for the task set	95
10	Discussion	98

10.1	What the degradation architecture bought, and what it did not	98
10.2	The reduced-degree-of-freedom premise, assessed	98
10.3	The workspace result and what it costs the study	99
10.4	Comparison against the planning report's targets	99
10.5	The safety layer is thorough about distance and silent about time	99
10.5.1	The fallback restores an assumption that is optimistic by 224 mm	100
10.5.2	And the clearance model itself is not yet trustworthy	100
10.6	What the perception stage did and did not change	100
10.6.1	What the real cameras changed about that assessment	101
10.7	Validity of the measurements	101
10.7.1	The base rate of instrument error, and what caught it	101
10.7.2	Simulation is not the arm	102
10.7.3	Latency figures are lower bounds	102
10.7.4	The one-stack constraint	102
10.8	The study the platform is not ready for	102
11	Conclusion	104
11.1	Against the objectives	104
11.2	What was not achieved	105
11.3	Further work	105
11.4	Closing	106
A	The route the design took	107
A.1	Phase 1: joint-space control in simulation	107
A.1.1	What the prior programs actually did	108
A.1.2	Why joint-space control was abandoned	108
A.2	Phase 2: the move to ROS 2	109
A.2.1	What was kept from simulation	109
A.3	Phase 3: from Cartesian control to a reduced-degree-of-freedom command	109
A.3.1	Alternatives considered at this branch	110
A.4	Summary of the sequence	110
B	Measurement validity: the full record	112
B.1	The rule	112
B.2	The cases	112
B.3	Four mechanisms	113
B.4	What actually caught them	113
B.5	Consequences for this thesis	116

C	Protocol for the two-person study	117
C.1	The question	117
C.2	Hypotheses	117
C.3	Design	118
C.3.1	Which cells exist	119
C.4	Tasks	119
C.5	Measures	119
C.6	Procedure	121
C.7	Analysis	121
C.8	Ethics and risk	122
D	The analysis procedure, and its behaviour on constructed data	123
D.1	The analysis, written out	123
D.2	What the analysis does to data whose answer is known	124
D.3	What the session will report	125
D.4	Safety events	126
D.5	Why no session has been run	126
E	Master arm firmware	127
E.1	Pin assignment as declared	127
E.2	Signal conditioning as implemented	127
E.3	Frame construction	128
F	Supporting records	130
G	Evaluation criteria	131
H	Equipment	133
I	Where the evidence lives	134
J	Participant documentation	135

List of Figures

1.1	The two-person arrangement	12
2.1	Two load paths for a supernumerary limb	18
2.2	Arbitration against prediction confidence	19
2.3	The platform as built	21
3.1	Master kinematic chain	23
3.2	The master arm, from model to kinematic description	24
3.3	Master arm electronics	25
4.1	Package architecture	31
4.2	The command path	32
4.3	Degradation ladder	35
4.4	Data path: direct teleoperation from the mannequin	37
4.5	Data path: direct teleoperation from VR	38
4.6	Data path: shared autonomy	39
4.7	Data path: full autonomy from an instruction	40
4.8	The safety layers	42
4.9	The operating window	44
4.10	The simulated scene	45
4.11	Calibration procedure	46
4.12	The model at the home pose	47
5.1	Measure, then plan	49
5.2	Detection: the support plane, fitted and scored	51
5.3	The measured map	53
5.4	The scene an instruction is resolved against	55
6.1	Positioning error budget	61
7.1	The clamp against its own step size	67
7.2	Intent inference against a mis-declared table	69

7.3	Instruction handling, before and after	69
7.4	Channel health	74
7.5	Progressive channel degradation	74
7.6	The lateral constraint	76
7.7	The readiness panel with faults present	80
8.1	The scene camera after the orientation correction	87
9.1	Continuous reachability from home	91
9.2	The mounted configuration at the home pose	92
9.3	The measured dead band	92
9.4	Mount parameter sweep	93
9.5	Two constraints, one boundary	94
9.6	Clearance by task	96
9.7	Compliant carrier geometry	97
C.1	Experimental design	118
C.2	Session timeline	121
D.1	Power against the number of dyads	124

List of Tables

3.1	Pin assignment. Sixteen analog channels are required and sixteen are available once the two occupied by the default I ² C bus are excluded.	22
3.2	Inertial sensor scaling at power-on defaults.	26
3.3	Filter output after reset, simulated from Eq. (3.1) with a constant input of 2000 counts and the seed used in <code>setup()</code>	27
3.4	Serial frame, in transmission order. Fields are comma-separated; the frame ends with a newline. Buffer is 460 bytes against a typical frame of roughly 285.	29
4.1	Operating modes. Autonomy runs slower than teleoperation by design: nobody is holding a master, so the only bound on a wrong motion is how long it takes to happen.	36
4.2	Blocker states. Expiry means the asserting loop stopped running, so the guarded condition is unknown rather than absent.	43
5.1	Measured map accuracy	52
5.2	Calibration lifetimes	52
5.3	Instruction handling	54
5.4	Wearer tracking rate and latency	55
6.1	Motion generation	58
6.2	Step size sweep	58
6.3	Reaction budget	62
7.1	Where each result comes from	64
7.2	The simulation campaign	65
7.3	The vision pipeline against a constructed scene	66
7.4	Three motion generators on one slew	67
7.5	Master smoothing	68
7.6	The smoother over a recorded session	68
7.7	The task set inside the simulated platform	70
7.8	Reach against orientation policy, at each sampling density	71
7.9	Per-channel verdicts	73
7.10	Reach observability from the surviving channels.	74

7.11	Commanded end-effector motion contributed by the incoherent channels alone, with the operator's input held constant.	75
7.12	Dead-man behaviour by failure mode.	77
7.13	Five-task verification. Both arms verified at home; ten repeats per pose; short-circuit on first failure.	79
7.14	Measured rates and latencies. All against mock hardware.	79
7.15	Verification totals	83
8.1	Stage outcomes by camera	86
8.2	Geometry recovered from each depth camera	87
9.1	Both-arm overlap under three test conditions	91
9.2	The centre, four measurements	94
9.3	Before and after the mount change	95
A.1	Development sequence, with the reason each stage ended.	110
B.1	Every occasion on which a measurement was wrong and the system was not. <i>Caught by</i> is the thing that actually revealed it, not the thing that should have.	114
B.2	Instrument faults, later phases	115
B.3	What revealed each of the twenty-six.	115
C.1	Design options costed before the study was fixed	119
C.2	Task set	120
C.3	Measures	120
C.4	Risks and mitigations	122
D.1	The analysis on constructed data	124
D.2	The reporting plan	125
D.3	Safety events during sessions	126
F.1	Supporting records and where they live.	130
G.1	Evaluation criteria	131
H.1	Equipment	133
I.1	Evidence index	134

Chapter 1

Introduction

A supernumerary robotic limb is an extra arm worn on the body, working alongside the wearer’s own limbs rather than replacing or rehabilitating them [35, 30]. The wearer keeps full natural function, which separates these devices from prostheses. What separates them from fixed manipulators is more consequential for control: the base of the robot is a person. It accelerates, sways and steps. It is soft and irreplaceable. It cannot be told to hold still.

This thesis reports work on the MUlti-limb Virtual Environment, known within Imperial College London as the Dr. Octopus platform [12], which mounts two Kinova Gen3 seven-degree-of-freedom arms [17] on a backpack frame. The problem addressed is the command interface. Two redundant seven-degree-of-freedom arms present fourteen commandable joints, and a person has two hands and a limited attention budget. Something has to close that gap.

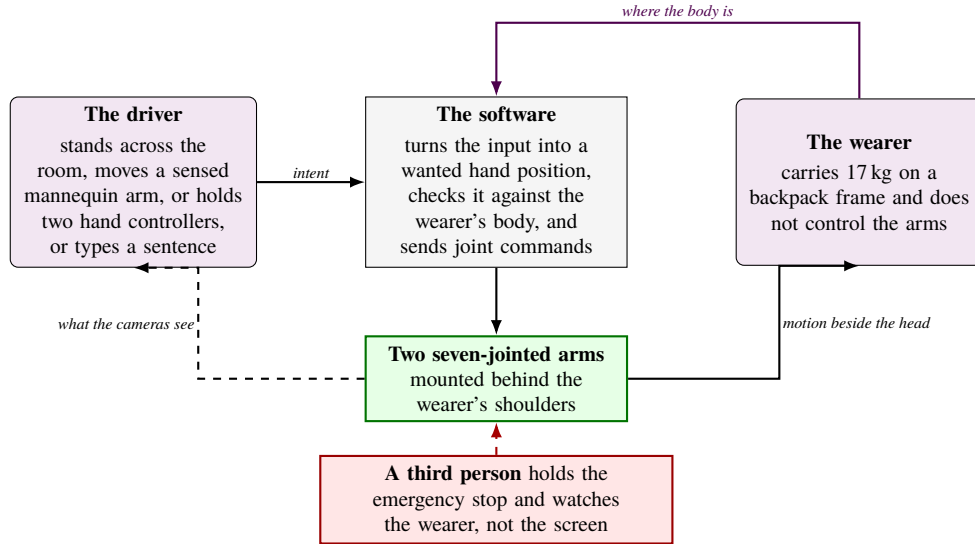


Figure 1.1: The arrangement this thesis addresses. The operator commands the arms and cannot feel them; the wearer carries them and does not command them. Neither person is in a position to guarantee the wearer’s safety, so the software between them is, and that allocation is what the safety architecture of [Chapter 4](#) exists to discharge.

1.1 Two design premises

The work rests on two premises that were adopted early and shaped everything downstream. Both are stated here because the rest of the thesis only makes sense if they are visible from the start.

1.1.1 Commanding fourteen joints is a usability problem, not a plumbing problem

The naive interface exposes every joint. It is also unusable. A body-worn master that reproduces the slave's full kinematic chain requires the operator to coordinate seven degrees of freedom per arm through finger motion, with no force feedback and no view of the joint the arm is actually in. Earlier work on miniature physical masters makes the case for morphological similarity between master and slave [10], and that case is accepted here. What does not follow is that every degree of freedom of the slave must be commanded independently by the operator.

The architecture adopted instead splits the command by what each source measures well. Position comes from three or four well-conditioned potentiometer channels resolved into a spherical form about the master's shoulder. Orientation comes from the inertial measurement unit at the wrist, which observes gravity directly and therefore needs no integration and no calibration against an absolute reference. What is left over, principally the wrist orientation required to grasp a particular object, is supplied by autonomy or is not commanded at all.

This is a reduced-degree-of-freedom design by intent. It is the reason the system uses a spherical decomposition rather than forward kinematics from seven joint angles, and the reason its default orientation mode pins the wrist rather than tracking it. Both choices cost capability, and both are quantified in [Chapter 7](#).

1.1.2 A body-worn master will lose channels, so losing them must be survivable

The master carries fourteen analog channels routed through seven rotating joints per arm. Connectors fatigue. Wires flex through joints that turn. Over the life of any such device some fraction of its channels will stop reporting usable values, and the fraction will grow. A design whose behaviour is undefined when one channel fails is not a design that has been finished.

The system therefore treats channel loss as an operating condition rather than a fault. Each channel carries a health verdict computed from its own recent history. Validation is scoped to the channels the active position mode consumes, so a dead channel that the mode does not read cannot reject a frame. Individual channels can be disabled at run time, and the resulting loss of capability is published rather than inferred. When enough channels are lost that position can no longer be estimated, the system reports position unavailable instead of extrapolating one.

That last behaviour is the design's centre of gravity. On a manipulator bolted beside a person's head, a fabricated position is worse than no position, and the difference between them has to be visible from outside the process.

1.2 Objectives

The planning report framed this project around a haptic master and slave pair with physical springs at the master and virtual impedance at the slave. The delivered system diverged from that plan, for reasons set out in [Chapter 10](#) and recorded in full in the accompanying divergence analysis, which is retained as a supporting file rather than as an appendix because it documents the route rather than the result. The objectives below are the revised set, aligned to what was built and measured.

1. To design and build an instrumented mannequin master reproducing the Gen3 morphology at reduced scale, and to characterise its sensing against recorded data rather than against specification.
2. To derive a mapping from master pose to slave end-effector command that degrades predictably as sensing channels are lost, and to quantify what each loss costs.

3. To implement a safety architecture appropriate to a manipulator mounted beside a person, and to validate it by injecting faults rather than by inspecting code.
4. To characterise the reachable workspace of the mounted configuration and determine which manipulation tasks it can support.
5. To establish a verification methodology producing traceable evidence for every claim, and to apply it across the task set.
6. To identify, by measurement, what must be resolved before hardware validation and a user study become possible.

1.3 Challenges

Four difficulties consumed most of the effort.

The master and the slave are not the same shape. The master reproduces the Gen3's joint sequence but not its proportions. Its links sum to 0.272 m against the arm's 0.902 m reach, a ratio of roughly three to one, so joint-to-joint correspondence is unavailable. Working in task space instead raises a second problem, which is that the two chains reach different sets of poses. The resolution was to map displacement of the master's tip onto displacement of the slave's end effector about an anchor that can be re-latched, described in [Chapter 4](#).

Two coordinate conventions meet at the driver. The manufacturer's controller reports joint positions in degrees over $[0, 360)$; the middleware uses radians over $(-\pi, \pi]$. The failure this invites is not a small error. An angle of 350° read as $+350$ rather than -10 commands very nearly a full revolution of a joint standing next to a person. The conversion was isolated in one module and tested at the seams.

Calibration references must survive a power cycle without producing a step. The potentiometers report absolute values whose meaning depends on how the master was assembled, so a zero reference has to be captured and stored. If that reference is taken from a frame that failed validation, or if the commanded pose is referenced to the origin of the master's own frame rather than to where the operator's hand actually is, then the first command issued after start-up is a step of the master's entire tip vector.

Almost every failure in this system is silent. An arm that has stopped moving looks the same whether it is blocked for a good reason, blocked for a bad one, or receiving no data at all. A large part of the engineering effort went into making internal state legible from outside. One of the more uncomfortable findings of this work is that the measuring instruments failed more often, and more quietly, than the system being measured.

1.4 Contributions

Each contribution is stated with its maturity, because the distinction between implemented, verified in simulation, and validated on hardware is load-bearing throughout. **No contribution in this thesis was validated on physical hardware.** [Chapter 10](#) explains why.

1. **A graceful-degradation architecture for a body-worn master.** Per-channel health classification computed over distinct sensor updates, validation scoped to the channels the active mode consumes, run-time channel disabling with published capability loss, and a reduced-degree-of-freedom fallback that keeps position available on a single potentiometer. The cost of each rung is measured on recorded hardware data. *Implemented; measured against 20 430 recorded frames.*

2. **A reduced-degree-of-freedom command mapping** that takes direction from gravity and magnitude from the best-conditioned joints, with an anchor that re-latches on clutch engagement so indexing is unbounded and jump-free. *Implemented; verified in simulation.*
3. **A safety architecture whose blocking is observable.** Every mechanism able to stop the arm registers by name, must state how it clears, and expires into an explicit unknown state if the loop asserting it stops running. *Implemented; fourteen of fourteen injected faults handled.*
4. **A workspace characterisation of the mounted configuration,** including the result that the two arms share no reachable workspace under the as-built mount. This blocks inter-arm handover for geometric reasons that no control or planning change can address. A buildable mount modification that would resolve it is quantified. *Verified in simulation; [Chapter 9](#).*
5. **A jerk-limited, phase-synchronised motion generator** replacing the per-joint step limiter that constituted the entire motion generation for teleoperation. It removes 51.3 mm of unrequested Cartesian excursion on a platform whose grasp capture gate is 30 mm, and brings peak commanded joint speed from 12.53 times the manufacturer’s declared limit to 1.00. *Implemented; measured offline against the platform’s own forward kinematics.*
6. **A calibrate-then-plan stage** that measures the working surface and the objects on it instead of being given their coordinates, and hands the result to a planner. Six objects are recovered with every centre within 1 mm and every extent within 3 mm, and the resulting map is the first thing in this project that a motion planner has ever consulted. *Implemented; verified against the simulated scene and against mock depth.*
7. **A positioning error budget** assembled from separately measured contributions, giving 18.6 mm combined in quadrature and 33.4 mm added, against a 30 mm requirement, and identifying the camera mounting transform as the largest unmeasured term. *Measured; one term explicitly unmeasured and labelled.*
8. **A stage-by-stage instrumentation of the perception pipeline on four physical cameras.** Thirty-three stages per camera, each drawing itself on the frame it was given and reporting its quantities with units; 96 of 132 stages produced a result and none crashed. It establishes the surface fit on a real table at 1.43 mm root mean square with 74 % inlier support at working distance, separates a refusal from an absence by design, and found within the first hour a scene-camera orientation that three files had asserted and none had measured. *Measured on physical cameras; [Chapter 8](#).*
9. **A validation of the study’s own analysis against constructed ground truth.** The pre-registered procedure recovers the effect the design was powered for on 97.3 % of 20 000 simulated studies family-wise but on only 63.8 % for any named task once the correction is applied, while rejecting a true null at exactly the nominal rate. The design’s own power calculation reports neither number. *Measured on constructed data; [Section 10.8](#), [Chapter D](#).*
10. **A reproducible simulation campaign.** Every simulation this project can run, behind one command, in two declared tiers, with each stage’s status and duration recorded and the thesis’s tables generated from that record rather than transcribed from a terminal. The generator refuses to emit a table whose stage did not run. 16 stages, all of which ran; 13 927 inverse-kinematics calls at zero failures; and one stage that took four passes, because its own monotonicity control refused a result three times — twice for genuine under-sampling, and once because the control was comparing a lower bound against a boundary. The control was corrected rather than the number, and every reach it reports is unchanged by the correction. *Measured; [Section 7.1](#).*

11. **A verification methodology** requiring an instrument to recover a known answer before a surprising measurement taken with it is reported. *Applied throughout.*

The last arose from necessity. Across this project, results that looked like system failures turned out to be defects in the measuring tool on twenty-six separate occasions, and validating the instrument first caught errors that would otherwise have entered the record as findings. Two of them would have removed results that are now central to this thesis. It is the most transferable outcome of the work. [Section 10.7](#) states what it means for the measurements reported here, and [Chapter B](#) is the whole body of evidence rather than a selection.

1.5 Report structure

[Chapter 2](#) reviews the field by theme and ends with the gap this work addresses.

Chapters 3 to 6 are the methods: what was designed and built. [Chapter 3](#) documents the master arm — its mechanical chain, its electronics and its firmware. [Chapter 4](#) describes the delivered system: the command path, the degradation architecture, the six operating modes, the safety layer and the operating window. [Chapter 5](#) covers the perception stage that replaced declared coordinates with measured ones, the planner that consults it, and the natural-language route into the task layer. [Chapter 6](#) covers motion generation and what is known about transferring a joint command to a physical arm, and assembles the positioning and reaction budgets.

Chapters 7 to 9 are the results. [Chapter 7](#) reports the simulation campaign and every measurement made of the system, each labelled with the strength of evidence behind it. [Chapter 8](#) reports the perception pipeline exercised on four physical cameras, and is together with [Section 7.14](#) the only part of this report whose numbers come from hardware in the laboratory. [Chapter 9](#) presents the workspace characterisation separately, because it is a chain of measurements whose consequences determine the shape of the task set.

[Chapter 10](#) interprets those results, states what the measurements are worth ([Section 10.7](#)) and where they stop being trustworthy, and [Chapter 11](#) states what was achieved against the objectives above.

The appendices carry what a final report should record but not argue from. [Chapter A](#) is the route the design took, including the approaches that were abandoned. [Chapter B](#) is the case-by-case evidence behind the validity argument. [Chapters C and D](#) are the protocol for the two-person study and the analysis that would evaluate it: the study was designed in full and could not be run, for the reasons in [Section 10.8](#).

Chapter 2

Background

This chapter is organised by theme rather than chronology. Five bodies of work bear on the system built here: supernumerary limb design, the operator and wearer split, master interfaces for dissimilar kinematics, shared autonomy, and the measurement of embodiment and trust. The chapter ends by stating what remains open, which is narrower than it first appears.

2.1 Supernumerary robotic limbs

The field's founding demonstrations are Parietti and Asada's bracing limbs for aircraft fuselage assembly. Their limbs anchor against the environment to stabilise the wearer's posture, and the same group later extended the approach to whole-body support [27], which turns the wearer's body from a disturbance source into part of a closed chain [26, 25]. That framing matters because it identifies the central mechanical problem early: an arm mounted on a person exerts reaction forces into the person, and the person moves.

Reviews since have mapped the design space along consistent axes [35, 30, 33, 36]. Mounting location trades reach against the moment arm imposed on the wearer. Actuation trades bandwidth against worn mass. Control input ranges from residual body signals to explicit manual command. A more recent quantitative survey adds a useful observation, that the field reports embodiment far more often than it reports task performance, and rarely both [39].

Worn mass is the constraint that binds hardest and is discussed least. Each Gen3 masses 8.2 kg, so the pair with harness exceeds 17 kg. Simulation of the loads such systems impose on the wearer suggests the biomechanical cost is not negligible [23]. [Chapter 10](#) returns to what this implies for session design.

2.2 The operator and the wearer as different people

The configuration in this thesis has the arms worn by one person and commanded by another. Fusion is the direct predecessor. It mounted a robotic head and two arms on a worn frame and drove them from a remote operator [31]. Two qualifications must travel with that citation. Fusion's three modes, Direct, Induced and Enforced, are levels of *physical coupling* between the robot arms and the wearer's own arms, not levels of machine autonomy; in all three the robot decides nothing. And the publication is a two-page Emerging Technologies demonstration with no user study and no performance numbers. It establishes that the architecture existed. It cannot support any claim about how well it worked.

The evidence about how such systems are experienced comes from elsewhere. Recent work on spatial guidelines for supernumerary limbs in near-body interaction ran a concealed operator, varied the apparent autonomy,

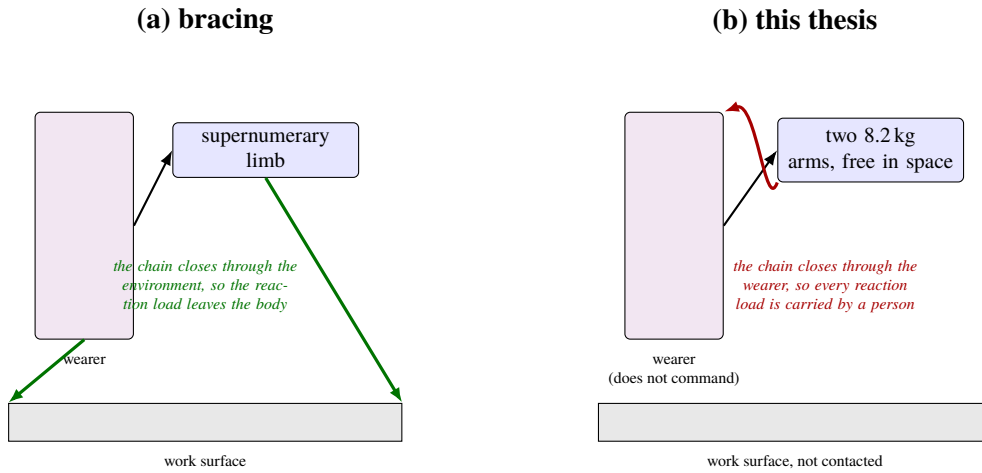


Figure 2.1: Two load paths, drawn from the mechanical argument rather than reproduced. In the bracing configuration that established the field [26] the limb anchors against the work surface, so the kinematic chain closes through the environment and the reaction load leaves the body. In the arrangement studied here the arms are free in space, so the chain closes through the wearer and every reaction load is carried by a person who is not the person commanding the motion. This difference, and not the control law, is what makes the wearer’s clearance the binding constraint of Chapter 9.

and measured the wearer’s physiological arousal and self-reported trust [40]. The finding runs against intuition: higher apparent autonomy made both worse, with skin conductance responses rising and trust ratings falling. Their autonomy was a hidden human and their wearer had no task of their own, so the result does not settle what happens when autonomy is real and the wearer is occupied. It does establish that the wearer’s experience is not a free variable.

Disturbance rejection for the two-person case is partly solved already. Zhang et al. compensate human-induced base motion for a supernumerary arm and report end-effector errors of (1.37 ± 0.58) mm with the human shoulder as a floating base [37]. What that work does not cover is the case where the disturbance originates in a nervous system the operator has no efference copy of. When wearer and operator are the same person, the base motion is partly predictable to the controller because the person commanding it also caused it. When they are different people, it is not.

2.3 Master interfaces for dissimilar kinematics

Where master and slave differ in scale or morphology, the mapping must be chosen rather than assumed. Guggenheim and Asada argue for miniature physical masters on the grounds that geometric similarity supplies feedback inherently, without a separate haptic display [10]. The argument is persuasive for similarity of *form*. It says nothing about similarity of *scale*, and scale is where the present system departs: a ratio of roughly three to one between master and slave reach means identical joint angles produce very different tip displacements.

Two mappings are available in principle. Joint-space correspondence sends master joint angles to slave joint angles directly. It preserves the arm’s posture, which matters when the posture itself must avoid something, and it requires every joint of the master to be measured accurately. Task-space correspondence maps tip displacement to end-effector displacement, decoupling the two chains at the cost of leaving the slave’s redundant degree of freedom unspecified. Chapter A records why this project moved from the first to the second.

Supplementary feedback is the other axis. Pinardi et al. review what additional sensory channels do for control

and embodiment in movement augmentation, and find the two do not move together [28]. Feedback that improves control does not reliably improve the sense of ownership, which is a caution against treating either as a proxy for the other.

2.4 Shared autonomy

The shared-autonomy literature addresses a different question from the teleoperation literature: not how to transmit the operator’s command, but how much of it to obey. Dragan and Srinivasa formalise the arbitration as policy blending, in which a predicted user goal and the user’s actual input are combined with a weight that depends on the confidence of the prediction [7]. The framework is explicit that blending too aggressively on a wrong prediction is worse than not blending at all, which is the property that governs the arbiter built here.

Javdani et al. recast the problem as a partially observable Markov decision process over the user’s goal and approximate the intractable solution with hindsight optimisation [16]. Their user study found that assistance which does not wait for confidence to be high lets operators finish faster with less input. The lesson taken here is narrower than the method: an assistance policy needs an explicit representation of its own uncertainty, and needs a defined behaviour for the case where the distribution over goals is flat.

Neither line of work was developed for a robot mounted on a second person. In both, the cost of assisting toward the wrong goal is a slower trial. Here it is a manipulator moving toward someone who did not command it.

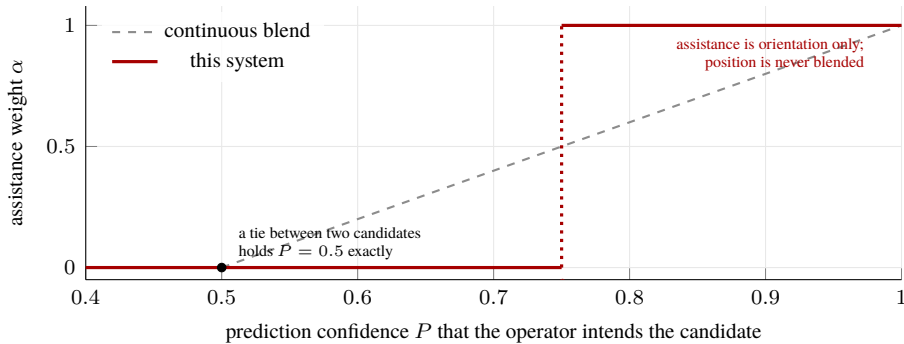


Figure 2.2: Arbitration as a function of prediction confidence, plotted from this system’s own constants. Policy blending in its original form [7] raises the assistance weight α continuously with confidence, which means that at low confidence the arm is already moving somewhere the operator did not ask for. The rule implemented here is a threshold: below θ the assistance weight is exactly zero, and a tie between two adjacent candidates holds $P = 0.5$ and is therefore never assisted. The distinction matters on this platform because the cost of assisting toward the wrong candidate is not a slower trial but a manipulator moving toward a person who did not command it.

2.5 Measuring embodiment, workload and trust

Body ownership over an artificial limb is measurable, and the instruments are long established. The rubber hand illusion showed that ownership can be induced over an object that is plainly not one’s hand [4], and Longo et al. decomposed the resulting experience psychometrically into components including ownership, agency and location [20]. The decomposition matters for a supernumerary limb because ownership and agency can move independently: an operator may feel authorship of a motion without feeling the arm is part of them, and a wearer

may feel neither while still bearing the consequences.

Workload is conventionally measured with the NASA Task Load Index [11], which separates mental, physical and temporal demand from effort, performance and frustration. For a two-person system the index has to be administered to both people, and the two will not load the same way. Physical exertion is better tracked with a Borg category-ratio scale [3], which is the appropriate instrument for the operator's uncompensated arm and for the wearer's carried mass.

Interface design has its own standard. ANSI/ISA-101.01 sets out a philosophy for operator displays in process automation [13], and its central principle transfers directly to a robot console: colour is a scarce alarm channel, so a normal state should be largely uncoloured rather than green. A display that paints five items red simultaneously has communicated the same amount as one that paints none.

2.6 The gap

The gap is narrower than a first reading suggests, and stating it precisely matters more than claiming it is wide.

Fusion established the operator and wearer architecture but measured nothing [31]. Shared autonomy has been shown to raise operator performance, in settings where the robot is not attached to anybody [7, 16]. Work on near-body interaction has shown that increasing apparent autonomy lowers the wearer's trust and raises their physiological arousal [40], in a setting where the autonomy was simulated by a concealed human and the wearer had no task.

So it is not true that nobody has looked at any of this. What has not been done is to measure both ends in one system: the operator's performance and the wearer's experience, under the same autonomy, in the same trial. Until that is done the exchange rate is unknown. If shared autonomy buys twenty per cent in completion time and costs a point of wearer trust, that may be a good trade or a bad one, and there is currently no basis for saying which.

The engineering contribution of this thesis is a system in which that question could be asked. The measurement itself is not made here, and Chapter 10 sets out what stands between the system as built and a session with two people in it.

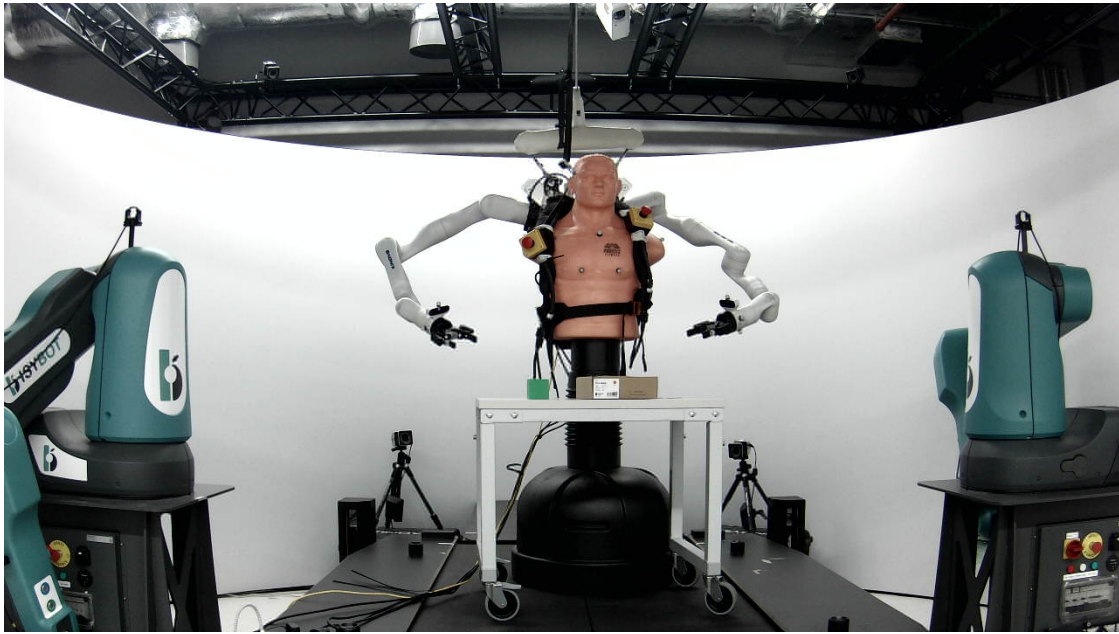


Figure 2.3: The platform as built, photographed by the scene camera during the vision session of 2026-08-30. Two Kinova Gen3 arms are carried on a backpack frame on a torso mannequin; the wearer's own arms are absent from this photograph because the frame is instrumented rather than worn during a bench session. The two teal columns are the laboratory's fixed manipulators and are not part of this system. This is a 1280×720 frame from the uncalibrated HD webcam of [Chapter 5](#), reproduced here because it is the only view that shows the mount, the arms and the work surface in one frame at the geometry every measurement in this thesis was taken at.

Chapter 3

The master arm

The master is a miniature mannequin reproducing the Gen3's joint sequence at roughly a third of its scale. The operator moves it with the fingers of one or both hands. This chapter documents it completely: the mechanical chain, the electronics, the firmware that turns fourteen analog channels and two inertial sensors into a serial frame, and the signal conditioning applied on the way. Source is at `firmware/master_arm/teensy_final.ino`.

3.1 Mechanical chain

Each arm carries seven potentiometer-instrumented joints in an alternating pattern: a roll about the preceding link's own axis, then a bend perpendicular to it, repeated. [Figure 3.1](#) gives the sequence with the measured link lengths, which sum to 0.272 m.

The alternation is not incidental to the sensing design. A roll joint moves the tip only through a downstream bend, because rotating a straight arm about its own axis translates the tip by zero. A bend moves the tip directly. That asymmetry is why the degradation ladder in [Chapter 7](#) is ordered the way it is, and why the loss of J2 or J4 costs reach while the loss of J3 or J5 costs almost nothing.

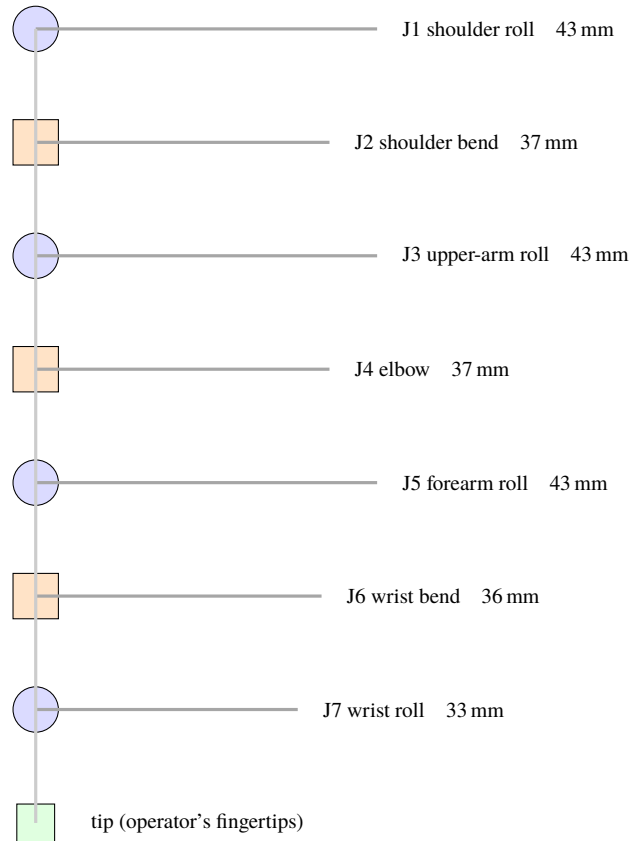
3.2 Electronics

A Teensy 4.1 reads every sensor and emits one text frame per cycle. The sensor complement is fourteen rotary potentiometers, two MPU-6050 inertial measurement units, two force-sensitive resistors and two momentary buttons. [Figure 3.3](#) gives the block diagram and the pin assignment; [Table 3.1](#) gives the assignment in tabular form.

Table 3.1: Pin assignment. Sixteen analog channels are required and sixteen are available once the two occupied by the default I²C bus are excluded.

Function	Count	Pins	Note
K1 (left) potentiometers	7	A0 A1 A2 A3 A6 A7 A8	no I ² C pins
K2 (right) potentiometers	7	A9 A10 A11 A12 A13 A16 A17	no I ² C pins
Force-sensitive resistors	2	A14, A15	left, right
Buttons	2	digital 2, 3	INPUT_PULLUP
I ² C (Wire)	2	pin 18 = A4, pin 19 = A5	excluded from analog use

Analog-capable pins on Teensy 4.1: 18. Reserved for I²C: 2. Remaining: **16**. Required: 14 + 2 = **16**.



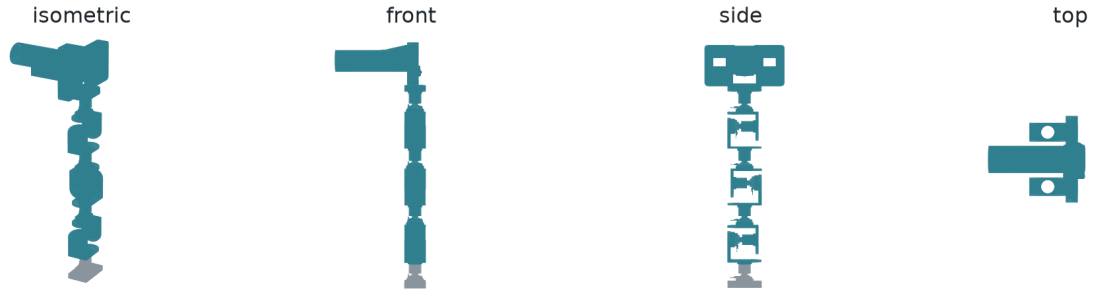
Total extension $\sum L_i = 0.272$ m, against the Gen3's 0.902 m reach. The ratio is 3.3:1, so no joint-to-joint mapping is available and the correspondence has to be made in task space.

○ roll about the preceding link's axis □ bend perpendicular to it.

The alternation matters for degradation: a *roll* moves the tip only through a downstream bend, so losing one costs little radial information, whereas losing a *bend* (J2 or J4) removes reach outright.

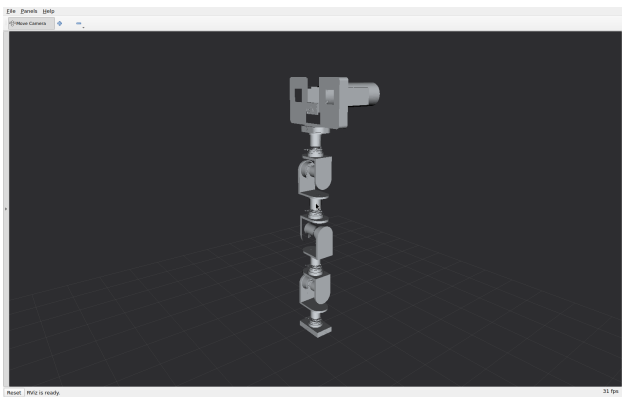
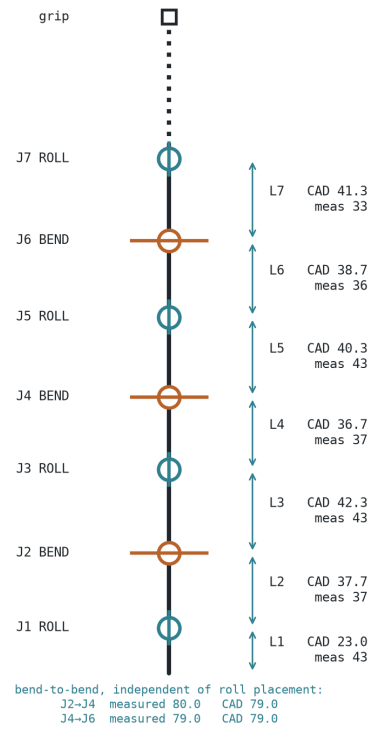
Figure 3.1: The master's seven joints with measured link lengths, drawn at all joints zero so the lengths read off directly.

Master arm, from CAD — zero configuration



(a) Four orthographic views of the assembly at the zero configuration, from the mechanical model.

Master arm chain — roll / bend alternation lengths in mm, CAD-inferred against measured



(b) The same geometry as a kinematic description, loaded and solved by the robot state publisher.

(c) Link lengths inferred from the model against the lengths measured on the built arm.

Figure 3.2: The master arm as designed and as described to the software. The mechanical model (a) carries seven potentiometer housings, a shaft coupling at each joint, a cable channel along each link, and a wrist platform for the inertial sensor and the force-sensitive resistor; the cable bundle crosses every rotating joint, which is where the channel failures of [Chapter 7](#) originate. Converting it to a kinematic description (b) is not a formality: [Section 3.6](#) shows that the model and the built arm disagree on link length by up to 9 mm (c), so the measured lengths and not the drawn ones are what the forward kinematics uses.

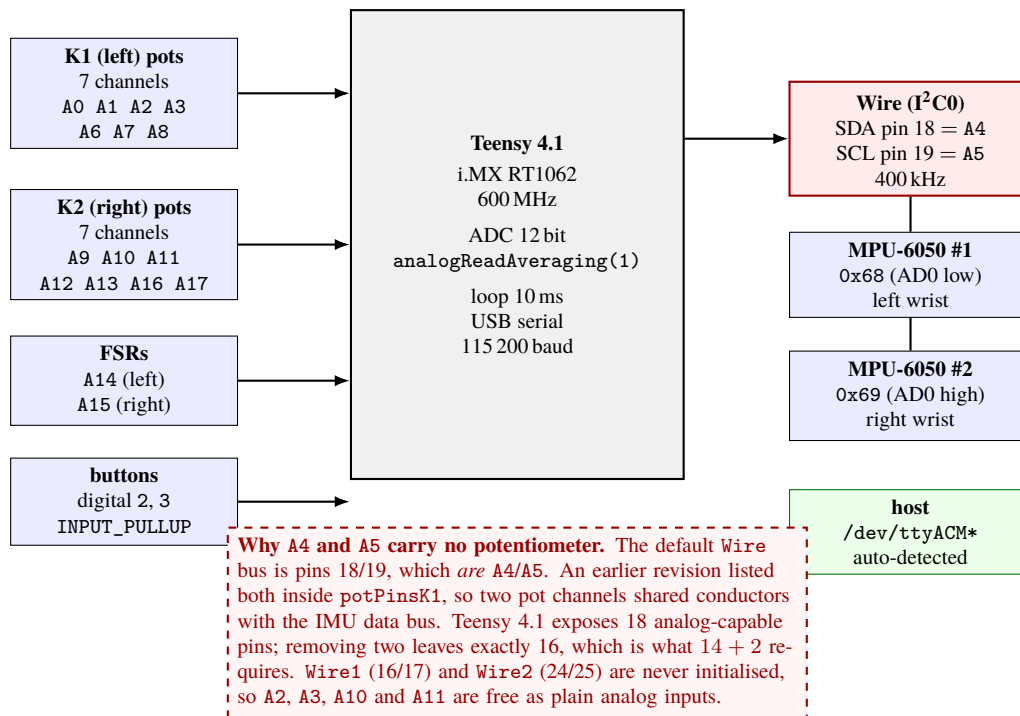


Figure 3.3: Electronics architecture and pin assignment. The call-out explains why two analog-capable pins carry no potentiometer.

3.2.1 The A4 and A5 collision

An earlier revision of the firmware listed A4 and A5 inside the left arm’s potentiometer array. Those two pins are not free. On the Teensy 4.1 the default Wire bus, which both inertial measurement units use, is physically pins 18 and 19, and those pins *are* A4 and A5. Two potentiometer channels were therefore sharing conductors with the inertial sensors’ data bus.

The failure mode this produces is not a clean one. Calling `analogRead()` on a pin that the I²C peripheral is driving reconfigures that pin, so the effect appears as intermittent bus corruption on the inertial sensors and as meaningless values on two potentiometer channels, with the severity depending on the phase relationship between the analog read and the bus transaction. Nothing about it looks like a pin conflict from the outside.

The resolution was to exclude both pins from analog use entirely. The arithmetic then works exactly: Teensy 4.1 exposes eighteen analog-capable pins, removing two leaves sixteen, and sixteen is what fourteen potentiometers plus two force sensors require. There is no spare, which is worth recording, because any future channel needs either a multiplexer or a second board.

The two secondary I²C buses are safe to use as plain analog inputs and the design does so [29]. Wire1 occupies pins 16 and 17 (A2, A3) and Wire2 occupies pins 24 and 25 (A10, A11), but neither peripheral is ever initialised: the firmware calls only Wire. A bus that is never begun does not claim its pins, so all four remain ordinary analog inputs and all four carry potentiometers.

That headroom is the obvious route to a third and fourth inertial sensor. The MPU-6050 family offers exactly two addresses, 0x68 and 0x69, selected by the AD0 line, and both are already taken by the two wrist sensors. A third sensor cannot join the same bus. Bringing up Wire1 on pins 16 and 17 would carry another pair, at the cost of the two potentiometer channels currently on A2 and A3. [Chapter 7](#) quantifies why that trade may be worth making.

3.3 The inertial measurement driver

The firmware includes a local MPU6050.h rather than an external library. The reason is build portability: a sketch that depends only on Wire compiles on any Arduino-compatible toolchain without a library manager resolving versions, and the driver's behaviour is then fixed by the source in the repository rather than by whatever version happens to be installed.

The driver header itself is not in the repository: it was not among the firmware files handed over. The interface described below is therefore established from the call sites in the sketch, which are certain, and the register sequence is as documented for the device rather than as read from the source. Nothing in this section depends on a line of that file that the sketch does not call.

Three entry points are used. `initialize()` wakes the device, `testConnection()` confirms it is present and addressed correctly, and `getMotion6()` returns six signed 16-bit values.

Waking the device. The MPU-6050 powers up in sleep, with the SLEEP bit of PWR_MGMT_1 (register 0x6B) set. Until that bit is cleared the sensor registers read as zero [15]. Clearing it is the whole of initialisation for this application, since the power-on defaults for range and filtering are the ones wanted.

Confirming presence. WHO_AM_I (register 0x75) returns a fixed identity value. Reading it distinguishes three states that otherwise look identical from the outside: a sensor that is present and responding, a sensor that is absent so the bus read returns nothing, and a sensor that is present but strapped to the other address. The firmware prints the verdict per sensor at start-up, and that line is the first thing to check when one arm reports no orientation.

Reading the measurement. All six axes plus temperature occupy fourteen consecutive registers beginning at ACCEL_XOUT_H (0x3B). The driver reads all fourteen in a single burst. This is not only a bandwidth optimisation. The device latches its output registers for the duration of a burst read, so a single transaction returns six axes from one sampling instant, whereas six separate reads can straddle an internal update and return a mixture of two. For an attitude estimate built from the ratio between axes, a mixed sample is a wrong attitude rather than a noisy one.

Each axis arrives high byte first and is reassembled as a signed 16-bit value. The two bytes of temperature sit between the accelerometer and gyroscope blocks and are discarded here.

3.3.1 Scale factors

At the power-on defaults the accelerometer is at ± 2 g full scale and the gyroscope at $\pm 250^\circ/\text{s}$, giving sensitivities of 16 384 LSB/g and 131 LSB/ $^\circ/\text{s}$ [14]. The firmware never changes the range registers, so these are the values that apply.

Table 3.2: Inertial sensor scaling at power-on defaults.

Quantity	Full scale	Sensitivity	Resolution
Acceleration	± 2 g	16 384 LSB/g	61 μg per LSB
Angular rate	$\pm 250^\circ/\text{s}$	131 LSB/ $^\circ/\text{s}$	$7.6 \times 10^{-3}^\circ/\text{s}$ per LSB

A value of 8192 LSB/g is widely copied in example code and is wrong for this configuration. It corresponds to the ± 4 g range, which the device only enters if ACCEL_CONFIG is written. Used against the default range it halves

every reported acceleration. That error does not announce itself in the elevation estimate, because elevation is computed from the direction of the acceleration vector and a uniform scale error cancels in a normalised vector. It announces itself in the accelerometer gate, which compares the vector’s magnitude against 1 g to decide whether the sensor is measuring gravity alone: with the wrong scale, a stationary sensor reports 0.5 g and the gate rejects every frame. The failure appears as a master that never produces a valid orientation, which looks like a dead sensor.

3.4 Signal conditioning

3.4.1 Potentiometers

Each channel is read at 12-bit resolution with hardware averaging disabled, then passed through an exponential moving average implemented in integer arithmetic:

$$e_{n+1} = \frac{\alpha r_n + (256 - \alpha) e_n}{256}, \quad \alpha = 64, \quad (3.1)$$

which is a smoothing factor of 0.25 per sample, realised as a shift rather than a division. Integer arithmetic was chosen because the loop runs at 100 Hz across sixteen channels and the Teensy’s floating-point unit is not needed for a filter whose input is a 12-bit integer.

The filtered count maps to degrees by a clamped linear transform:

$$\theta = 360^\circ \cdot \text{clamp}\left(\frac{a - 50}{4045 - 50}, 0, 1\right). \quad (3.2)$$

The clamp limits are the substantive part. A potentiometer wiper never reaches the extreme of its track, and the ends of the ADC range are where the reading is least linear, so the transform discards the outer 50 counts at the bottom and 50 at the top. The remaining 3995 counts span 360° , giving 0.090° per count.

3.4.2 A start-up transient in the filter initialisation

[Equation \(3.1\)](#) operates in raw ADC units: the steady state of the recurrence for a constant input r is $e = r$. The initialisation in `setup()` does not match. It seeds each accumulator with `analogRead(pin) << 8`, which is 256 times the raw reading, a value consistent with a fixed-point formulation the filter does not use.

Simulating the recurrence from that seed gives the transient in [Table 3.3](#). Every channel reads a railed 360° for roughly the first 190 ms after reset and takes 440 ms to settle.

Table 3.3: Filter output after reset, simulated from [Eq. \(3.1\)](#) with a constant input of 2000 counts and the seed used in `setup()`.

Cycle	Accumulator	Reported angle
1	384 500	360.0° (railed)
10	30 719	360.0° (railed)
20	3616	321.3°
25	2382	210.1°
30	2090	183.8°
44	2000	180.0° (settled)

Two consequences follow, and the second is the one that has cost time. Any calibration capture taken within half a second of a board reset records a railed or settling value rather than a joint angle. And re-opening the serial port resets the board, so a calibration script that opens the port and immediately samples is guaranteed to capture the transient. This is the same window in which the inertial sensors are also settling, and the effects are indistinguishable at the ROS level: both appear as a channel reporting a plausible but wrong constant.

The existing calibration procedure already requires a settled stream before sampling ([Fig. 4.11](#)), for reasons that were diagnosed from the gyroscope alone. The analysis here shows the potentiometers need the same protection for a different reason. Correcting the seed to `analogRead(pin)` removes the transient entirely and changes nothing else, since the steady-state behaviour of [Eq. \(3.1\)](#) is unaffected. This settling figure is arithmetic on the shipped recurrence rather than a measurement of the board. It is stated as such: the correction is derived from the filter as written, and confirming it requires timing the settling window on hardware, which is listed among the outstanding hardware measurements in [Chapter 10](#).

3.4.3 A candidate explanation for the observed update rate

The frame is emitted at 100 Hz, but recorded captures show each potentiometer producing a new *distinct* value at only 14.7 to 17 Hz, so 82 to 88 % of consecutive rows repeat. That aliasing invalidated every per-row statistic computed before it was noticed ([Chapter 7](#)).

The firmware offers a mechanism. Angles are printed with one decimal place, and 0.1° is 1.11 ADC counts. A smoothing factor of 0.25 moves the filter only a quarter of the way to a new input in one cycle, so a slow joint motion produces sub-count filter steps and the printed value changes only when the accumulated motion crosses a printing boundary. The effect is quantisation in the output format compounded with the filter, not a slow sensor.

This explanation is consistent with the firmware but has not been isolated, and is offered as a candidate rather than as a finding. It is directly testable without the board: `scripts/virtual_teeny.py` streams frames in the same wire format from known ground truth, so sweeping a channel at a known rate and counting distinct printed values against the predicted boundary crossings would settle it. If it is confirmed, printing two decimal places or raising the smoothing coefficient recovers the full rate.

3.4.4 Force-sensitive resistors

Both force sensors are read on the same 12-bit ADC as the potentiometers and transmitted as integers. Measured pad values are 6 to 191 counts unsqueezed and 3603 (left) and 3842 (right) at full squeeze, with cross-talk of 3.7 % and 0.4 % under load only, so no mixing matrix is required.

The firmware as handed over and the design as described do not agree on the force channels. The design applies a floating-point exponential average, so those channels would carry decimal precision, and the parser on the software side accepts a decimal in both fields. The firmware in hand transmits an integer and applies no filter to either channel. Either the delivered build is not the one described or the change was never made; the discrepancy is recorded rather than resolved, because the gripper mapping does not depend on it: the mapping applies its own deadband and hysteresis downstream.

3.4.5 Buttons

Both buttons are wired as two-wire switches to ground with `INPUT_PULLUP`, so the pin reads high when open and low when pressed and no external resistor is needed. The firmware converts each press into a latching toggle:

a transition is accepted only if the raw level differs from the last accepted level and at least 40 ms have passed since the last accepted transition, and the output state flips only on the falling edge.

Debouncing at the source matters more here than it usually would, because the button is the clutch. A bounce that registers as two transitions toggles the clutch twice, which disengages and re-engages it within a few milliseconds and re-latches the anchor at whatever pose the master happened to be in. The downstream clutch logic adds its own debounce and a quasi-static gate for the same reason.

3.5 The serial frame

One frame is emitted per cycle, at 100 Hz, as ASCII text terminated by a newline. Table 3.4 gives the fields in order. The format is deliberately human-readable: a frame can be read directly from a terminal to tell a wiring fault from a firmware fault without any host software running.

Table 3.4: Serial frame, in transmission order. Fields are comma-separated; the frame ends with a newline. Buffer is 460 bytes against a typical frame of roughly 285.

Field	Count	Format	Meaning
k1j1...k1j7	7	%.1f	left arm joint angles, 0 to 360°
k2j1...k2j7	7	%.1f	right arm joint angles, 0 to 360°
K1IMU	6	%d × 6	left $a_x a_y a_z g_x g_y g_z$, raw LSB
K2IMU	6	%d × 6	right, same order
fsr1, fsr2	2	%d	grip force, raw counts
btn1, btn2	2	%d	latched toggle state, 0 or 1

Three properties of this format are relied on downstream. Inertial values are transmitted *raw*, in LSB, so the scale factors are applied on the host and can be corrected without reflashing. The token k1j1 at the start of every frame is what the host’s port auto-detection sniffs for, which is how the board is found when it moves between /dev/ttyACM0 and /dev/ttyACM1 across re-attachments. And the buttons carry a latched state rather than an instantaneous level, so a host that misses frames cannot miss a press.

3.6 The CAD is not the kinematic source

This section exists so that nobody later mistakes the mechanical drawings for the model the software uses. Three statements, each checked rather than assumed.

The CAD carries no articulated joints. Both STEP files declare the AUTOMOTIVE_DESIGN (AP214) schema, which *is* capable of carrying kinematic structure. Searching them for every entity that would express it, KINEMATIC_JOINT, REVOLUTE_PAIR, PRISMATIC_PAIR, KINEMATIC_LINK and MECHANISM, returns zero occurrences in either file. They are solid geometry for manufacture, drawn for three-dimensional printing, and they describe one fixed configuration.

No robot description derives from them. No URDF or xacro anywhere in the workspace references PotArm.step, backpack.step, or any .step file. More strongly: **the master arm has no URDF at all.** It is not modelled as a robot description in any form.

The kinematics are seven measured constants. What the software actually uses is a single line, `srl_teleop/master_calibration.py`:

```
LINK_LENGTHS = [0.043, 0.037, 0.043, 0.037, 0.043, 0.036, 0.033]
```

Those values did not come from the CAD. They originate in `basic_control/kinematics.h` from the prior work ([Chapter A](#)), where they are declared as “physical link lengths (metres), shaft-center to shaft-center, measured directly on the physical master arm”. The chain is measured, not designed.

Why this matters, and why the agreement does not change it. [Chapter 7](#) reports that the CAD’s joint-module pitch agrees with the forward-kinematic link lengths to 1.2 %. That is a genuine cross-check and it is worth having, because until it was made those seven numbers had never been compared against anything. It is emphatically **not** a licence to treat the CAD as the source: the agreement was established by measuring the CAD *against* the constants, and the constants remain the definition. If the two are ever found to disagree, the question of which is correct is open and must be settled on the physical arm, not by preferring whichever artefact is more convenient.

A successor tempted to regenerate the kinematics from the CAD should note that it cannot be done: the geometry gives centre-to-centre pitch for a *pair* of joints and does not distinguish the 43 mm roll from the 37 mm bend within it.

3.7 Summary

The master supplies fourteen joint angles, two three-axis accelerations, two three-axis angular rates, two grip forces and two latched buttons, at 100 Hz over USB serial. Of that, the position pipeline in [Chapter 4](#) consumes three or four potentiometer channels per arm and one acceleration vector. The remainder is either used for orientation, used for the gripper and clutch, or held in reserve against the loss of a channel that is being used.

Chapter 4

System as built

This chapter describes the delivered system: how it is decomposed into packages, how a master pose becomes an arm command, what happens when sensing degrades, and what can stop the arm. [Chapter 3](#) covered the master hardware and firmware; this chapter starts where the serial frame arrives.

4.1 Architecture

The workspace is six packages with a one-way dependency graph ([Fig. 4.1](#)). The direction is a design constraint rather than a convention. Direct teleoperation is the baseline condition of every planned experiment, so if the teleoperation package imported anything from the autonomy package the comparison would be circular. This was verified two ways: no such import exists anywhere in the source, and the inverse-kinematics follower still reaches its running state with the autonomy and perception packages physically deleted from the install tree.

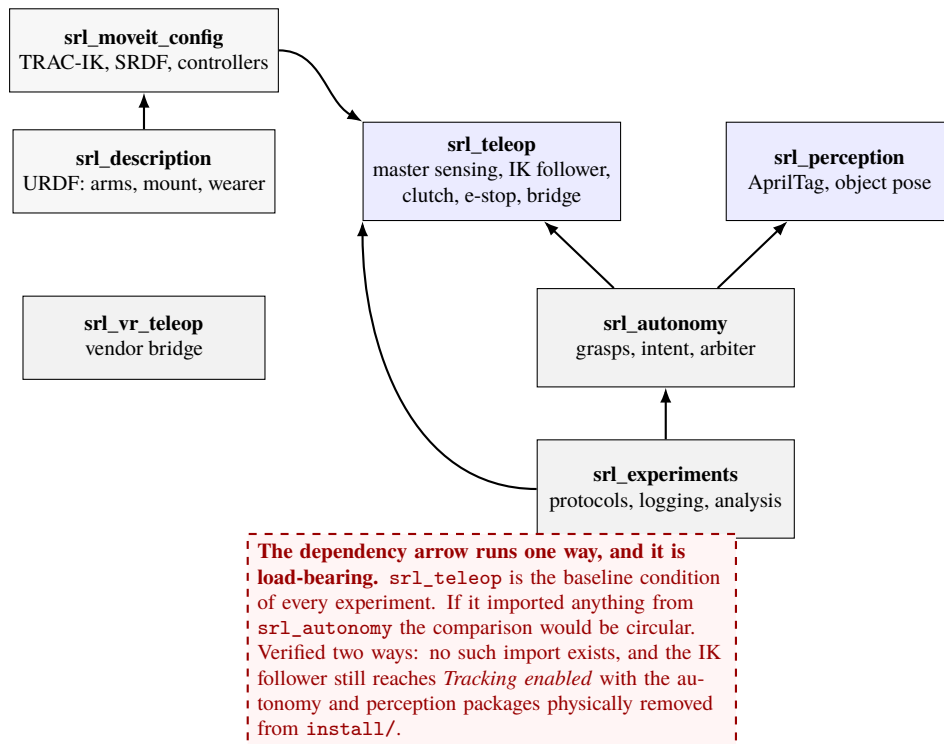


Figure 4.1: Package decomposition and the one-way dependency graph.

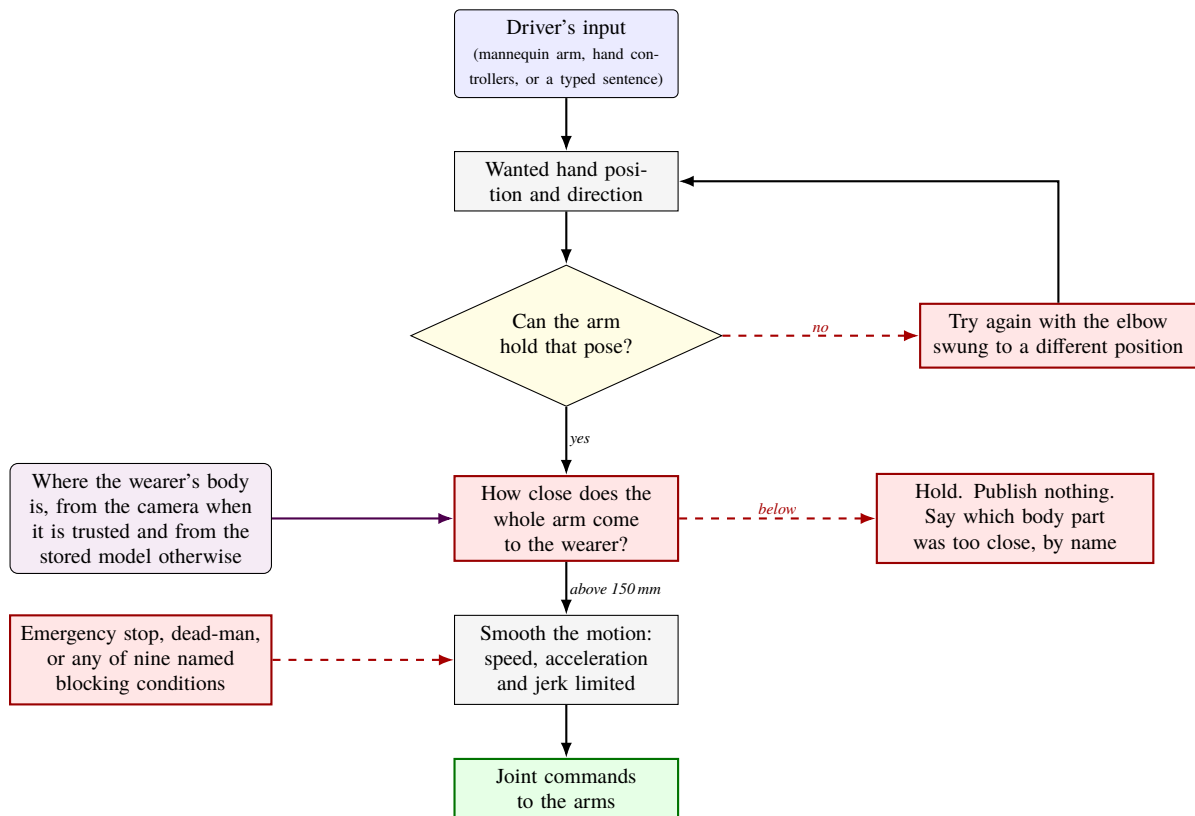


Figure 4.2: The path every commanded pose takes, in any mode. Red stages can stop the arm and the green stage is the only one that moves it. The two that matter most are the redundancy re-seed, which recovers poses a single solve would drop, and the clearance stage, which is measured geometrically against the wearer rather than taken from the semantic collision description. The per-mode diagrams later in this chapter expand the top of this path; the bottom of it is identical in all of them.

4.2 From master pose to arm command

4.2.1 Spherical decomposition

The commanded position is built in spherical form about the master's shoulder rather than by forward kinematics from seven joint angles. [Chapter A](#) gives the measurements that forced the change. Each component comes from whichever sensor observes it most directly:

$$\phi = \arcsin\left(\hat{a} \cdot \frac{\mathbf{g}}{\|\mathbf{g}\|}\right) \quad \text{elevation, from the gravity vector} \quad (4.1)$$

$$\psi = \theta_1 - \theta_1^0 \quad \text{azimuth, from the shoulder roll} \quad (4.2)$$

$$r = \|\text{FK}(\theta_1, \theta_2, \theta_3, \theta_4, 0, 0, 0)\| \quad \text{radius, magnitude only} \quad (4.3)$$

Here $\hat{a}$ is the arm's long axis expressed in the sensor's own frame, captured once with the master hanging at rest, and $\mathbf{g}$ is the measured acceleration.

Three properties follow, and each addresses a specific failure of the previous formulation. [Equation \(4.1\)](#) is mount-independent, because $\hat{a}$ is captured through the same mounting it is later used with, and drift-free, because it integrates nothing. [Equation \(4.2\)](#) depends on one directly measured zero-referenced quantity rather than on four compounded ones. [Equation \(4.3\)](#) takes a magnitude, which is dominated by the bend joints and insensitive to errors in the rolls, so a dead roll channel can be passed through as zero without corrupting it.

The sign convention in [Eq. \(4.1\)](#) is asserted at capture time: hanging is -90° , horizontal is 0° , vertical is 90° . The specification originally called for a negation in both $\hat{a}$ and the elevation expression, which compound and place the resting pose at 90° , inverting the vertical axis alone. Only one negation is correct.

4.2.2 Anchoring, scaling and indexing

The commanded end-effector position is

$$\mathbf{p} = \mathbf{p}_{\text{anchor}} + s(\mathbf{d} - \mathbf{d}_{\text{ref}}), \quad (4.4)$$

with $\mathbf{d}$ the master tip in the robot frame, $\mathbf{d}_{\text{ref}}$ the reference latched at clutch engagement, and s a per-arm scale.

The reference is latched from the first *valid* frame after boot, at the master's actual pose. An earlier version initialised it to the origin of the master's frame, which made the first command $\mathbf{p}_{\text{anchor}} + s\mathbf{d}$: a step of the master's entire tip vector, roughly 0.22 m, in whatever direction the master happened to be lying. The only master pose that did not produce a jump was $\mathbf{d} = 0$, which is why the procedure previously required the operator to hold a calibrated neutral before start-up. Requiring validity is not incidental. Latching from a frame that failed validation offsets every subsequent command.

Scale is re-read every cycle so it can be retuned live, and a change re-bases the anchor so the commanded pose stays continuous:

$$\mathbf{p}_{\text{anchor}} += (s_{\text{old}} - s_{\text{new}})(\mathbf{d} - \mathbf{d}_{\text{ref}}). \quad (4.5)$$

Without [Eq. \(4.5\)](#) a scale change displaces the arm by $(s_{\text{new}} - s_{\text{old}})$ times the current displacement, which is

largest exactly when the operator is furthest from the engagement point.

Releasing the clutch freezes the command; re-engaging re-latches $\mathbf{d}_{\text{ref}}$ at the master's new pose. Successive cycles therefore accumulate, and the reachable set is bounded by the robot rather than by the master's range. Engagement is gated on the master being quasi-static, because latching a reference while the master is moving captures a reference that is already stale.

4.3 Graceful degradation

4.3.1 Channel health

Each channel carries a verdict computed from its own recent history, over *distinct* sensor updates rather than over transmitted rows. The distinction is not pedantic: at the frame rate roughly 82 to 88 % of consecutive rows repeat, so any statistic computed across rows is dominated by duplicates. Every channel statistic computed that way returned a median absolute difference of exactly zero, which was read for a long time as evidence of dead channels.

Four verdicts are assigned, in order:

Dead. Circular range below 5° , or dropouts above 90 %.

Incoherent. More than 5 % of updates jump by more than 60° , which at the observed update rate is roughly $2500^\circ/\text{s}$.

Intermittent. Dropouts above 2 %.

Alive. Coherent, range at least 20° , dropouts at most 2 %.

The incoherent verdict is the one that did not exist in earlier versions and is the one that matters. A failing potentiometer does not go quiet. It returns values spanning a wide interval which are not a trajectory, with consecutive updates unrelated to one another. Range alone calls that healthy, which is how a broken channel keeps being trusted.

Range must be computed as the smallest arc containing every sample. Unwrapping and subtracting is invalid for a rotary sensor, because across a dropout the unwrap cannot know how far the joint moved and the accumulated offset runs away; that method reported $314\,363^\circ$ on a 360° sensor. Samples outside $[0, 360]$ are rejected first, since a small fraction are serial parse glitches arriving in bursts.

4.3.2 Mode-scoped validation

Validation rejects a frame only on the channels the active position mode consumes. In spherical mode those are joints one, two and four. A dead joint three, five, six or seven cannot reject a frame, which is the entire purpose: the arm keeps working with four of its seven channels gone.

A rejected frame holds the last good value rather than substituting a fabricated one. The cost is staleness rather than error, and it is charged honestly: a burst of dropouts freezes the command for the duration of the burst, which flattens measured displacement and adds apparent lag.

4.3.3 The degradation ladder

Figure 4.3 gives the measured cost of each reduction, against 20 430 recorded frames. Two orderings in it are not intuitive and both are mechanical rather than statistical.

channels retained	mean tip error	capability
4 pots	0 (reference)	full position and reach
3 pots, no J3	41.6 mm	roll lost, reach intact
2 pots, no J2 J3	29.2 mm	shoulder lost
2 pots, no J3 J4	66.1 mm	elbow lost, reach degraded
1 pot + 2nd IMU	63.8 mm	elbow from IMU
0 pots	n/a	POSITION UNAVAILABLE

Measured on 20 430 recorded frames from the left arm, against the shipped four-potentiometer reach. The error is charged to *reach magnitude* only, because in spherical form direction comes from the inertial measurement unit; scoring raw seven-joint forward-kinematic error instead would charge each reduction for a direction error the pipeline never incurs.

Two orderings in this table are not intuitive and both are mechanical. Dropping J3 costs 41.6 mm on the left arm and 4.5 mm on the right, because a roll carries no radial information on its own. And dropping J2 alone costs *less* than dropping J3 and J4 together, because the elbow dominates reach and the shoulder does not.

The last rung is the design decision that matters. With no well-conditioned channels the node reports position *unavailable* rather than extrapolating one, because a fabricated position on a manipulator bolted to a person is the worst available failure.

Figure 4.3: Capability against retained channels, with the cost of each rung measured on recorded hardware data.

The last rung is the design decision. With no well-conditioned channels the node reports position unavailable and publishes that state, rather than extrapolating a position from what remains. A fabricated position on a manipulator bolted to a person is the worst available outcome, and it is indistinguishable downstream from a real one unless the distinction is made explicit at the source.

4.3.4 Reduced-degree-of-freedom fallback

When both bend joints of an arm are unavailable, the forward-kinematic magnitude in Eq. (4.3) becomes constant and the arm's commanded set collapses from a shell of finite thickness to a surface. Elevation and azimuth are unaffected, because they are angular and come from elsewhere.

The fallback restores the missing dimension on a channel that is alive, coherent, and not otherwise consumed. Deflection of the wrist roll past a deadband commands a *rate* of change of radius rather than an absolute radius. Rate rather than position is necessary because that joint's usable range is small, so an absolute mapping would give coarse resolution over the whole radial extent and would require the operator to hold a wrist angle to hold a radius.

The channel choice is constrained. The gripper is on the force sensors and the clutch is on the buttons, so both are unavailable without conflict. The wrist roll conflicts with nothing, which means the operator can extend while

gripping and while indexing.

4.4 Control modes

Operating modes are defined in a single registry, in the package that depends on nothing else, so every other package can read it without inverting the dependency graph. Table 4.1 summarises the delivered set. The numbering is 01, 02, 03, 04 and 06: there is no 05, and the gap is preserved rather than closed because recorded data carries these labels and renumbering would silently re-attribute it. The four that are implemented and distinct in their data path are given their own diagrams below.

Table 4.1: Operating modes. Autonomy runs slower than teleoperation by design: nobody is holding a master, so the only bound on a wrong motion is how long it takes to happen.

Mode	Input	Robot decides	$v_{\max}$ (rad/s)
01 direct, mannequin	master arm	nothing	0.60
02 direct, controllers	tracked controllers	nothing	0.60
03 shared autonomy	master, vision	wrist orientation	0.60
04 shared autonomy, controllers	controllers, vision	wrist orientation	0.60
06 full autonomy	typed or spoken goal	target, approach, grasp, transport, place	0.15

Orientation assist is a stub and is marked as one. It would supply wrist orientation under an orientation constraint, and the inverse-kinematics plugin in use ignores the constraint field entirely: measured over 270 solver calls, success was identical to the digit with and without the constraint. It needs a constraint-aware plugin or explicit yaw sampling.

One safety stack serves all six. Every mode transition asserts the invariant, and dropping any one of the six shared mechanisms is refused in every mode rather than being available as a per-mode option.

4.4.1 The transport, and why it changed

An immersive browser session requires a secure context. The first implementation obtained one the cheap way, by tunnelling the socket over USB with `adb reverse` so that the client addressed `ws://127.0.0.1`: a loopback address is a secure origin by definition, which removes the certificate, the firewall exception, and the prospect of an operator dismissing a browser warning from inside a headset they are wearing. That route was abandoned because it requires `adb`, which requires developer mode on the headset, which cannot be assumed on a borrowed device.

The delivered route serves the client page and the `wss` socket from *one* origin, so a single certificate exception covers both, and the certificate carries the host's address in its `subjectAltName`. Mirrored networking under the Windows Subsystem for Linux makes the Linux host's address the Windows host's, so the address in the certificate is the address the headset dials. The path was verified end to end on 2026-08-19 and requires no `adb`, no sideloading and no account on the headset. Bringing it up for the first time exposed two defects that no unit test would have found: the bridge node had never started at all, because an attribute named `clients` shadowed a read-only property of the base class; and its axis map had been written for the conventional robotics frame rather than for this project's, so a hand moved 3 m forward commanded 3 m to the wearer's right.

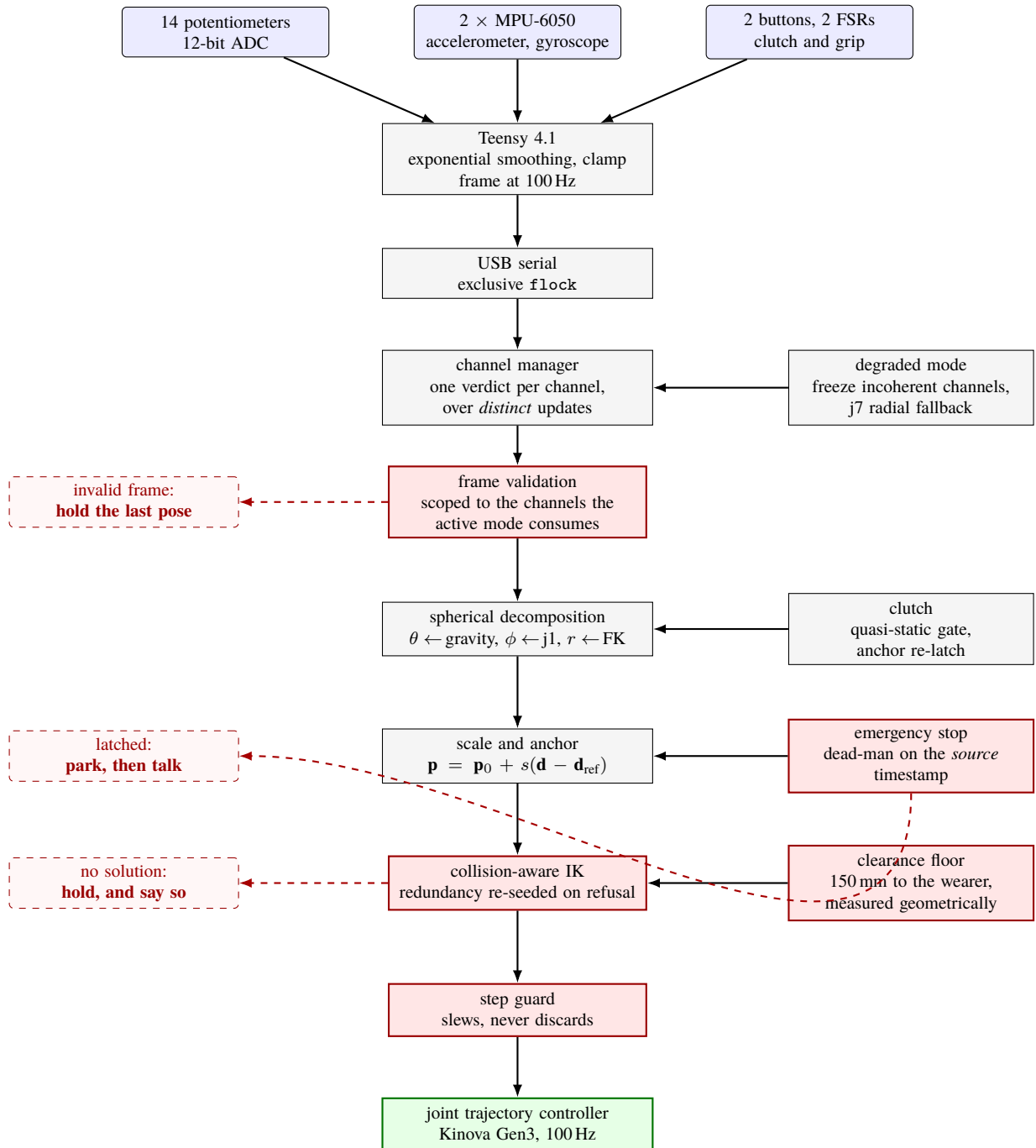


Figure 4.4: Mode 01, direct teleoperation from the mannequin master. The centre column is the command spine; guards and side inputs enter from the right; a dashed red arrow leaves to the left for each way a stage can refuse, and names what the arm does instead. Blue is sensing, grey is processing, red is a stage that can stop the arm, and green commands motion. The same three-column grid is used for Figs. 4.5 to 4.7, so the four paths can be read against one another stage by stage.

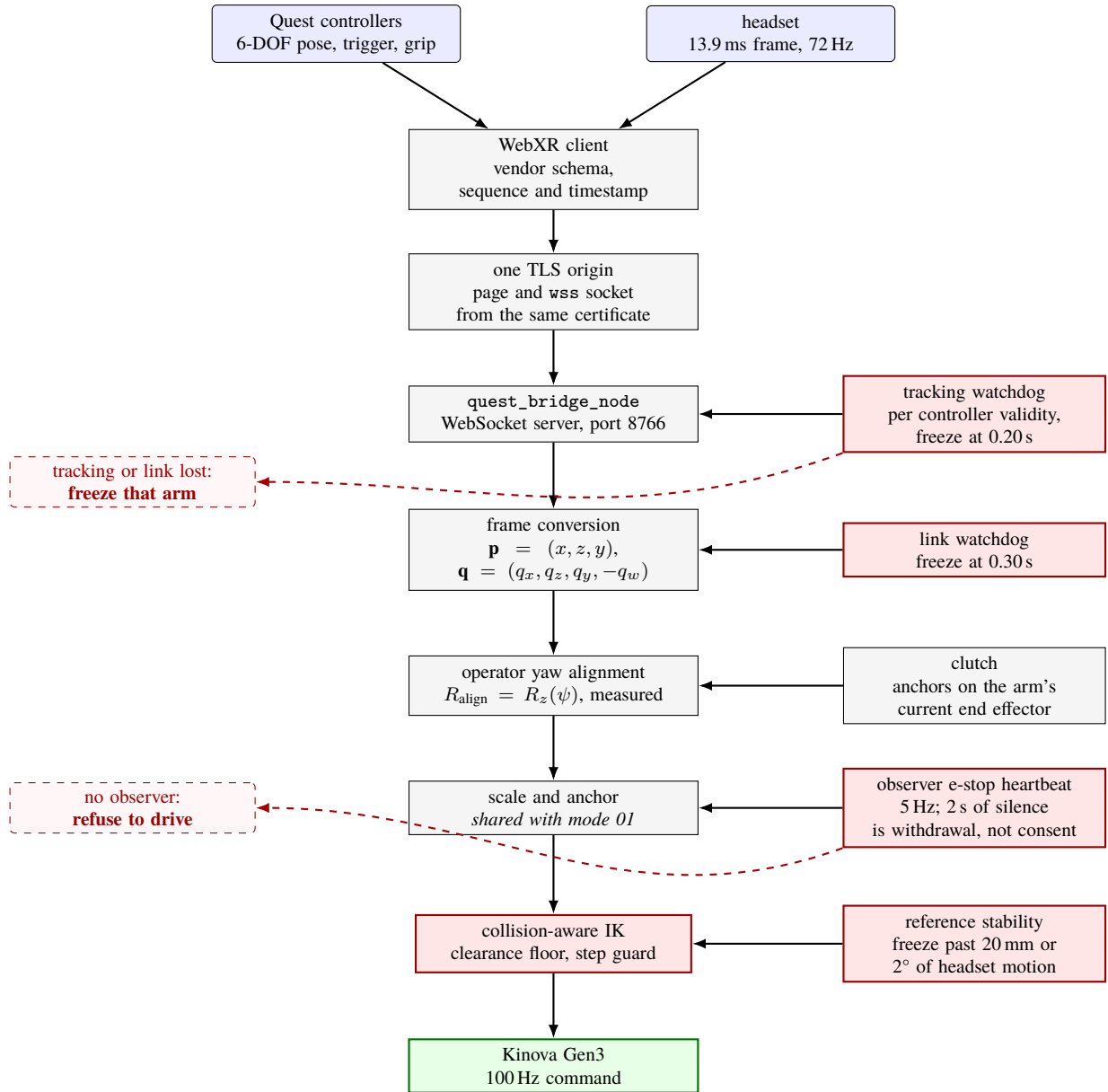


Figure 4.5: Mode 02, direct teleoperation from a Quest headset. The headset supplies six degrees of freedom directly, so neither the spherical decomposition nor the channel-health layer of Fig. 4.4 appears; everything from *scale and anchor* downwards is the same code. Four guards are specific to this path, and three of the four were added after a measurement showed the fault they cover was undetectable (Section 4.4.2).

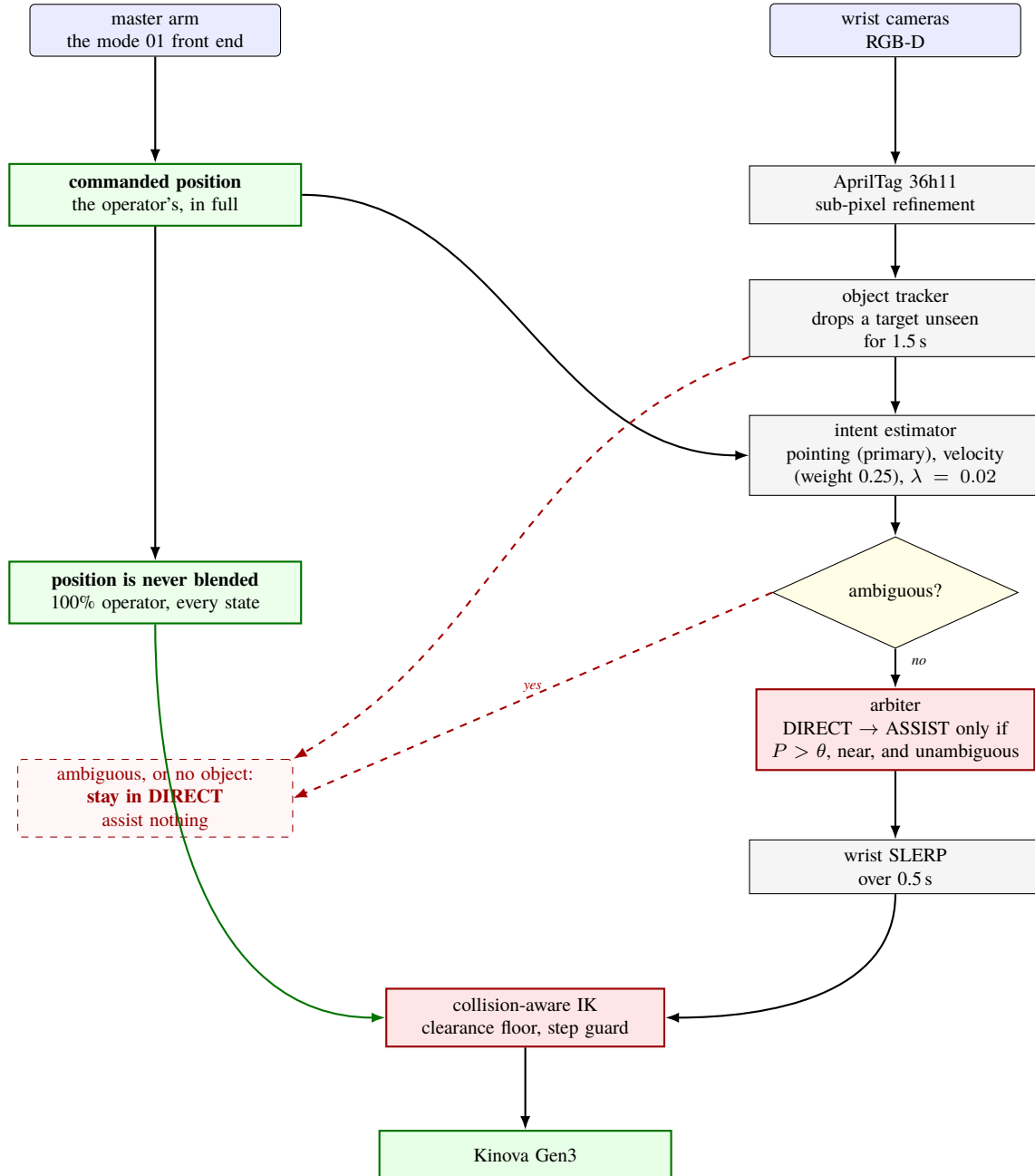


Figure 4.6: Mode 04, shared autonomy. The green spine carries commanded position and does not pass through the autonomy stack, so no failure of intent estimation, object tracking or grasp synthesis can move the arm; the autonomy branch reaches only the orientation blend. A tie between two adjacent candidates holds $P = 0.5$ exactly and the arbiter remains in DIRECT, because assisting toward the wrong one of two candidates is worse than not assisting.

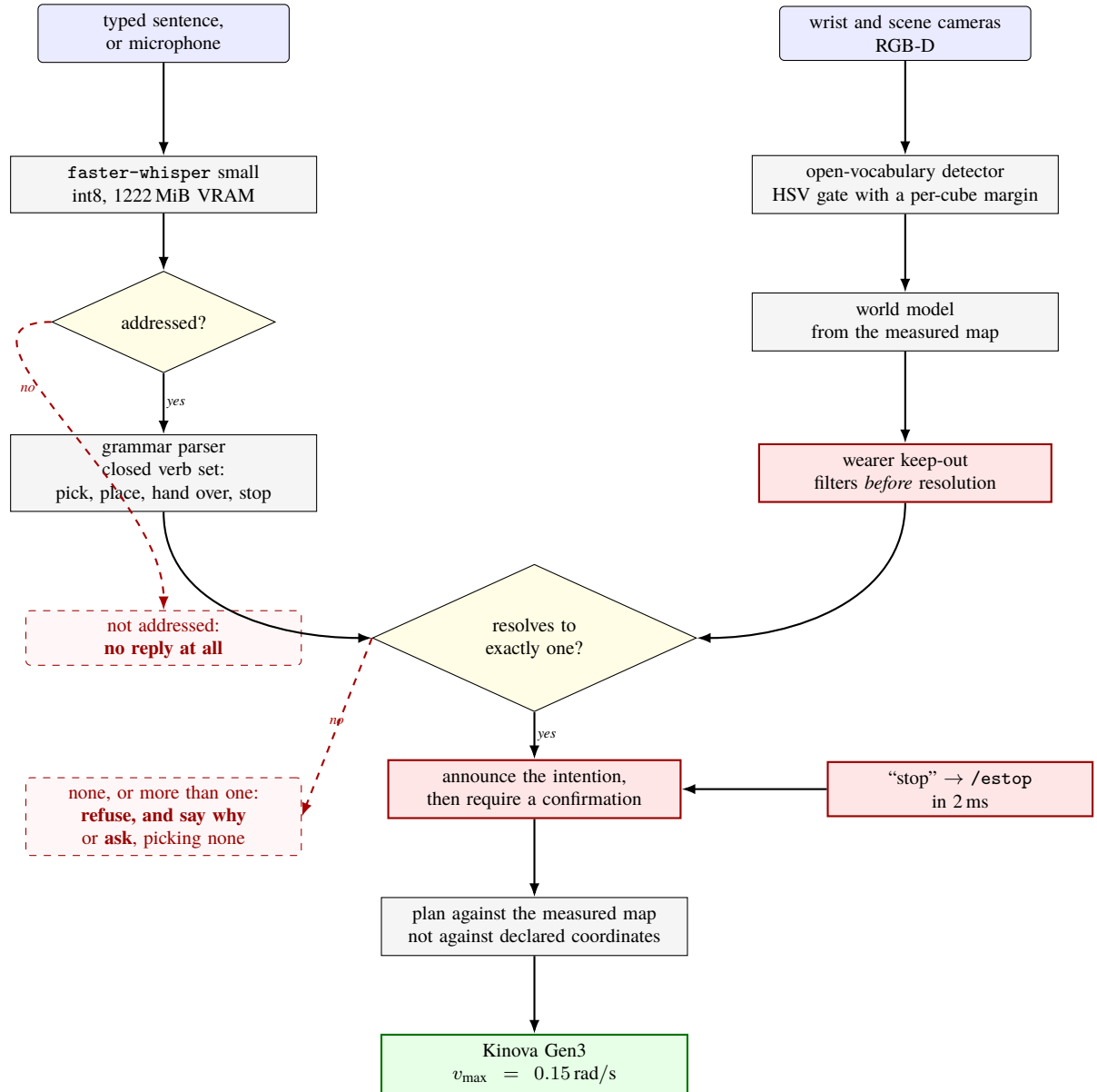


Figure 4.7: Mode 06, full autonomy from a typed or spoken instruction. Refusals are drawn as first-class terminals rather than as error paths, because the mode declines more utterances than it accepts and the distribution of outcomes is itself the result (Section 7.12). An utterance the system was not addressed in produces no reply at all: an earlier version answered “sorry, I did not understand”, which defeats the purpose of a wake word by replying to conversations the robot was never part of.

4.4.2 The controller route is motion capture, not virtual reality

The arrangement the laboratory actually operates has the operator seated *across the room, facing the wearer*, holding the two controllers as a six-degree-of-freedom input device and watching the physical robot directly. **Nobody wears the headset.** It stands on a shelf and serves as the tracking reference for the controllers. Every head-worn assumption in the earlier design documents was wrong about who is where, and two of them were load-bearing.

The alignment between the operator's frame and the robot's is a yaw, and it can never be a mirror. Facing a person and copying their motion is a reflection, of determinant -1 . Applied to position alone it looks correct; applied to a pose it mirrors every orientation, so the gripper rolls the wrong way while the positions appear right. That is the hardest class of defect to detect in a recording, and it is why the alignment is exposed as a single angle rather than as per-axis sign flips: no value of one angle can produce a reflection, which is asserted over $\pm 720^\circ$. The angle is measured rather than assumed, by having the operator move approximately 0.4 m directly toward the robot, a direction both people in the room can identify; a second motion is a cross-check rather than an input, and the calibration refuses to write a value if the two are not perpendicular and horizontal.

Two failures become routine off the head. The tracking reference is a physical object that can be knocked, and a knock announces nothing: poses stay valid, the rate stays at 90 Hz, the controllers stay tracked, and every subsequent pose is expressed in a frame that has rotated. The safety node latches the first reference pose and freezes past 20 mm or 2° , and the freeze deliberately survives the ordinary unfreeze path, because valid poses flow throughout the fault. And a controller can be occluded: the tracking-validity signal was derived from the arrival time of the last frame rather than from the per-controller validity flag, so covering a controller for five seconds produced no reaction at all while the stream continued at full rate. Worn, this is nearly harmless because the operator's hands are in the headset's own cameras. With the headset on a shelf, occlusion is the normal case.

4.4.3 Why the voice mode uses a grammar and not a language model

The command set is closed and small: grab, handover, place, stop. A quantised seven-billion-parameter model would occupy three to four of the four gigabytes of video memory available on this machine and add several hundred milliseconds, and it would place a non-deterministic component in the path that decides whether a robot beside a person's head begins moving.

The failure modes differ in kind, which is the substantive argument. A grammar that does not recognise an utterance returns unparsed, and the robot asks again. A language model asked for structured output can return a different, plausible verb, and nothing downstream can distinguish that from a correct parse. For five verbs that is a poor exchange.

4.4.4 Why not an end-to-end vision-language-action policy

Three reasons, recorded so the decision is not silently reversed. Such models are trained on embodiment collections that contain no backpack-mounted dual Gen3, and zero-shot transfer to a novel embodiment is poor while fine-tuning needs demonstration data this project does not have. They emit actions directly, which bypasses every gate in this system, since collision-aware inverse kinematics, the clearance floor and the step guard all sit between a *pose* and the arm. And their failures are unattributable, where the modular pipeline fails at a named stage with its inputs logged.

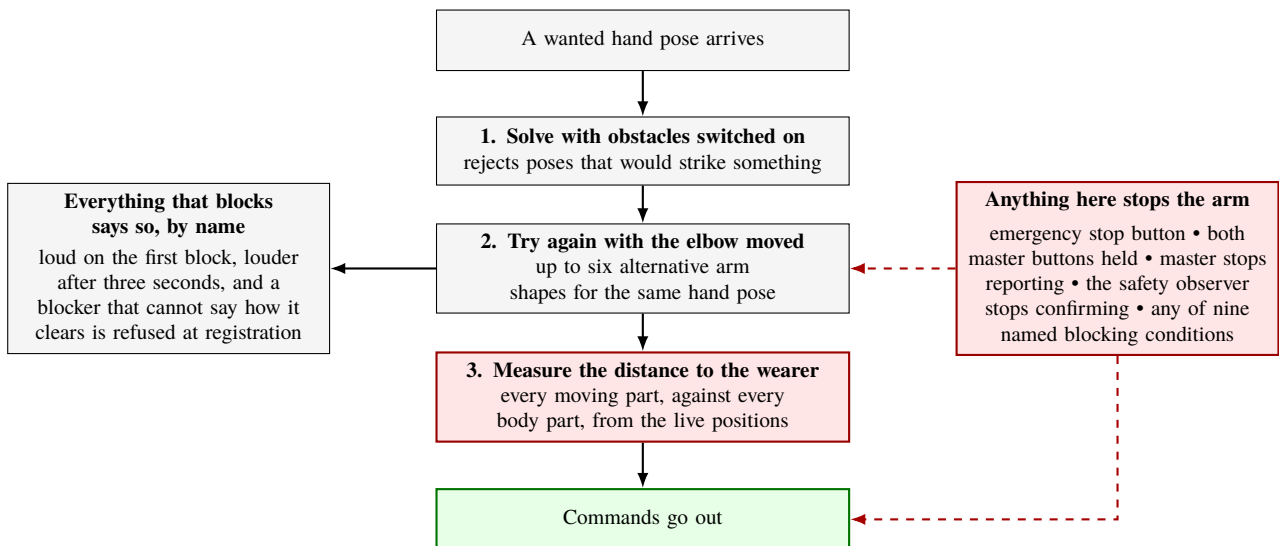
4.5 Safety architecture

4.5.1 Collision avoidance in three stages

The wearer is in the collision model. Adding this was not a matter of setting a flag: the flag was already set, but the wearer's model carried twenty-seven visual elements and one collision element, and no wearer link appeared in the semantic description. A planner cannot avoid what it cannot see.

Thirteen collision elements were added, paired with thirty-six exclusions for the mount-adjacent proximal links. The arm bases are bolted to the harness and are permanently adjacent, so without those exclusions every pose is trivially in collision. Everything distal of the first arm link stays enabled, which is what actually protects the wearer.

Avoidance is graduated. Collision-aware inverse kinematics rejects a configuration; on rejection the redundant degree of freedom is re-seeded and the solve retried, which recovers poses that would otherwise be dropped; and a geometric clearance floor measured from the transform tree provides a final margin. The floor is 150 mm between any moving link of the arm and any primitive of the wearer, and two facts about that number are needed to read any clearance figure in this thesis. It is measured against the fused body of [Section 5.6](#) rather than against the semantic collision description, which excludes forty-four proximal pairs and would therefore report a pose valid with the upper-arm tube inside the person. And `base_link` sits 0.1610 m from the torso and is moved by no joint, so under the original mount the floor had 11 mm of headroom before the arm did anything at all: a clearance reading of exactly 0.1610 m is a measurement of the mount and not of the arm. The mount change reported in [Chapter 9](#) raises that ceiling to 0.2202 m. Driving the commanded pose directly at the wearer's head and torso and holding it there stopped the arm at 0.072 to 0.079 m with zero clearance-floor blocks, so the earlier stages did the work and the floor remained a backstop. Every zone resumed on retreat.



Stage 3 is measured from the shapes of the arm and the body directly. It does not use the simulator's own collision settings, because those deliberately ignore the parts of the arm closest to the wearer's shoulders and would report a clear pose with the tube inside the person.

Figure 4.8: The three avoidance stages and the mechanisms that can stop the arm outright. The third stage is the one that discharges the safety case, and it is measured from the geometry of the arm and the body rather than from the semantic collision description, which excludes the proximal pairs a shoulder mount actually threatens.

4.5.2 Blocking must be observable

Four mechanisms in this system have historically stopped all motion silently. The common failure was never the blocking, which is usually correct. It was that blocking looked identical to working.

Every mechanism that can stop motion now registers with a monitor that enforces three properties: it announces itself by name on the first block and repeats; sustained blocking escalates in severity, because a block held for seconds is never normal; and a recovery string is mandatory at registration, so a blocker that cannot say how it clears is a latch and registration is refused.

4.5.3 Expiry is not permission

Every blocker in the system is written as a guard that blocks and returns, and clears on the fall-through path. That is correct while the guard is mutated elsewhere, and it latches the moment the guard also stops the callback running. This happened five times, the fifth inside the code written to catch the first four.

The structural fix is auto-expiry: a blocker not re-asserted for two seconds is treated as abandoned rather than held, since a genuinely held condition is re-asserted every cycle by design. Expiry is a third state, not a clear one.

Table 4.2: Blocker states. Expiry means the asserting loop stopped running, so the guarded condition is unknown rather than absent.

State	Meaning	Treated as blocking
active	the condition is held	yes
expired	the asserting loop stopped; condition unknown	yes
clear	live code says the condition is gone	no

Reading an expiry as clear would turn a crashed guard into a green light, which is worse than the latch it replaces. Two placement rules follow and both are load-bearing. The follower's refusal to publish sits at the publish and not at callback entry, because the clears that resolve an expiry are upstream of it and refusing at entry would prevent the re-assertion that clears the unknown. And the unknown-state blocker is excluded from its own trigger, or it re-arms itself indefinitely.

4.5.4 Emergency stop

Three independent triggers latch until an explicit reset: a service or topic call, both master buttons held simultaneously, and a dead-man on the master data going stale.

The dead-man keys on the *source* timestamp of the data, not on message arrival. This is the correction that made it work at all. The master node does not go silent when the master degrades: on a validation failure it substitutes the last good value and keeps publishing at full rate. It used to stamp that substituted frame with the publication time, so on the failure mode that actually occurs, a board emitting corrupted values rather than vanishing, the topic never went quiet and the timestamp never went stale. A dead-man keyed on arrival could not fire. Only a clean unplug produced silence, which is why every cable-pull test passed.

4.6 The operating window

Every capability reaches one window, and this is a constraint on development rather than a convenience. A feature reachable only from a terminal does not exist for the person running a session, who has a wearer standing in a rig and no attention to spare. Three rules follow and are enforced by an audit rather than by intention: a capability added or changed acquires a control in the same change; a control that alters what the next run does must log its new value, so that a session record can be reconstructed; and after every control has been exercised the window must still work, which is checked at the end of the audit sweep rather than at the start.

`scripts/verify_gui_buttons.py` presses every control and then checks what the press did: the emergency stop must reach the stop topic, a service control with no stack behind it must say so rather than failing silently, and the experiment panel's own defaults must produce a command the dispatcher accepts. It currently makes 257 checks. The first time it ran to completion it found four defects, of which two are worth recording: five session controls raised an exception inside a Qt slot, which the framework swallows, so they did nothing at all and said nothing; and the stray-process preflight, which is the guard for the one-stack constraint, had never executed because its module was never imported.

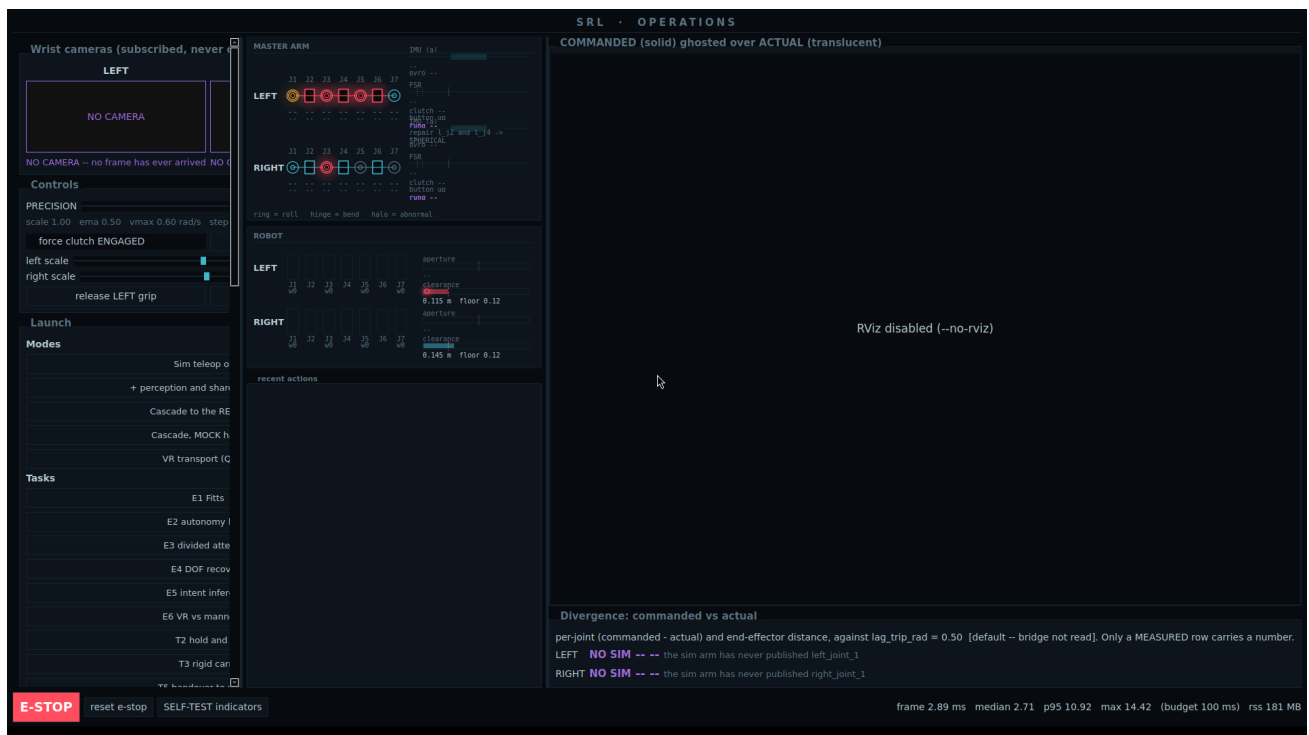
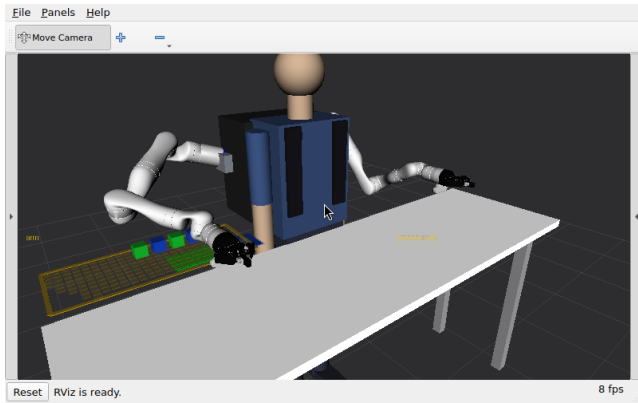
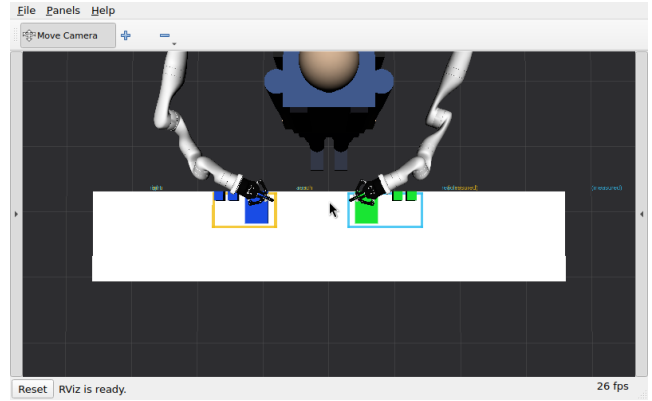


Figure 4.9: The window a session is run from. One command opens it. The left column is grouped into set-up, run and status; the centre carries the mode registry and the per-mode controls; the right is the three-dimensional view of the commanded configuration beside a panel drawn from the arms' own reported joint states, with the divergence between them read out beneath both. The arrangement exists so that a difference between what was commanded and what the arms report is visible without reading a log, which is the failure that motivated [Section 4.5.2](#). The emergency stop is the red control at the lower left and is reachable from every tab.



(a) Oblique view.



(b) Plan view, with the two coloured pads and the marked working region.

Figure 4.10: The simulated scene at the home pose. The wearer is drawn from the same primitives the clearance check evaluates, so the display and the enforced geometry cannot diverge: a body that appears in one and not the other has been seen and fixed twice in this project ([Chapter B](#)). The working region is drawn as an outline at the plane it bounds rather than as filled tiles, because a filled surface hides the objects standing on it.

4.7 Verification methodology

Twenty-six times in this project a result that looked like a system failure was a defect in the measuring tool. [Section 10.7](#) states what that implies for the measurements in this report and [Chapter B](#) is the case-by-case record; the rule below is the practice that came out of them, and it is part of the method rather than a reflection on it. A noise floor of exactly zero on all fourteen channels was the recorder sampling faster than the sensor updates. An isotropy figure swinging by a factor of two between runs was a statistic chosen to be maximally sensitive to a rare tail failure. A detection rate of 0 to 4 % was a renderer producing images unlike anything the detector was trained on. A path-verification failure was the verifier carrying its test path in the one direction that leaves the feasible region.

The rule adopted is that a surprising failure is evidence about the instrument until the instrument has been cleared against a known answer. In practice this means every analysis script has a known-answer test, real recorded data is preferred to generated data, synthetic data is used only where the ground truth is constructed rather than rendered, and a result contradicting an earlier measurement is treated as an instrument check until the two are reconciled.

A corollary governs every coordinate in this thesis. The inverse-kinematics solver restarts randomly, so a pose at the edge of the feasible set is a coin flip and one call is one flip. Verification is therefore ten repeats over the whole path, densified to 20 mm, not two calls at the endpoints. Poses are graded solid, marginal or failed, and marginal is not usable.

4.8 Calibration

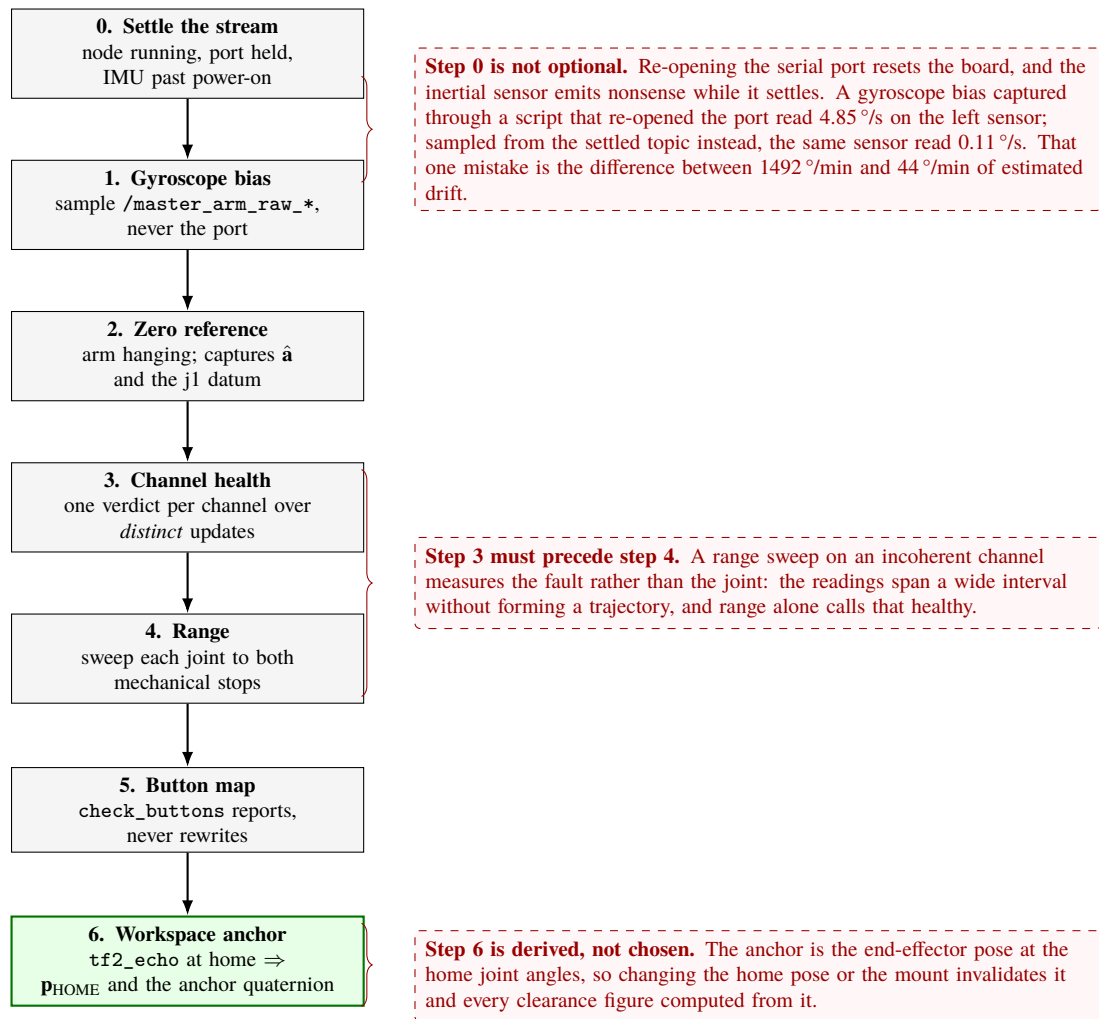


Figure 4.11: The calibration procedure, in order. Each step consumes the previous step’s output, and the two orderings that are load-bearing are braced to the call-out that explains them. Steps 0 and 3 are both consequences of the same general point: a calibration routine measures whatever the instrument was actually doing at the time, so establishing that the instrument is in a valid state is part of the measurement and not a preliminary to it.



Figure 4.12: The complete model at the home pose: both Gen3 arms, the backpack frame, and the wearer built from `wearer_posture.py`, which is the single source that writes both the kinematic description drawn here and the primitives the clearance guard evaluates. Reading a clearance figure from this model requires two facts from [Section 4.5.1](#): the mount origin sits 0.1610 m from the torso and is moved by no joint, so a clearance reading of exactly that value is a measurement of the mount rather than of the arm; and the semantic collision description excludes the forty-four proximal pairs a shoulder mount actually threatens, so clearance is measured geometrically and not by asking the planner whether the pose is valid.

Chapter 5

Perception, environment mapping and autonomous manipulation

The system described so far commands the arm from a human input. This chapter covers the parts added afterwards that let it command the arm from a *measurement* instead: object detection, grasp synthesis, a sweep that measures the working surface and the objects on it, a planner that consults the result, and a natural-language route into the same task layer. It closes with the camera that observes the wearer, which is a safety component rather than a manipulation one.

5.1 The defect that motivated it

Almost everything in the delivered task layer runs on *declared* coordinates. The blocks in the pick-and-place task are at their coordinates because a source file says so; the working surface is at its height because a constant says so. All of it is verified at ten repeats over densified paths ([Chapter 7](#)), and none of it survives a different table, a different object, or the same object displaced by 50 mm. The number in the file becomes a proposition the arm acts on with complete confidence, and nothing in the system is positioned to disagree with it.

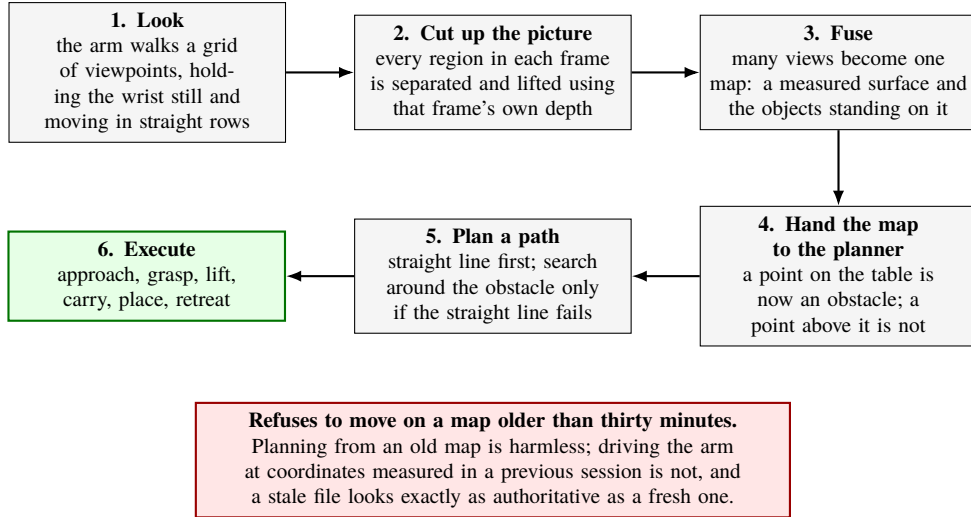
Three components capable of sensing the same quantities already existed and had never been composed. `srl_perception/table_scene.py` finds a support plane and the objects resting on it, and was used to print a description. `srl_perception/joint_planner.py` defines a `VoxelWorld` so that a planner can avoid obstacles that are not the wearer, and was constructed nowhere outside its own self-test, which means no path this project had ever planned knew that the table existed. `srl_experiments/work_surface.py` exposes `set_measured()` so that the surface height can come from depth, and nothing called it.

[Figure 5.1](#) is the stage that replaced this.

5.2 Detection

Detection is open-vocabulary rather than class-fixed: the stage takes a noun phrase and returns instances. `srl_perception/prompt_detector.py` runs YOLO-World for boxes [\[5\]](#) with FastSAM for masks [\[38\]](#), and falls back to a hue-and-depth filter when neither model is resident. **The backend that produced a detection is reported with the detection**, so a result can never imply tacitly that the stronger model was running.

Two properties of the deployment are worth recording because both were forced by measurement rather than chosen.



Before this existed, every coordinate the arm used was written in a file by hand. All of it was verified, and none of it survived a different table, a different object, or the same object moved fifty millimetres.

Figure 5.1: The calibrate-then-plan stage. The sweep is deliberately shaped like a printer’s bed probe: one wrist attitude per pass with pure translation between cells, rather than re-aiming the camera at every cell.

Range is used as a discriminator where hue cannot be. The laboratory contains teal robots whose shadowed hue is 76; the target green block sits at 74. Two units apart is inside the noise, and a colour filter had already been fooled by them three times. The robots stand 2.8 m away while the rig works within 1.6 m, so range separates the two populations cleanly and is applied at detection time.

Range is a discriminator that costs nothing to compute. A blob of s pixels across at range Z subtends a physical width

$$w = \frac{s Z}{f}, \quad (5.1)$$

so the same expression that rejects the distant robots also rejects a 210 mm pad presented as a 40 mm cube, without a second model and without a threshold on appearance.

Grasp synthesis is geometric and deliberately not learned. `srl_perception/rgbd_grasp.py` takes the principal axes of the object’s depth cloud. Writing the mean-centred points as $\mathbf{P}_k$, the axes are the eigenvectors of the scatter matrix

$$\mathbf{C} = \frac{1}{N} \sum_{k=1}^N (\mathbf{P}_k - \bar{\mathbf{P}})(\mathbf{P}_k - \bar{\mathbf{P}})^\top, \quad \mathbf{C} \mathbf{u}_j = \lambda_j \mathbf{u}_j, \quad \lambda_1 \geq \lambda_2 \geq \lambda_3, \quad (5.2)$$

and the jaw closes across $\mathbf{u}_3$, the direction of least extent, while approaching along whichever of the remaining axes is free. The width to be closed on is not the eigenvalue but the extent of the projected points, which is the quantity the jaw physically has to span:

$$w = \max_k \mathbf{u}_3^\top \mathbf{P}_k - \min_k \mathbf{u}_3^\top \mathbf{P}_k \leq w_{\max} = 85 \text{ mm}. \quad (5.3)$$

A parallel jaw closing on a rigid convex object is a solved problem; a learned alternative would need video memory that is not available beside the detector; and, decisively, a geometric method can *name* the cause of a refusal. Equation (5.3) gives one of the three it names — wider than the jaw, and by how much — alongside no

depth returns and too few points to fit an axis. All three were exercised on real objects in [Chapter 8](#), where the width test refused every object in the scene and reported the arithmetic that made it do so.

5.2.1 A reachability gate that could not fail

`srl_perception/grasp_pipeline.py` checks a candidate grasp for reach and for wearer clearance before offering it. The clearance half of that check was inoperative for the life of the module, and the mechanism is the same one that recurs throughout this thesis.

The scorer returns two clearances. One is taken over the whole kinematic chain, including the immobile mount stub between `base_link` and `shoulder_link`; that stub is nearer the wearer than anything the joints can move, so the quantity reads a constant 0.2202 m in every pose the arm can adopt. The gate compared it against the 0.15 m floor. **Every grasp this pipeline ever approved was tested against a predicate that could not be false.** It now reads the moving-chain clearance, which is 0.4031 m at the home pose, and a regression test aims the hand at the centre of the wearer's torso and requires a refusal that names the wearer.

5.3 Measuring the working area

5.3.1 Why the sweep holds one wrist attitude

The first version of the sweep aimed the tool axis at each grid point in turn. That is the intuitive design and it is wrong for a measurement. Re-aiming means the wrist attitude differs between neighbouring cells 90 mm apart, so a residual error in the camera-to-wrist transform becomes a *different* error in every frame: it can neither cancel across views nor be estimated from them.

A printer's bed probe does not re-aim. It holds one attitude and translates. `srl_perception/calibration_sweep.py` does the same. One wrist direction per pass, with the camera grid being the work grid rigidly translated back along that fixed ray, so consecutive cells differ by a pure translation; several passes at different yaws supply the object sides, which a single elevation cannot see. The sweep is two layers by three yaws by twenty-four cells by two arms, and the traverse order is a boustrophedon rather than a typewriter raster: 1.140 m of travel against 1.702 m for the same coverage, measured by the module's own self-test rather than asserted.

5.3.2 Stationary is not arrived

A joint can sit 0.02 rad from its commanded value and still be moving through it. A frame captured at that moment is smeared across two poses, while every statistic computed downstream reports normally. The sweep therefore requires no joint to move by more than 0.0015 rad over 400 ms, which is a tenth of the arrival tolerance, before a frame is accepted.

5.3.3 What the map contains

An earlier surface error of -5.2 mm was itself an instrument defect and is recorded in [Chapter B](#): the reported height was the fitted plane's intercept *at the origin*, which is a point inside the wearer, rather than its height over the working area.

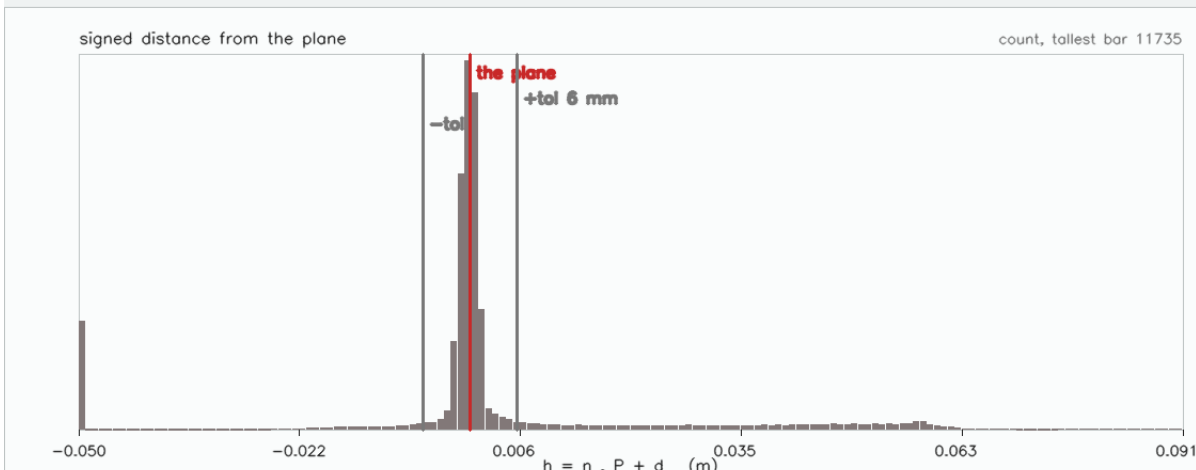
18 RANSAC PLANE FIT

FORMULA draw 3 points $\rightarrow n = (b-a) \times (c-a) / |...|$, $d = -n.a$; score = $\#\{ |n.P + d| < 6 \text{ mm} \}$; keep the best of 400 draws

SOURCE scripts/servo_pick_left.py:fit_plane, imported; fitted on 54789 of 54789 points



green = inlier (on the plane), red = off it



every point's distance from the fitted plane

MEASUREMENTS

quantity	value	unit	quantity	value	unit
normal x	-0.2488	-	inliers	40517	count
normal y	-0.8831	-	inlier fraction	73.95	%
normal z	-0.3977	-	residual RMS	1.428	mm
offset d	0.295	m	inlier tolerance	6	mm
points fitted	54789	count	RANSAC iterations	400	count
points scored	54789	count	angle to the optical axis	66.56	deg

SUMMARY normal $(-0.2488, -0.8831, -0.3977)$, offset 0.2950 m | 40517 of 54789 points are inliers (74.0%) | RMS 1.43 mm | 66.6 deg between the plane normal and the camera's optical axis

NOTE A plane with too few inliers is not a plane. The pick refuses below 200 inliers rather than returning a geometry whose RMS is NaN -- everything downstream would treat that as a fix and the arm would move on it.

Figure 5.2: One stage of the detection pipeline as it is actually instrumented. Every stage draws itself on the frame it was given, states the operation as a formula, and reports the numbers it produced with units, so that a figure captioned “the vision works” can be replaced by one a reader can check. Here the support plane is fitted by random sample consensus on the left-hand wrist camera: 40 517 of 54 789 deprojected points are inliers at a 6 mm tolerance, giving a residual of 1.43 mm root mean square. The inlier fraction is the quantity that matters, not the residual: a plane can fit three points perfectly.

Table 5.1: Contents of the fused map after one full sweep, against the scene it was measuring. Objects are cut from the image by `segment_lift` and lifted through each frame’s own depth, rather than clustered from the accumulated point cloud, which is pinned by `test_objects_come_from_the_picture.py`. Record at `recordings/baselines/world_map.json`.

Quantity	Measured
Support plane inliers	694 455
Surface height	1.250 26 m against a declared 1.250 m
Surface tilt	0.036°
Objects recovered	6 of 6
Object centres	every one within 1 mm in x and y
Object extents	every dimension within 3 mm
Cells swept	109, of which 74 reachable
Coverage of reachable cells	98.65 %
Stillness refusals	0
Elapsed	13 min 15 s with the cache warm

5.3.4 Two lifetimes, one command

Bundling reachability and scene mapping into a single calibration command obscured that they are facts about different things, and the distinction governs how often each must be redone.

Table 5.2: The two products of the calibration stage and when each expires.

Record	A fact about	Redo when
<code>reachable_cells.json</code>	the robot : which camera poses the arm can reach	once. It is keyed on every bound, layer, facing, elevation and step, so moving the mount, changing the home pose or editing the description invalidates it automatically
<code>world_map.json</code>	the scene : the surface and what stands on it	whenever the scene changes. There is no incremental re-map: <code>build()</code> fuses every view from scratch

Until an incremental re-map exists, `scripts/pick_from_map.py` reports the map’s age on every run and **refuses to move from a map older than thirty minutes**. Planning from a stale map is harmless and is reported; driving the arm at coordinates measured in a previous session is the failure that costs something, and a stale file is indistinguishable from a fresh one by inspection.

5.4 Planning against the measurement

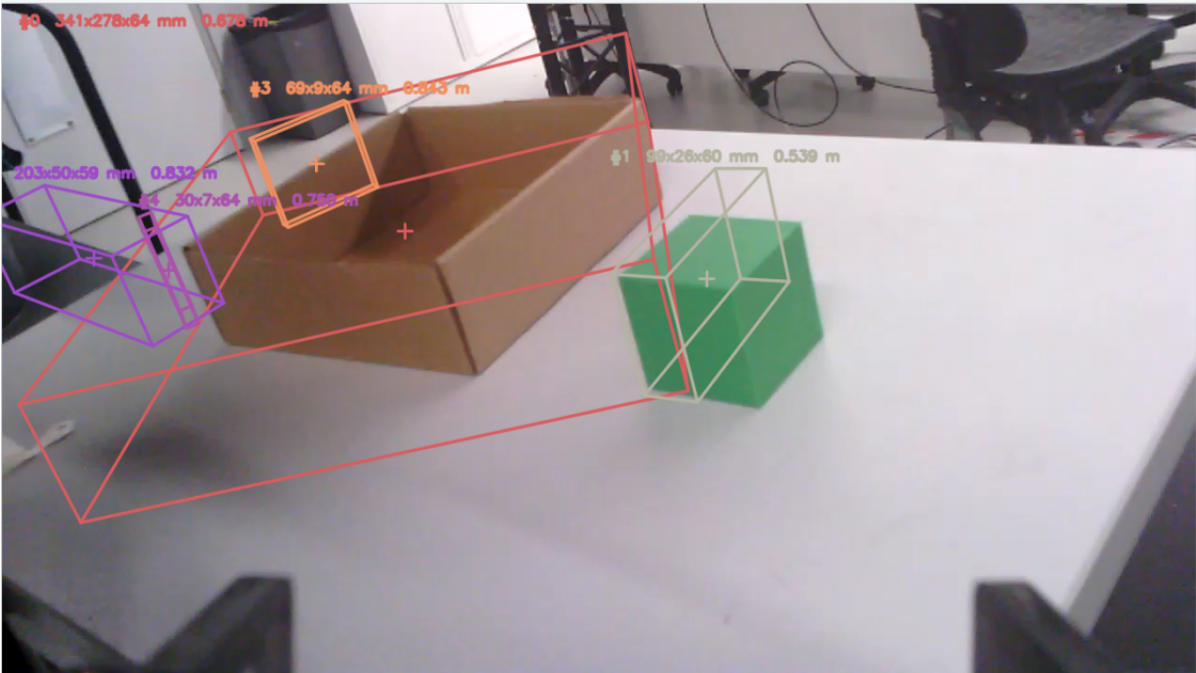
For teleoperation there is no planner and there should not be: the operator’s hand is the plan and the correct problem is smooth, safe following (Chapter 4). For the autonomous mode there is a goal, and the absence of a planner had a measurable cost. A pick whose pre-grasp and grasp poses are both individually clear, streamed as a straight line in joint space, passes within 0.1462 m of the wearer at step twenty-nine of sixty-four. The path checker catches it, and catching it means the run stops.

`joint_planner.py` implements RRT-Connect [18] with four choices made for this platform specifically.

30 ORIENTED 3-D BOXES

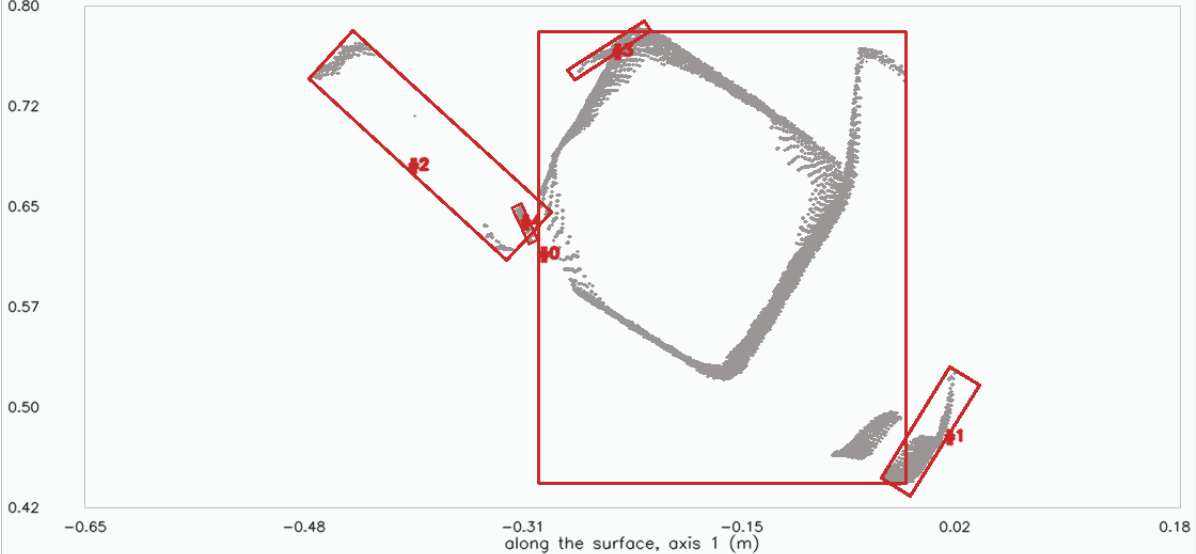
FORMULA centre = mid-extent ACROSS the surface + (foot + top/2) ALONG the normal; axes = (minimum-width direction, its perpendicular, the plane normal); size = the spans along them

SOURCE segment_lift_measure for the horizontal centre and rgbd_grasp.shell_corrected_centre's reasoning for the vertical -- the combination is checked to 2 mm against a constructed cube in --self-test



each object's box, projected back into the picture

along the surface, axis 2 (m) plan view: every object on the surface, to scale



the same objects, looking down on the surface

MEASUREMENTS

quantity	value	unit
objects posed	5	count
nearest object	0.5389	m
furthest object	0.8427	m
largest footprint	341	mm
tallest object	63.9	mm
graspable within the 85 mm jaw	4	count

every object, in the camera frame

#	points	area px	x m	y m	z m	range m	length mm	width mm	height mm	yaw deg	graspable
0	51424	104550	-0.0971	0.0032	0.6706	0.6776	341	277.7	63.9	0	no
1	10697	34542	0.0554	0.0234	0.5355	0.5389	99.4	25.9	59.7	148.3	yes
2	861	10222	-0.3103	0.0208	0.7715	0.8318	203	49.6	58.7	47.5	yes
3	701	17453	-0.1798	-0.0419	0.8222	0.8427	68.6	9.2	63.9	122.8	yes
4	343	5923	-0.2415	0.0264	0.7094	0.7498	30.2	7.3	63.9	24.8	yes

SUMMARY 5 objects, 0.539 to 0.843 m away

Joint space, not Cartesian. The binding constraint is the distance from the whole *arm* to the wearer, not from the hand. A Cartesian planner would solve inverse kinematics at every node in any case.

The wearer floor is the collision predicate , evaluated through the same moving-chain clearance the guard enforces, rather than through the semantic description, which is blind to the proximal pairs that matter.

Every edge densified at 0.05 rad , matching the path checker. A planner that checks only its nodes reproduces, one level up, the defect it exists to remove.

Straight line first, then shortcut, then resample to the follower’s own step size. Handing the follower waypoints spaced further apart than its maximum step means it invents the intermediate motion by unchecked interpolation, which reintroduces the same defect at the last stage.

Larger alternatives were assessed against the platform’s video memory and rejected on stated grounds rather than by omission. CUDA-accelerated motion generation is the strongest option in this space and needs roughly 2.5 GiB at batch 1024 on a card with 48 GiB; GPU-accelerated simulation for policy learning lists an eight-gigabyte card as its minimum [24]. This machine has 4 GiB in total, shared with the detector.

5.5 Commanding a task in natural language

The full-autonomy mode accepts a typed or spoken instruction. The parser is a grammar, for the reasons given in Chapter 4, and the design question that matters is what it does when an instruction is ambiguous.

Asked to *put the blue block on the blue mat* with two blue blocks present, the system **asks** rather than choosing. Nearest, leftmost and first-detected are all defensible and all guesses, and the manipulator is bolted to a person. Asked to *put the blue ones on the blue mat*, it does not ask, because the plural settles it. The distinction lives in the instruction’s own quantity marker rather than in a heuristic over how many candidates happen to match, which is why the evaluation scene contains two blocks of each colour rather than one.

Table 5.3: Seventy-five phrasings of the same task, scored against the grammar as committed six days earlier. The earlier grammar was checked out of version control into a temporary tree and run in its own interpreter, so the two columns differ by the code and by nothing else. Replies to a clarifying question are scored on a separate line and are never counted as correct. Produced by `scripts/sweep_t1_instructions.py`.

Outcome	Earlier grammar	Delivered
Correct	21	34
Asked a clarifying question	14	14
Refused	37	27
Misunderstood	3	0

The bottom row is the one that matters, and the three misunderstandings share a mechanism worth stating. Each was a compound instruction of the form *put the blue ones on the blue mat and the green ones on the green mat*: two complete instructions sharing a verb. The destination split cut at the first destination cue, so the head of the sentence was *put the blue ones*; the two-target check counted targets within the head and found one; and the planner emitted a plan for the first clause, discarded the second in silence, moved two objects of four, and reported success. Every component behaved as specified.

A refusal costs the operator a repetition. A misunderstanding moves the arm, and it is therefore the quantity this evaluation is designed around.

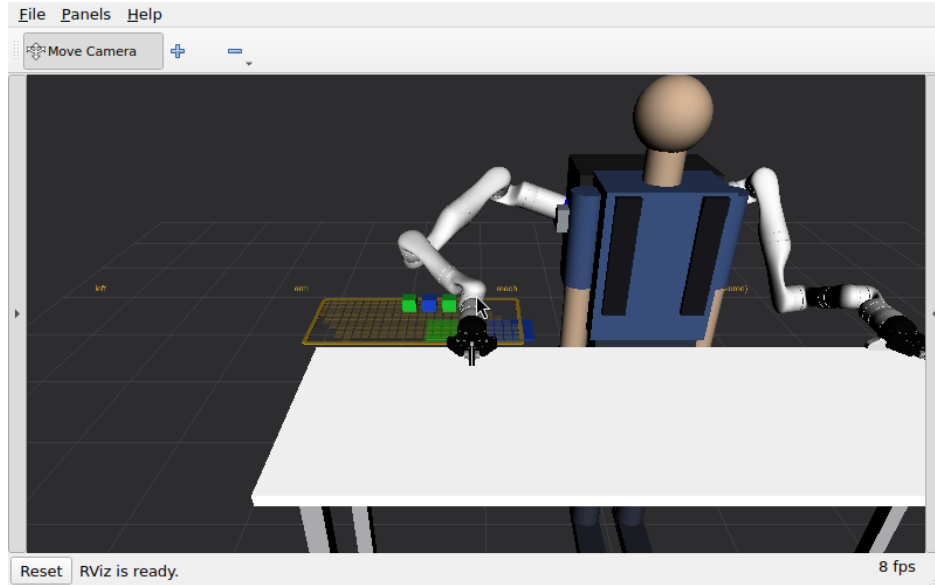


Figure 5.4: The scene an instruction is resolved against. The operator types a sentence; before anything moves, the window shows the parsed interpretation, the detections the plan was formed from with the classifier’s own hue margin for each, and the intention in plain words, and waits for a confirmation. The instruction is then handed to the runner verbatim, so the record of what was asked cannot diverge from what was done, and that is what the verification script checks: that the argument vector the runner receives carries the task, the mode, the detections it was planned from, and the sentence unaltered.

5.6 Observing the wearer

The clearance floor of [Chapter 4](#) is enforced against a body. Until late in the project that body was always the mannequin in the robot description. `srl_perception/wearer_tracker_node.py` replaces it, part by part, with a measured one, using a markerless pose model with metric landmarks lifted by perspective-n-point [21].

Table 5.4: Measured by `scripts/measure_wearer_tracking_rate.py`. The end-to-end spread is a configured publication rate limit, not the estimator.

Quantity	Measured
Pose inference	28 to 34 ms per frame
Frame to estimate	61.6 ms median
End to end	62 to 562 ms
Primitives maintained in the planning scene	12

The safety argument for admitting a camera into the clearance check is a single monotonicity rule, and it is enforced in one function:

The camera may only ever make the wearer larger. Per body part, `fuse()` retains whichever of the tracked and mannequin primitives is *nearer* the robot. There is no mode switch.

A body injected 0.10 m, 0.30 m, 1.00 m and 5.00 m further away in turn moves the enforced margin by nothing at all. Six faults were then injected through the live node, comprising a dark frame, a blank frame, two people in shot, an empty frame, occluded arms, and a control; the node refused seven of seven cases, and each refusal

was required to *name* the injected fault. That naming requirement is what exposed a false pass in which the two-person gate had never executed.

The substitution is per part, so a measured torso can stand beside a mannequin upper arm within one scene, and `/wearer/enforced` publishes which body is in force. Before this was connected, the tracker reached the planning scene and *not* `mount_guard_node`, which constructs its wearer at import time: the body the planner avoided and the body the floor was enforced against were two different bodies, and the floor held the mannequin.

The honest limitation is on the fallback side and is quantified in [Chapter 10](#). If the wearer raises their arms and tracking is then lost, the guard reverts to the assumed posture, which has the arms down, and believes there is 0.317 m of clearance where there is 0.093 m.

5.7 The scene camera

A second camera, observing the workspace from outside the arms rather than from a wrist, is implemented. `scene_camera_node` refuses to republish rather than passing through stale frames and publishes a zero intrinsic matrix until calibrated; both calibration tools carry self-tests that pass to 0.43 % in focal length, 4.5 px in principal point and 0.0000 m and 0.000° in the extrinsic.

No camera has ever been attached to it. The device is the host’s own webcam, and forwarding it into the Linux environment requires an administrator-level bind that was refused. Everything in this section is verified against mock inputs, and the project keeps “as built” and “inferred” in separate records for this reason.

Chapter 6

Motion generation and simulation-to-hardware transfer

Two questions are answered here. What shape does the commanded motion have between one solver output and the next, and what happens to it when the same joint command is replayed onto a physical arm. Both were assumed rather than measured for most of the project, and both turned out to carry errors comparable to the grasp tolerance.

6.1 The step limiter was the entire motion generator

Until it was replaced, the last stage before publication was `ik_follower_node.clamp_towards()`: take the solver's joint vector and walk toward it, limiting *each joint independently* to `max_step_rad` per control cycle. The diagnosis was reproduced to the decimal place before anything was changed, on the left arm's home configuration displaced by $[0.50, -0.30, 0.10, 0.25, -0.05, 0.15, 0.02]$ rad at 50 Hz. Three separable defects follow, and only the first had been named previously.

It is not synchronised. A joint needing 0.10 rad arrives in one cycle while a joint needing 0.50 rad takes two, so the joints sit at different fractions of their own travel at every instant and the end effector leaves the path the solver's solution implies.

It is not continuous. From rest, the first cycle commands a whole step. Its implied acceleration is 1343 times the jerk-limited one.

It never read the joint limits. `max_step_rad` divided by the cycle time is 17.5 rad/s against the 1.3963 rad/s and 1.2218 rad/s declared in `joint_limits.yaml`: a factor of 12.53, and the declared limits differ per joint, which a single step size cannot express.

6.1.1 The reported metric had to change first

The obvious metric, the distance between the commanded hand position at cycle k and where it would be if every joint were at fraction k/N of its own travel, conflates two different things: leaving the path, and not being at k/N along it. A jerk-limited generator is deliberately not at k/N , because that is what a velocity profile is, so scoring one on that metric charges it for the feature. It reads 69.8 mm on the replacement, and the number means nothing.

The figure reported below is therefore *off-path* distance: the distance from each commanded hand position to the curve that the straight joint-space line traces, which is free of any parameterisation.

Table 6.1: Measured by `scripts/measure_teleop_motion.py` against the platform’s own forward kinematics. The capture gate for a grasp on this platform is 30 mm, which the first row exceeds by itself.

Generator, left arm	Off-path	Same-cycle	Peak joint speed	Arrival
Per-joint clamp, as shipped	51.3 mm	135.3 mm	$12.53 \times$ limit	2 cycles
Jerk-limited, delivered	0.0 mm	69.8 mm	$1.00 \times$	1 cycle
Synchronised clamp, fallback	0.0 mm	22.7 mm	$1.00 \times$	1 cycle

The right arm behaves the same way, at 39.2 mm reduced to 0.0 mm.

6.1.2 It is not a tuning parameter

Swept against its own step size, the excursion does not tend to zero. It settles.

Table 6.2: Off-path excursion against the step limit. Each row is a genuinely different path; densifying at 0.02 rad changes none of these figures, which was checked rather than assumed.

max_step_rad	Cycles to arrive	Off-path
0.35	2	51.3 mm
0.20	3	34.4 mm
0.10	5	68.0 mm
0.01	50	68.0 mm

The desynchronisation is proportional to the travel, so it survives any step size. It is a property of the algorithm rather than of its parameter.

6.1.3 Why a per-joint clamp cannot stay on the line

The limiter it replaced applied, once per control cycle and independently to each joint i ,

$$q_i^{(k+1)} = q_i^{(k)} + \text{clip}(q_i^* - q_i^{(k)}, -\Delta, +\Delta), \quad (6.1)$$

with one step bound Δ shared by all seven joints. The defect is structural rather than a matter of tuning. Write the required joint displacement as $\mathbf{e} = \mathbf{q}^* - \mathbf{q}^{(k)}$. What the straight line in joint space requires is that the applied step be *parallel* to $\mathbf{e}$; Eq. (6.1) applies $\text{clip}(\mathbf{e}, -\Delta, +\Delta)$ component-wise, which is parallel to $\mathbf{e}$ only when every component is inside the bound or every component is outside it with the same magnitude. In every other case the saturated joints advance at Δ while the unsaturated ones advance at e_i , the direction of travel rotates, and the end effector leaves the Cartesian path the inverse-kinematics solution implies. The excursion is therefore a property of Eq. (6.1) and not of the value of Δ , which is why sweeping Δ makes the excursion settle at 68 mm rather than approach zero.

6.2 What replaced it

Ruckig [2] was adopted: time-optimal, jerk-limited, per-degree-of-freedom limits, and it accepts a non-zero *target* velocity, which is what a streaming teleoperation target actually is. The generator lives in `srl_teleop/motion_`

generator.py and is pure, with no dependency on the middleware or on a display, so the measurement script and the follower consult the same module and the measurement therefore describes the code the robot runs.

Three implementation properties are load-bearing.

Phase synchronisation rather than time synchronisation. Both make every joint arrive together; only phase synchronisation keeps them on the straight line between the endpoints, and that line is the entire point. The generator solves, for each joint, the time-optimal profile subject to

$$|\dot{q}_i| \leq v_i^{\max}, \quad |\ddot{q}_i| \leq a_i^{\max}, \quad |\dddot{q}_i| \leq j_i^{\max}, \quad (6.2)$$

and then, rather than merely stretching each profile to the slowest joint's duration, drives every joint from a single scalar phase $\sigma(t) \in [0, 1]$:

$$\mathbf{q}(t) = \mathbf{q}^{(k)} + \sigma(t) (\mathbf{q}^* - \mathbf{q}^{(k)}), \quad \sigma(0) = 0, \sigma(T) = 1. \quad (6.3)$$

Because the same σ multiplies the whole displacement vector, the commanded configuration is by construction on the segment between the endpoints for every t , which is exactly the property Eq. (6.1) lacks. Measured on the same case: phase 0.02 mm of path deviation, time synchronisation 9.5 mm, clamp 51.3 mm.

The velocity limits are the robot's; the acceleration and jerk limits are assumed and say so. joint_limits.yaml declares has_acceleration_limits: false and a zero maximum acceleration for all fourteen joints, so there is nothing to read. Both are derived from two named ramp times and the generator's provenance field reads ASSUMED for as long as that remains true. A test writes a configuration that *does* declare an acceleration limit and requires the label to follow the file, so the honesty is a property of the code rather than a hard-coded string.

The fallback is synchronised and names itself. Without the library the generator does not silently revert to the per-joint clamp: a synchronised clamp scales the whole joint vector by one factor, so the path stays on the line and every joint still arrives together. It is not jerk-limited and it reports that it is not, in its status field and in the operating window.

6.2.1 Five defects in the replacement, each caught by a check

Recorded because four of the five would have passed a conventional test suite.

1. **Re-planning every cycle destroys phase synchronisation.** The first version called the library's planning entry point every control cycle. A re-plan from a mid-profile state cannot reproduce the phase-synchronised profile it is halfway through: the joints' fractions of travel spread to 0.07 by cycle ten and re-converged, leaving the commanded path 0.0264 rad off the straight line, against 8.9×10^{-17} rad when a single trajectory is ridden through the update entry point. Every other check passed either way.
2. **Floating-point noise forced the re-plan.** Even after switching, it re-planned on all thirty-six cycles, because the target was recomputed each cycle as $q + \text{wrap}(q_{\text{target}} - q)$, which is not bit-identical to q_{target} . The goal moved by about 1×10^{-17} every cycle and the library compares by value.
3. **The fallback claimed to be jerk-limited by reporting zeros.** It returned an acceleration vector of zeros because it does not model one, and the continuity check reads that field, so the fallback passed by fabricating the quantity being checked. It reports its real implied acceleration now, 62.5 times the jerk-limited step, and a test requires that figure to exceed ten.

4. **The off-path metric had a resolution floor.** The reference curve was 4001 points over a 0.4 m path, a 0.1 mm grid, so nearest-sample distance bottomed out at half a grid step and a generator on the line to 2×10^{-16} rad measured 0.0509 mm off it. The instrument's resolution was being reported as the robot's behaviour. Refined by bisection on the real forward kinematics, the same path reads 9.4×10^{-9} mm.
5. **An arrival test used an absolute tolerance on a per-joint quantity.** Under a 1×10^{-6} rad gate a phase-synchronised move reported arrivals spread over two cycles: every joint was at the same fraction throughout, and those moving 0.02 rad were inside the gate a cycle before the one moving 0.50 rad. The travels span a factor of twenty-five, so an absolute gate measures the instrument's scale.

6.2.2 One definition of which joints wind

The single-source test written for this change found that `CONTINUOUS_IDX` carried six independent definitions across the teleoperation package. They agreed, which is luck rather than design: which joints are declared continuous is a fact about the robot description, and seven copies of it have the same shape as two home poses. There is one now, with a test that walks the package's syntax trees and requires exactly one assignment.

6.3 The simulation-to-hardware gap, measured

Replaying recorded joint commands onto the physical arm and comparing the achieved position with the commanded one gives a systematic result rather than a noisy one.

Every joint parks 0.305° short of its target, on the side it approached from.

Of 252 recorded per-joint errors, 211 lie between 0.25° and 0.30° with the sign following the direction of travel. A distribution concentrated at a magnitude with a sign determined by direction is a *bound*, not a spread, and that is what identifies it as a terminal offset rather than as noise. At the end effector it is 7.2 mm RMS on a 100 mm move, on both arms.

Two candidate models are distinguishable only at short travel. Writing the commanded displacement of joint i as $\delta_i = q_i^* - q_i^0$, the two are

$$\underbrace{q_i^{\text{ach}} = q_i^* - \varepsilon_i \text{sgn}(\delta_i)}_{\text{terminal offset}}, \quad \underbrace{q_i^{\text{ach}} = q_i^0 + (1 - g_i) \delta_i}_{\text{proportional gain error}}. \quad (6.4)$$

They coincide when $\varepsilon_i = g_i |\delta_i|$, so at a single travel distance no amount of data separates them; at $|\delta| = 100$ mm they predict the same Cartesian error and at 20 mm they differ by a factor of five. Propagating the recorded joint errors to the end effector through the manipulator Jacobian,

$$\Delta \mathbf{x} = \mathbf{J}(\mathbf{q}) \Delta \mathbf{q}, \quad (6.5)$$

reproduces the measured Cartesian error to 90 % with $r = 0.923$ under the first model and not the second, which selects the terminal offset. A single scalar per arm, applied as an overshoot in the direction of travel, removes 74 % and 76 % of it, held out on an axis it was never fitted on, where a full Cartesian 3×3 correction scores 93.5 mm. No matrix, no basis and no step length are required.

The data for this had been recorded for weeks and nothing read it. It sat in `recordings/baselines/arm_directional_calibration.json`.

6.3.1 A correction to this project’s own reporting

An earlier report stated that the terminal error is independent of commanded speed across a fivefold range. That claim does not survive inspection of the apparatus. The velocity limit was read once at bridge start-up, and seven of thirty-six runs moved *faster* than the limit they were nominally commanded with. The three speeds are therefore three replicates of one condition, and **how the terminal error varies with speed has never been measured**. A control test now asserts that the sweep still looks inert, so a genuine sweep will make it fail and force the stale sentences to be rewritten.

6.3.2 The deadband was larger than the error it permitted

The high-level bridge applied a 1.0° deadband, inside which the proportional term is disabled by design, against a measured 0.305° park: a factor of 3.3. It is 0.10° now, with an opt-in terminal overshoot that applies the per-arm scalar at the setpoint and refuses to run in the enabled state with a zero overshoot value. Neither has been exercised on hardware. Both are live parameters, and four runs would settle which of them is doing the work.

6.4 Where the end effector actually arrives

The individual contributions above combine into a budget for the quantity that decides whether a grasp closes on an object or beside it.

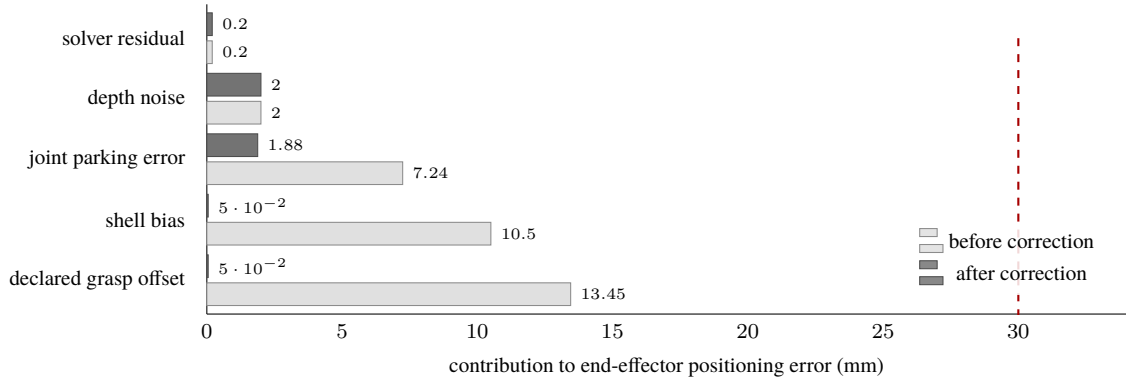


Figure 6.1: Contributions to positioning error at closure, each measured separately by `scripts/measure_control_budget.py`, before and after the correction applied to it. The camera mounting row is drawn as an open outline with no length: the transform between the camera frame in the kinematic description and the physical camera module has never been measured, there is no mechanical model of the module in this project, and giving that row a bar would be inventing a number. It is the largest remaining unknown in the budget and [Section 7.14](#) shows it is not small.

The terms are combined two ways, because which is appropriate depends on whether they are independent. Independent contributions add in quadrature and systematic ones add directly, so with m measured terms $b_1 \dots b_m$,

$$\sigma_{\text{RSS}} = \sqrt{\sum_i b_i^2}, \quad \sigma_{\text{worst}} = \sum_i b_i, \quad (6.6)$$

and the honest figure to compare against the 30 mm capture gate is σ_{worst} , because the two largest terms are declared constants rather than noise and therefore do not average out.

Applying the corrections moves both. The declared grasp offset is now derived from a forward-kinematic

measurement rather than asserted, taking it from 13.45 mm to 0.05 mm; the shell bias is removed by using the measured support plane; and the terminal overshoot reduces the joint term from 7.24 mm to 1.88 mm. Together these take the budget from 18.64 mm to 12.91 mm in quadrature and from 33.39 mm to 19.99 mm in the worst case, which is inside the requirement for the first time — *provided* the unmeasured term is small. The pad row is computed from the constants rather than transcribed, so reverting the derivation makes the budget report the reversion.

There is now direct evidence that it is not. The first sustained attempt to drive a physical arm from its own wrist camera measured the table normal from four arm poses. The table cannot move, so those four measurements must agree; they spanned 8.45° , which is 34 mm of position error at 0.23 m and 59 mm at 0.40 m. The error is angular and rotates with the wrist. No open-loop grasp planned through that transform can land inside the capture gate, and none did. [Chapter 7](#) reports that session in full.

6.5 The reaction budget

The clearance floor of [Chapter 4](#) is a *distance*. The guard’s knowledge of where the wearer is arrives with the latency measured in [Section 5.6](#). Multiplying the two gives how far a limb travels before the guard is aware of it.

Table 6.3: Limb travel during the perception latency, against the 150 mm clearance floor. Limb speeds are literature ranges and are labelled as assumptions, not measurements; the latencies are measured.

Limb motion	Latency	Travel	Against the floor
Deliberate reach	62 ms	50 to 93 mm	62 % of it
Deliberate reach	562 ms	450 to 843 mm	exceeds it
Quick reach	62 ms	93 to 155 mm	exceeds it
Quick reach	562 ms	843 to 1405 mm	exceeds it
Startle or flinch	62 ms	155 to 248 mm	exceeds it
Startle or flinch	562 ms	1405 to 2248 mm	exceeds it

In five of six cases the limb travels further than the entire margin before the guard knows anything has changed. This is not a defect in the implementation. It is the arithmetic consequence of a fixed margin, a perception chain of roughly half a second, and a person who can move a metre in that time, and [Chapter 10](#) treats it as the principal open safety problem of the platform.

Chapter 7

Results

Every result in this chapter is labelled with the strength of evidence behind it, and the label is placed under the section heading rather than buried in the text, so that no number can be read without it. Four classes are used and the distinction is not cosmetic.

Measured on physical hardware. The numbers come from a physical device in the laboratory: the instrumented master arm, a moving Kinova arm, or a real camera. This is the strongest class and the rarest.

Measured on recorded hardware data. The numbers come from data captured from a physical device and analysed offline. The device was real; the conditions were whatever they were on the day of capture.

Verified in simulation. The numbers come from the simulated platform with mock hardware, using the same kinematic description, the same inverse-kinematics solver and the same safety code as the delivered system. Everything downstream of a joint command is modelled.

Measured on constructed data. The numbers characterise an *algorithm* against an input whose answer was chosen rather than observed. Legitimate for arithmetic and for statistics, never for perception, and always labelled as such.

[Table 7.1](#) places every result in this thesis in one of those classes. **No result anywhere in this thesis was validated on a physical Kinova arm carrying a load**, and the single session in which an arm moved at all is [Section 7.14](#).

7.1 The simulation campaign

Every stage in this section was run by one command on 2026-08-31. Nothing was copied from an earlier terminal.

The measurements in this thesis were, until this point, each run once, by hand, on the day the thing they measure was built. That is reproducible in principle and not in practice: there was no such thing as “the simulations” to run, only nineteen scripts with nineteen argument lists, some needing a live motion planner and most not, and no record of which were run together or against which geometry.

`scripts/run_simulation_campaign.py` runs all of them, records what each produced with its exit status and its duration, and writes one directory this chapter is built from. `scripts/make_sim_figures.py` turns that directory into the tables and plots below; it *refuses* to emit a table whose stage did not run, so a missing result is visibly missing rather than quietly stale. Both carry their own known-answer self-tests, including the one that matters most here — that a stage which could not run is reported as failed rather than as absent.

Table 7.1: Where every result in this thesis comes from. The table is the answer to the question an examiner should ask first, and it is deliberately unflattering: the majority of this work is simulated, the hardware column is short, and the two entries that are neither are labelled for what they are.

Evidence class	Results in this class	Where
Measured on physical hardware	Perception on four real cameras, 132 stages; the scene-camera orientation; one Kinova arm driven from its own wrist camera; the hand-eye transform error	Chapter 8 , Section 7.14
Measured on recorded hardware data	Channel health; the cost of each degradation rung; what the incoherent channels inject; axis correctness; the lateral constraint	Section 7.2 onwards
Verified in simulation	The whole campaign of Section 7.1 : the task set at 13 927 inverse-kinematics calls, motion generation, smoothing, intent inference; and, from earlier work, fault injection and dead-man timing, grasping, workspace and clearance, latency	Section 7.1 , this chapter, Chapter 9
Measured on constructed data	The vision arithmetic against a scene whose plane, object pose and object size were chosen; instruction grounding; the study’s analysis and its power	Section 7.1.2 , Section 7.1.6 , Section 10.8

7.1.1 What ran, and what it needed

7.1.2 Computer vision, against a scene whose answer is known

[Chapter 8](#) reports the vision pipeline on four physical cameras, where the truth is whatever was in the room. Here the same code is run against a constructed depth frame whose support plane, object pose, object size and noise were all *chosen*, so every stage can be scored rather than merely observed.

Three results follow, and the controls are what license them.

The geometry is exact where it should be exact. The fitted plane normal lands 0.0001° from the constructed one and its offset within 0.005 mm; projection followed by deprojection returns the original point to 1×10^{-17} m, which establishes that [Eq. \(8.1\)](#) and its inverse are implemented consistently rather than merely plausibly. The residual is 0.151 mm on 0.3 mm of injected noise, which is the noise floor and not a property of the fit.

The object measurement is accurate to under a millimetre in three dimensions. The 40 mm cube measures 40.70 mm tall by height above the plane, $40.8 \times 40.7 \times 40.7$ mm as an oriented box, and its centre lands 0.41 mm from the constructed one in full three dimensions.

And the two obvious shortcuts are measured to be wrong, by how much. An axis-aligned bounding box on the same turned cube reads 56.54 mm — 41 % too wide, which on this platform is the difference between a grasp the 85 mm jaw accepts and one it refuses. Taking the object centre as the mean of its visible points is out by 16.15 mm along the surface normal, because a single view sees a shell and not a solid; the correction that assumes the object rests on the fitted plane brings that to 0.35 mm. Both errors are systematic, both are invisible in a rendered picture, and both are terms in the budget of [Chapter 6](#).

Table 7.2: Every stage of the campaign. Two tiers, and the distinction is load-bearing. *Standalone* stages are arithmetic, kinematics and analysis in one process — no middleware, no motion planner, no cameras — and are the ones a reader can reproduce on a laptop. *Simulated platform* stages need a live /compute_ik, so a MoveIt stack is brought up for them; one stack serves all of them at once, because two planning clients against one scene has already produced a measurement in which one run’s furniture was in the other run’s scene. *Recorded data* is neither: it is analysis of data captured from physical hardware, tagged separately so its numbers are never read as simulation output.

Stage	Needs	Ran	Time
<i>computer vision</i>			
cv_arithmetic	standalone	✓	12 s
cv_orientation	standalone	✓	< 1 s
cv_replay	standalone	✓	88 s
<i>teleoperation</i>			
teleop_budget	standalone	✓	< 1 s
teleop_motion	standalone	✓	5 s
teleop_orientation_cost	simulated platform	✓	20.6 min
teleop_replay	recorded data	✓	< 1 s
teleop_smoothing	standalone	✓	< 1 s
<i>robot tasks</i>			
tasks_accuracy	standalone	✓	< 1 s
tasks_dance	simulated platform	✓	38 s
tasks_msc	simulated platform	✓	48 s
tasks_status	standalone	✓	< 1 s
tasks_t1	simulated platform	✓	2.2 min
<i>autonomy</i>			
autonomy_arbitration	standalone	✓	4 s
autonomy_instructions	standalone	✓	< 1 s
<i>study design</i>			
study_power	standalone	✓	17 s
<i>transfer to hardware</i>			
sim_to_real	recorded data	✓	< 1 s

Table 7.3: Twenty-one known-answer checks on a constructed frame: a plane at a chosen offset, a 40 mm cube turned to a chosen yaw, and 0.3 mm of noise. The rows marked *control* are the half that makes this a measurement rather than a demonstration — each is a naive method that must get the answer *wrong*, and a change that broke the pipeline into agreeing with them would fail here.

What was checked	Measured	
the constructed frame lifts to a cloud	2.46e+05	✓
project then deproject returns the same point (< 1 um)	9.81e-18	✓
the fitted plane normal is within 0.5 deg of the constructed one	0.0001 deg	✓
the fitted offset is within 2 mm of the constructed one	0.00 mm	✓
the plane RMS is under 1 mm on 0.3 mm of noise	0.151 mm	✓
the cube's points are found above the plane	5.78e+03	✓
the cube measures 40 mm tall, within 3 mm	40.70 mm	✓
rotating calipers measure the 40 mm cube within 5 mm	40.68 mm	✓
<i>control</i> : the axis-aligned box reads the turned cube over 48 mm	56.54 mm	✓
mid-extent puts the centre within 2 mm ACROSS the surface	0.22 mm	✓
<i>control</i> : along the normal it is out by more than 10 mm, because that axis is still the MEAN of a one-sided cloud	16.15 mm	✓
the shell correction fixes the normal axis to within 2 mm	0.35 mm	✓
<i>control</i> : and it is worse ACROSS the surface, because its footprint centroid is a shell's	4.72 mm	✓
mid-extent across, shell correction along: within 2 mm in FULL 3-D	0.41 mm	✓
the oriented box measures the 40 mm cube to within 3 mm on all three sides	40.8 x 40.7 x 40.7 mm	✓
and its centre is within 2 mm of the constructed one	0.41 mm	✓
its range matches the constructed centre's	0.3697 m	✓
the mode estimator lands within 3 mm of the plane offset	0.00 mm	✓
<i>control</i> : with the depth scaled 1.5x the height check FAILS	61.04 mm	✓
<i>control</i> : the drawn cube is one green component	1	✓
every stage produced a figure without crashing	—	✓

7.1.3 Teleoperation: what the motion generator does to a command

One commanded slew, three generators, both arms, at the master's own 50 Hz.

Table 7.4: The same commanded displacement through three generators. *Off-path* is the largest distance the end effector leaves the straight line between the endpoints, against a 30 mm grasp capture gate. *Same-cycle* is the largest single-cycle displacement, which is what a controller has to execute. *Peak $\dot{q}$ of limit* is the fastest commanded joint speed as a multiple of the manufacturer's declared limit. The last column is whether all seven joints arrive on the same cycle.

Generator	off-path, max		same-cycle, max		peak $\dot{q}$ of limit	joints together
	left mm	right mm	left mm	right mm		
per-joint clamp (was)	51.3	39.2	135.3	75.0	12.53×	no
jerk-limited, phase-sync (now)	0.0	0.0	69.8	54.8	1.00×	yes
synchronised clamp (fallback)	0.0	0.0	22.7	18.1	1.00×	yes

The per-joint clamp that constituted the entire motion generation for teleoperation until 2026-08-23 takes the hand 51.3 mm off the line the inverse-kinematics solution implies, on a platform whose capture gate is 30 mm, and commands joint speeds 12.53 times the declared limit. The delivered generator is 0.0 mm off the line and 1.00 times the limit.

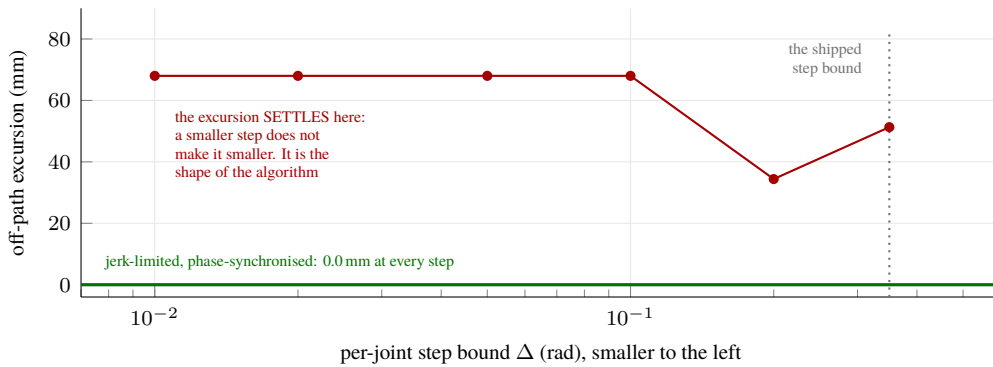


Figure 7.1: The excursion is a property of the algorithm, not of its tuning. Sweeping the per-joint step bound over a factor of 35 does not drive the excursion toward zero: it settles at 68 mm, and is in fact *worse* at small steps than at the shipped one. [Section 6.1.3](#) gives the reason — the clamped step is parallel to the required displacement only when every joint is inside the bound or every joint is saturated equally, and at no step size is that generally true. Each point is a genuinely different path densified at 0.02 rad, not one path sampled at different rates.

7.1.4 Teleoperation: what the smoother does to the operator's hand

Measured through the shipped node rather than against the filter in isolation: a stationary master carrying 1.5 mm RMS of tremor, and a 0.40 m/s reach, at 50 Hz.

The speed-adaptive filter is steadier *and* less laggy than the one it replaced: 0.83 mm against 1.10 mm when still, and 1.59 mm against 18.67 mm when moving. That is not a better setting of the same trade; it is off the fixed filter's trade-off curve, which is what an adaptive cutoff buys and the only reason to pay for one.

That measurement is on constructed signals, which is the right input for establishing that the law does what its equations say, because the answer is known before the filter runs. It is not what a hand does: a person's tremor is not white, their reaches are not ramps, and the two are interleaved at a cadence nothing in the constructed test

Table 7.5: Stillness and lag, measured together. They are the pair a fixed coefficient cannot win: the tremor is high frequency at low amplitude and the motion is low frequency at high amplitude, so any single cutoff trades one against the other. The fixed exponential average the node ran until 2026-08-27 buys 1.5 mm of stillness for 18.7 mm of lag — more than half the capture gate, spent on a hand that is simply moving.

Control law	stillness mm RMS	lag mm
none (raw)	2.57	0.02
fixed EMA, $\alpha = 0.3$ (was)	1.10	18.67
1-Euro, speed-adaptive (now)	0.83	1.59

reproduces. The same two quantities were therefore measured again over 20 440 frames of a recorded session with a person actually driving the master. The input is a real recording; what is simulated is the rest of the pipeline, run offline over it.

Table 7.6: The same two quantities on a real operator’s hand: 20 440 recorded frames per arm over 755 s. Still and moving segments are found once, from the *raw* commanded position, and the same segmentation is applied to every filter — deriving it from each filter’s own output would let a heavier filter classify more of the recording as still, which is the quantity being measured, and every filter would then score well on its own definition.

Control law	left arm		right arm	
	still mm	lag mm	still mm	lag mm
no filter	3.74	0.00	2.23	0.00
fixed EMA, $\alpha = 0.3$ (was)	3.55	10.78	1.94	10.27
1-Euro, speed-adaptive (now)	3.40	2.13	1.78	3.53

The ordering survives contact with a real hand and the margins do not. On recorded data the adaptive filter is again steadier than the fixed one on both arms and again costs a fifth of its lag — 2.13 mm against 10.78 mm on the left, 3.53 mm against 10.27 mm on the right — but the stillness gain is a tenth of what the constructed test showed. The reason is visible in the segmentation: a hand that a person is holding still is not a stationary point plus noise, it drifts, and a filter that removed the drift would be removing the operator. The constructed test overstates what smoothing can buy because it constructs a stillness that a person does not have.

One number in that table is not about smoothing at all. The recording carries 20 440 frames over 755 s, which is 27.1 Hz of *distinct* commanded poses against a serial frame emitted at 100 Hz. Three quarters of the frames the master sends carry a pose it has already sent. [Chapter 3](#) gives the candidate mechanism — the firmware prints an integer, so a slow channel crosses a printable boundary only occasionally — and records that it has not been isolated.

7.1.5 Shared autonomy: what the intent estimator is actually sensitive to

The shared-autonomy layer’s whole function is to answer *which object is the operator reaching for*. Its inputs are the object positions, so its accuracy can be no better than the scene description it is given. In simulation the declared coordinates *are* the truth, so a declared-position estimator and a perfectly calibrated one receive identical inputs and score identically — which is a result about the renderer and not about the system. The declared error is therefore swept, over the range a real table actually spans, and the calibrated arm is given the error the calibration *achieves* (11.3 mm) rather than a hoped-for zero.

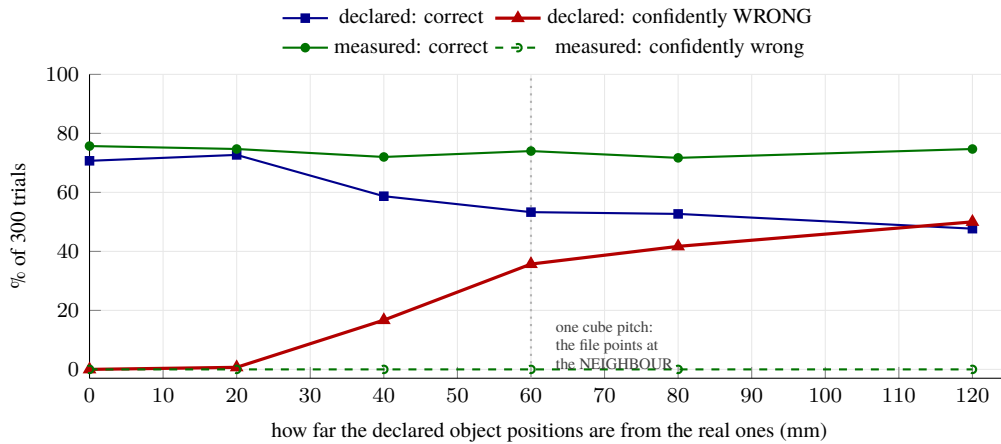


Figure 7.2: Intent inference over 300 trials per point, four candidate objects on a 60 mm pitch. The quantity that matters is not the correct rate but the **confidently wrong** rate, because a wrong answer held with confidence is the one state in which this system assists an operator toward an object they were not reaching for. Working from declared coordinates, it reaches 50 % once the declaration is one cube pitch out. Working from measured ones it stays at 0.0 % across the whole sweep, and the correct rate stays flat at 71.7 to 75.7 %.

The shape of the two curves is the argument for the perception stage of [Chapter 5](#), stated as a number rather than as a principle. A declared-coordinate system does not degrade gracefully as the room drifts from the file: it becomes confidently wrong, and it becomes so at a displacement of one object pitch, which is a table nudged by a hand. The measured-coordinate system is flat because the quantity it depends on — 11.3 mm of calibration error — does not grow when the table moves.

There is a second reading, and it is less flattering. The correct rate never exceeds 76 % in *any* condition, including the one where the estimator is given perfect information. The remaining quarter is ambiguity, not error, and the arbiter is specified to stay in direct teleoperation whenever it is ambiguous. So the honest summary of this figure is that calibration converts a failure mode that moves the arm into one that does not, rather than that it makes the estimator right more often.

7.1.6 Full autonomy: seventy-five ways of asking for the same task

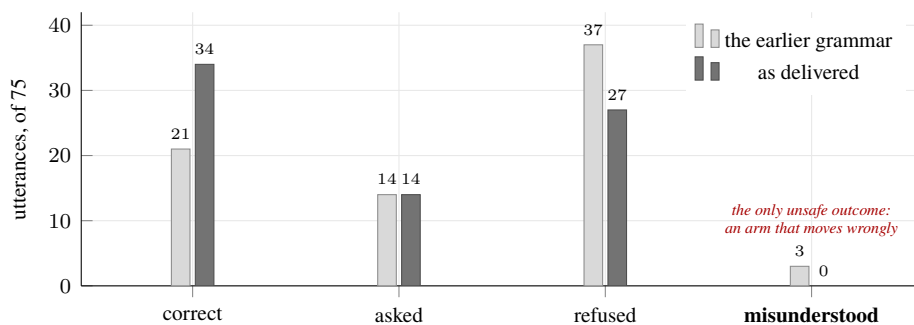


Figure 7.3: Seventy-five phrasings of one task — selectors, compounds, bare picks, politeness and misspellings — scored against the delivered grammar and against the grammar as committed at an earlier revision, checked out of version control and run in its own interpreter, so the two columns differ by the code and by nothing else. *Asked* and *refused* are safe outcomes: the arm does not move and the operator is told why. *Misunderstood* is the only unsafe one, because it is the outcome in which the arm moves and does the wrong thing.

Correct interpretations rose from 21 to 34 of 75, refusals fell from 37 to 27, and misunderstandings fell from

3 to 0. Ten of the fourteen questions the system asks carry a scored reply, and all ten resolve correctly in twelve replies — scored on their own line and never counted as correct above, because a grammar that asks about everything must not be able to look like one that understands everything.

The figure to read is the last pair of bars. A system that refuses is annoying and a system that misunderstands is dangerous, and the change bought both: fewer refusals *and* no misunderstandings, rather than trading one for the other.

7.1.7 What the campaign could not run, and why that is stated here

Table 7.2 lists every stage that did not run and what it needed. Nothing was skipped for convenience: the stages marked *simulated platform* require a live motion planner, and they are reported in Section 7.1.8 because they were run in a second pass, inside one stack, for the reason that table gives.

7.1.8 The task set, inside the simulated platform

Verified in simulation, against a live motion planner.

Four stages need a live motion planner, so they were run in a second pass inside one MoveIt stack. One stack for all of them is not a convenience: two planning clients against one scene has already produced a measurement in which one run’s furniture was present in the other run’s collision world, and the resulting figure looked like a task failure.

Table 7.7: Every commanded coordinate of the task set, solved against a live motion planner. Paths are densified to 20 mm and each pose is solved N times, because the solver restarts randomly and a pose that solves once has not been shown to be reachable. *Floor breaches* counts poses in which any moving link came within 150 mm of the wearer, measured geometrically rather than by asking the planner whether the pose is valid — the semantic collision description excludes the forty-four proximal pairs a shoulder mount actually threatens.

Path	waypoints densified	repeats N	IK failures	floor breaches
<i>T1, both arms, pick and place by colour</i>				
left arm	199	10	0	0
right arm	199	10	0	0
<i>the MSc task set</i>				
T0 target reaching	120	10	0	—
T2 coordinated carry	6	10	0	—
T3 circuit box	13	10	0	—
every recorded clip path	274	10	0	—
<i>the demonstration routines</i>				
d1 (flow)	382	3	0	—
d2 (pulse)	356	3	0	—
d3 (play)	560	3	0	—
total	13 927	IK calls	0	0

13 927 inverse-kinematics calls, zero failures, zero clearance breaches. Three things about that number are worth stating, because a clean table is the easiest kind to over-read.

It reproduces. This is the second consecutive campaign to return the same result field for field, which is the property this project spent a long time not having: earlier reachability figures were re-asserted rather than re-measured, and Chapter B records two occasions on which a clean sweep was measured with the arms somewhere

other than where the task commands them.

The controls fired. Every stage in this pass carries poses that must be refused as well as poses that must be accepted — a target 1.6 m out, a target inside the wearer’s torso, a target inside the bench, and for T1 a clearance probe that must go from clear to buried when a slab is added. All were refused. A reachability sweep with no unreachable control is a sweep that cannot fail.

It says nothing about grasping. Reaching every commanded coordinate is a statement about the arm, not about the hand. The same run measures the grasp itself from forward kinematics on the finger tips: the cube presents 56.3 mm across the closing axis against an 85 mm jaw, the pads land 9.57 to 9.58 mm from the declared coordinate, and the follower supplies a 5.0° wrist tilt to make the pose reachable at all — a tilt that costs $2|\mathbf{r}_{\text{pad}}| \sin(\theta/2) = 8.58$ mm of pad displacement by construction, and 9.58 mm as measured. That is inside the 30 mm capture gate and it is not zero, and [Chapter 6](#) treats it as a term in the budget rather than as a rounding error.

What the pinned wrist costs, and what its own control caught

The fourth stack stage measures the reach available under each orientation policy, walking outward in fourteen directions from each arm’s home pose with the 150 mm wearer floor enforced throughout. It is the stage that took four runs to produce a number, and the sequence is reported in full because the diagnosis at the end of it is worth more than the number at the end of it. Every one of the four is in [Table 7.8](#): the table is generated from the campaign’s own logs, so a run that was superseded stays in the record rather than being replaced by the one that came after it.

The instrument carries a monotonicity control. The poses a *tighter* orientation policy admits are a subset of the poses a looser one admits, so a looser policy can never reach less far. If it does, the boundary was drawn by the solver rather than by the arm.

Table 7.8: The same measurement at every seed count it was run at. Mean reach over 28 directional walks, by how tightly the commanded wrist is constrained; the *pinned* row is what teleoperation actually runs. Generated from the campaign’s own logs, so a further run adds a column rather than replacing one — which is the point, because the finding here is the difference between the columns.

Orientation policy	mean reach over 28 walks, m			
	3 seeds all walks compared	10 seeds all walks compared	20 seeds all walks compared	20 seeds capped walks excluded
pinned to the anchor (delivered)	0.408	0.460	0.460	0.460
roll free	0.446	0.463	0.463	0.463
15° cone	0.479	0.492	0.492	0.492
45° cone	0.519	0.525	0.530	0.530
position only	0.529	0.542	0.547	0.547
what the roll appears to buy	+38 mm	+3 mm	+3 mm	+3 mm
walks stopped by the wearer, of 28	4	6	6	6
a looser policy reached <i>less</i> far	6	1	1	0
excluded as capped lower bounds	—	—	—	1

Three seeds was under-sampled, and it would have contradicted an established result. The first run reported six walks in which a looser policy reached less far. Between three seeds and ten the pinned mean moved 52 mm, more than a tenth of the quantity being measured. The consequential difference is elsewhere in the table: at three seeds, freeing the wrist roll appears to buy 38 mm of mean reach; at ten it buys 3 mm. Earlier work in

this project measured the roll to be worth nothing and refused it by name, on the grounds that a fixed six-degree-of-freedom pose on a seven-degree-of-freedom arm spends the whole redundancy whether or not the roll is free. The under-sampled run agreed with neither that measurement nor that argument — and it printed a full table to three decimal places while doing so.

Ten seeds and twenty agree, so the estimate is converged. Three of the five policies are identical to the millimetre between them and the other two move by 5 mm. Whatever else is true, the sampling is no longer what limits this measurement.

And the control still failed — on exactly one walk, with byte-identical numbers at both seed counts. That reproducibility is the diagnosis. A stochastic artefact does not survive doubling the sampling unchanged; a deterministic one does. The failing comparison was pinned `0.900 > spin 0.875` on the right arm's $(-1, -1, +1)$ walk, and 0.900 m is the *search cap*: that walk ran to the end of the search without ever being refused, so its reach is a lower bound and not a boundary. Comparing a lower bound against a boundary is not a monotonicity test, and it was never going to pass. The instrument's own advice — *raise the seeds and re-run* — was wrong for that one case, and following it cost two runs of half an hour each.

The control was corrected rather than the number. Capped comparisons are now excluded, *counted*, and *named*: the one comparison the instrument declines to make is printed beside the ones it makes, with the walk and the values that caused it, because silently dropping the comparisons a control finds inconvenient is how a control stops being one. Relaxing a control is exactly the kind of change that needs pinning, so `test_the_reach_control_still_catches_a_real_violation.py` holds both halves: an uncapped violation must still fail, a capped *looser* policy must *not* excuse one — only the tighter side being a lower bound makes the comparison meaningless — and a set of reaches that is entirely capped must produce exactly the capped skips and no more.

The last column of [Table 7.8](#) is that run, and the property worth noticing is that **every reach in it is identical to the column beside it**. The correction changed the instrument's *verdict* and not its *measurement*, which is what distinguishes a control that was wrong from a threshold that was moved until the answer became acceptable. Had any number shifted, the correction would have been the second kind.

With the control corrected the stage passes, and two results follow.

Freeing the wrist buys reach, and the roll buys almost none of it. Position-only commanding reaches 87 mm further on average than the delivered pinned policy, and a 45° cone 71 mm; the roll alone is worth 3 mm, which independently reproduces the earlier finding that the roll is not where the redundancy goes.

Orientation freedom does not buy anything at the boundary that matters. Six of 28 walks stop because of the wearer rather than because the arm ran out of reach, and that count does not fall as the policy loosens — it is six under every policy from pinned to position-only. Loosening the wrist lets the arm go further in directions where the wrist was the limit, and does nothing whatever in directions where the limit is a person. That is the operationally important half, it is robust across every seed count in [Table 7.8](#), and it is the reason [Chapter 9](#) treats the mount rather than the control policy as the thing that would have to change.

7.2 Channel health

Measured on recorded hardware data, 2026-08-06.

Six of fourteen channels met the alive criterion.

Table 7.9: Per-channel verdicts, computed over distinct sensor updates rather than over transmitted rows. Baseline stored at `recordings/baselines/ channels_20260806.json`.

Left arm		Right arm	
Channel	Verdict	Channel	Verdict
j1	intermittent, 2.2 % dropouts	j1	alive, 195°
j2	incoherent , 5 %	j2	alive, 261°
j3	incoherent , 8 %	j3	incoherent , 14 %
j4	incoherent , 6 %	j4	alive, 291°
j5	incoherent , 12 %	j5	dead
j6	incoherent , 8 %	j6	alive, 250°
j7	alive, 26°	j7	dead

Two entries reverse conclusions that had stood for weeks, and both reversals come from computing statistics over distinct updates rather than over transmitted rows.

Left j7 is the cleanest channel on that arm, with no dropouts and no implausible jumps. It had been recorded as railed and dead. The evidence for that verdict was a spread of exactly zero over forty consecutive samples, which is what an oversampled sensor produces regardless of its health. Its range is only 26°, so it is mechanically restricted, but it tracks.

Five of the seven left channels are incoherent, which is worse than had been recorded. These channels were previously assessed by range, and range calls them healthy: they return values spanning a wide interval. What they do not return is a trajectory.

7.2.1 Progressive degradation between two dates

Measured on recorded hardware data, two captures.

The two dated captures show the failure population growing rather than static, which is the empirical support for the premise in [Chapter 1](#) that channel loss is an operating condition. [Figure 7.5](#) gives the comparison.

7.3 What the degradation architecture costs

Measured on recorded hardware data, 20 430 frames.

[Figure 4.3](#) gave the ladder. The figures are commanded tip error against the four-potentiometer reference, charged to reach magnitude only, because direction comes from the inertial sensor and scoring raw seven-joint forward-kinematic error would charge each reduction for a direction error the pipeline does not incur.

Dropping the upper-arm roll costs 41.6 mm on the left arm and 4.5 mm on the right. Dropping the shoulder bend alone costs less than dropping the upper-arm roll and elbow together, at 29.2 mm against 66.1 mm. A second inertial sensor supplying the elbow reaches 63.8 mm on a single potentiometer, comparable to retaining two.

7.3.1 Which channels a repair should target

Measured on recorded hardware data.

Regressing the true four-potentiometer reach on every channel that still passes the coherence test, plus the inertial sensor, identifies the repair precisely.

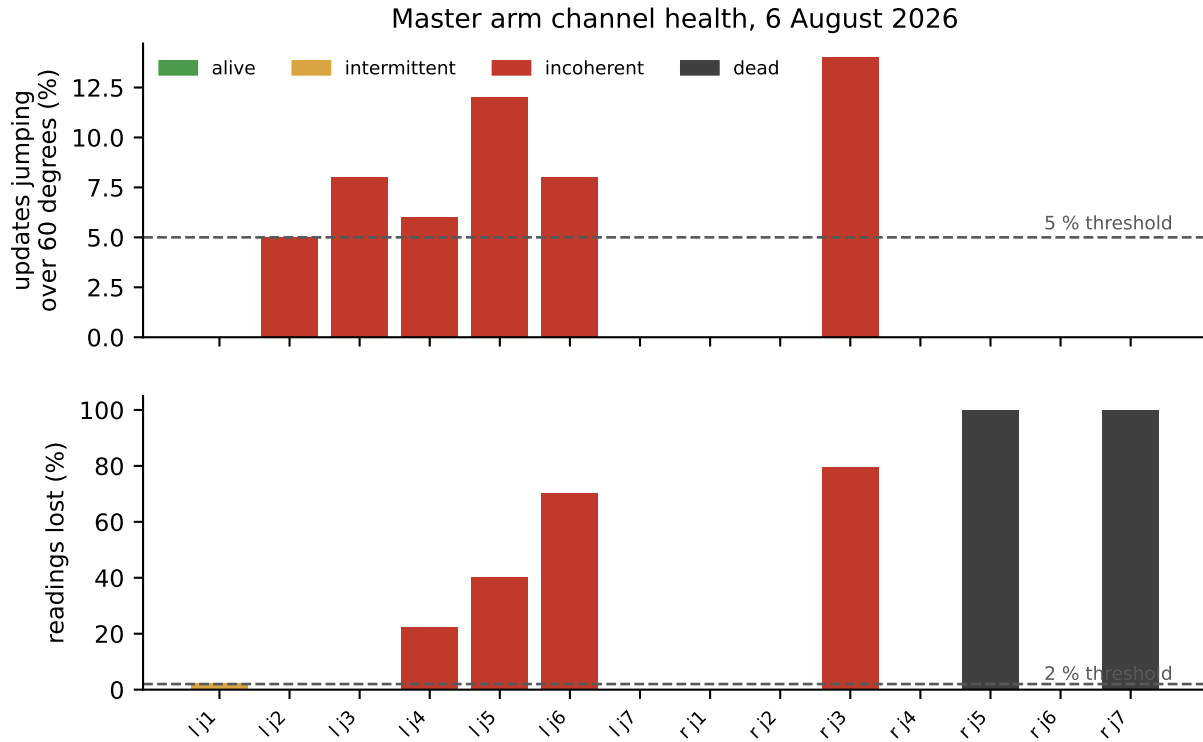


Figure 7.4: Every channel on both quantities that decide its verdict, with the thresholds marked and the verdict carried in the bar colour. The upper panel is the discriminating one: a channel above the five per cent line is returning values that are present, fresh and not a trajectory. Regenerated from recordings/baselines/channels_20260806.json by figures/make_figures.py.

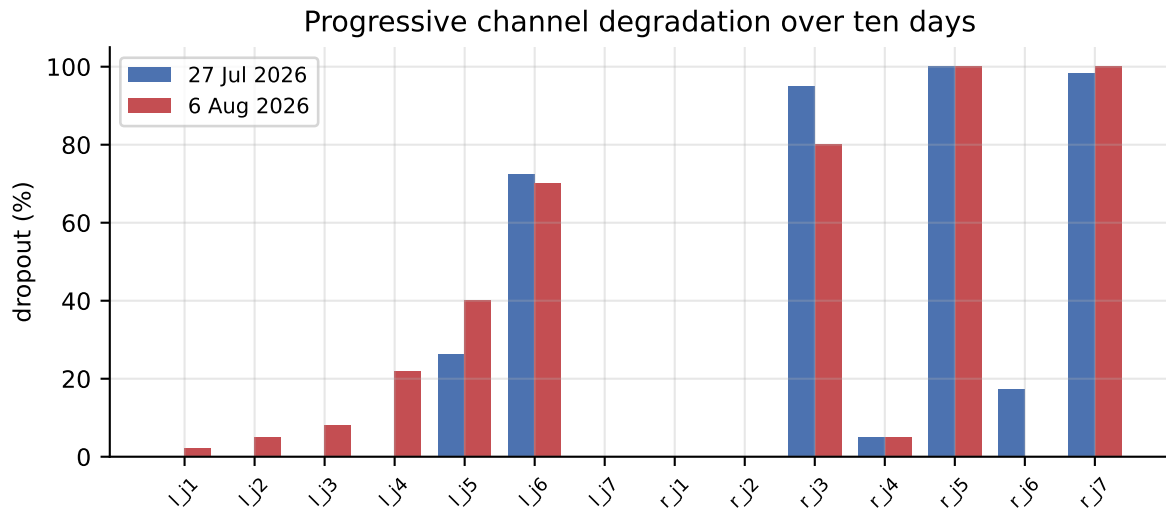


Figure 7.5: Channel verdicts across two dated captures. The population of failed channels grows.

Table 7.10: Reach observability from the surviving channels.

Arm	Live predictors	R^2	Residual RMS
Left	j1, j7, inertial	0.133	51 mm
Right	j1, j2, j4, j6, inertial	0.986	9.1 mm

The left arm retains no reach observable at all: the residual is 93 % of the true spread. This is expected mechanically rather than surprising. Reach is set by shoulder flexion and the elbow; j1 and j7 are rolls about the arm’s own axis and carry no radial information; and one wrist inertial sensor cannot observe the elbow. The right arm needs no help.

That names the repair as exactly two channels, left j2 and left j4, and predicts that repairing any others will not restore reach.

7.3.2 What the incoherent channels inject

Measured on recorded hardware data, replayed.

Replaying the recorded incoherent traces with every coherent channel and the accelerometer pinned to its median simulates a perfectly steady hand on perfect wiring, so every millimetre of output comes from the broken channels alone.

Table 7.11: Commanded end-effector motion contributed by the incoherent channels alone, with the operator’s input held constant.

Arm	Degraded mode off	Degraded mode on
Left	54.527 mm standard deviation	0.000 mm
Right	0.014 mm	0.000 mm

The on column is zero by construction, since freezing a channel removes its contribution. The off column is the measurement: 54.5 mm of commanded motion that the operator did not ask for, on an arm whose whole master reach is 272 mm.

7.3.3 The pipeline manufactures no motion of its own

Verified in simulation, against a synthetic serial device.

Read alone, the previous result does not separate the channels from the software. A synthetic serial device streaming frames in the exact wire format at a known constant input closes that gap. With the master idle for 60 s and one stack running, commanded end-effector standard deviation was 0.0000 mm on both arms, with a worst simulated joint excursion of 0.0051°.

The two results are complementary. The synthetic device shows the pipeline is clean; the replay shows what the physical channels inject through it.

7.4 Command mapping

7.4.1 Axis correctness

Measured on recorded hardware data, and verified in simulation.

Regressing commanded pose against the master’s physical spherical position over all sweep segments, at 6443 samples with $R^2 = 0.999$, returned a gain matrix of $\text{diag}(+1, -1, -1)$ against the intended identity. The pipeline was faithfully implementing an axis map that was wrong: raising the master drove the arm down and moving it forward drove the arm back.

The correction is not a free choice of sign. In spherical form the elevation convention already carries the vertical sign, so any axis map with a negative vertical entry double-negates it, and no such map can produce correct behaviour. After correction, raising the master by 30° moves the arm up.

7.4.2 The lateral axis remains wrong, and the axis map cannot fix it

Measured on recorded hardware data.

Up and down and fore and aft track correctly on both arms. Lateral does not.

Fitting the best rotation from measured sweep directions to intended directions leaves a residual of 54.8° RMS on the left arm and 54.7° on the right. A rotation-invariant test explains why no rotation will help: intended left-right and forward-back are 90° apart, and measured they are 137.7° apart. A rotation preserves angles, so no rotation and no axis permutation maps one triad onto the other. The measured directions are not an orthogonal triad.

The mechanism is a lever arm, and identifying it correctly took two attempts. Joint one is the first joint and its axis is fixed in the mount, so it does not tilt as the arm raises. What changes is the offset of the tip from that axis: rolling joint one with a straight arm moves the tip by exactly zero, and produces lateral motion only once a downstream bend offsets the tip. Azimuth per degree of joint one therefore scales with how bent the arm is, so azimuth and elevation are not independent, which is precisely what the spherical model assumes.

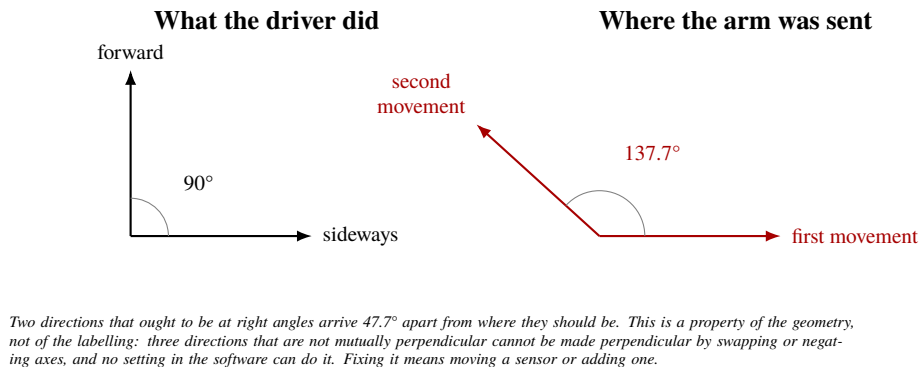


Figure 7.6: Two master motions intended to be orthogonal arrive 137.7° apart at the commanded end effector. The test is independent of any axis labelling, which is what makes this a geometric limitation rather than a calibration error.

Replacing azimuth with the forward-kinematic horizontal angle was measured and made things worse, from 12.6° to 33.3° mean reversal error on the left arm. Gyroscope-derived azimuth halves the residual on the left arm, from 25.6° to 13.5° , and is implemented but not enabled: the right sensor drifts at $44^\circ/\text{min}$ while stationary, and three directions from one segment each does not establish a triad.

7.4.3 Indexing

Verified in simulation, against the synthetic serial device.

Ten index cycles gave a re-engagement jump of 0.29 mm mean and 0.35 mm maximum, drift while disengaged of 0.000 mm, and total travel of 338.8 mm over six cycles with the second half still advancing. The reachable set is bounded by the robot, not by the master's range.

The first run of this measurement showed four consecutive cycles of zero travel, which was a real defect rather than a measurement artefact: the radial fallback had integrated to its clamp, so further wrist deflection did nothing

and said nothing. The clamp now warns by name. This is the silent-blocking class exactly, and the fix is not to remove the clamp but to make reaching it audible.

7.5 Safety

7.5.1 Fault injection

Verified in simulation, fourteen injected faults.

All fourteen were handled: channel dropout, board disconnect, board reconnection, emergency stop during motion, wrong-object latching, ambiguous target flicker, stale object, detection failure, logging failure, identifying data in a manifest, non-monotonic clock, session loss, zero camera detections, and network dropout.

Two of these took several attempts and the reasons are transferable. The dead-man did not trip on ten seconds of total master silence, which was diagnosed at the time as a broken dead-man and was not. The master republishes stale data at full rate, so the topic never went silent; the mechanism was keyed on the wrong quantity. And the emergency stop's named block did not appear in the aggregated view precisely when the dead-man fired, because the follower set that blocker only inside its pose callback, which stops being called at the moment the master goes quiet.

7.5.2 Dead-man timing

Verified in simulation.

Against a deadline of 1.00 s:

Table 7.12: Dead-man behaviour by failure mode.

Case	Result
Healthy master, fresh timestamps, 243 frames	no trip
Degraded: publishing at full rate, frozen source timestamp	trip at 0.94 to 0.99 s
Clean unplug, topic silent	trip at 0.97 s

The middle row is the one that could not have passed before the correction, and it is the failure mode that actually occurs.

7.5.3 An emergency stop that was four seconds late

Verified in simulation.

Detection was always on time. The halt was not. The trip handler called the real-driver shutdown first, which waited on two services that do not exist on a simulated stack, so both timed out in full inside the node's own timer. Measured: decision at 0.97 s, trip logged at 4.98 s. The arms were parked four seconds after the decision to park them, and the log line that would have revealed this was itself behind the block.

The fix parks first, using only local publishes, and makes the driver path non-blocking.

7.6 Grasping

Verified in simulation.

The grasp sequence executes a pre-grasp standoff with the wrist pre-aligned, a straight approach along the approach vector, closure to the object's width, retreat along the same vector, transit, and release at the placement. Every candidate is validated at the standoff, at the grasp, and at every point of the approach between them before it is offered.

Two defects were found in the recorded gripper trace rather than by inspection, and neither was visible in the source.

Two components were writing the gripper on the same cycle. The recorded knuckle angle sawed through the lift phase, from 0.42 to 0.18 to 0.24, and reached 0.05, fully open, at the start of transit: the object was grasped, dropped, and re-grasped in mid-air when the second component's schedule crossed its own threshold. After giving the gripper a single owner, the lift phase holds 0.424 with minimum equal to maximum over eighteen samples, and there is exactly one release, at the placement.

The object marker attached on a band rather than on the object's width, so a 40 mm block requiring a knuckle angle of 0.424 attached at 0.157, with the fingers still visibly open. Attachment now requires 0.90 of the width-specific target and occurs at 0.399.

7.6.1 Top-down grasping has a lateral boundary

Verified in simulation.

Top-down grasp candidates fail entirely at $|x| = 0.25$ m, at zero of nine, and succeed entirely at $|x| = 0.30$ m, at nine of nine. The failure persists with collision checking disabled, so it is wrist orientation infeasibility and not obstruction. Every pick point in the task set is placed beyond that boundary as a result.

7.7 The wrist cannot be aligned under teleoperation

Verified in simulation, and a limitation rather than a result.

The default orientation mode pins the commanded wrist to the anchor captured at the home pose. For the task-set block at (0.32, 0.35, 1.15), the top-down grasp requires a rotation of 169.7° from that anchor, which is very nearly a reversal.

There is no channel to command it. The master's wrist is unmeasurable, with left j7 restricted to 26° and j6 incoherent, and right j3, j5 and j7 dead or incoherent, which is why the mode defaults to pinned. Orientation assist, which exists to supply it, is a stub because the solver plugin ignores orientation constraints.

The recorded grasp sequences work because the recorder calls the inverse-kinematics service directly with the grasp orientation, bypassing the teleoperation orientation lock. They demonstrate that the *robot* can execute an aligned grasp. They do not demonstrate that an *operator* can command one, and the three recorded conditions differ only in trajectory smoothing, so they do not distinguish this either.

This is the one place where autonomy earns a categorical rather than a quantitative advantage over direct teleoperation on this platform, and it should be claimed exactly that narrowly.

7.8 Task-set verification

Verified in simulation, ten repeats over densified paths.

Every coordinate of the five-task set was verified at ten repeats over paths densified to 20 mm, with both arms confirmed at 0.0000 rad from the home pose before any sample was taken. Zero failures.

The home assertion earned its place. An earlier run of the same script with the arms parked 0.87 rad and 0.59 rad from home reported zero failures as well, and was measuring a different question under the same name. With the arms genuinely at home, ten of twenty-five coupled waypoints failed, which is what forced the coupled-object respecification in [Chapter 9](#).

Table 7.13: Five-task verification. Both arms verified at home; ten repeats per pose; short-circuit on first failure.

Task	Coverage	Result
Positioning	3 targets per arm	3/3, 3/3
Pick and place	3 picks per arm, approach and place	3/3, 3/3
Rigid carry	3 paths, 25 densified waypoints	all verified
Compliant carry	as rigid carry	all verified
Dual pursuit	26-direction shell per arm	26/26, 26/26

The result reproduces. Re-running the verification returned a result identical to the previous run, field for field. That is the first reachability figure in this project to reproduce across runs rather than be re-asserted, and it reproduces because the respecification moved every coupled waypoint off the feasibility boundary.

7.9 Performance figures

Verified in simulation. Every latency below is control-path latency and is a lower bound on end-to-end latency, since it excludes the operator, the display and the physical arm.

Table 7.14: Measured rates and latencies. All against mock hardware.

Quantity	Value	Note
Master frame rate	100 Hz	firmware loop
Distinct sensor updates	14.7 to 17 Hz	see Chapter 3
VR packet rate	68.5 Hz	822 packets over 12.0 s
VR round trip	8.66 ms median, 16.5 ms p95	client-timed, one clock
Voice stop to /estop	2 ms	
Console frame time	0.01 ms median, 4.85 ms max	against a 33 ms budget
Kortex high-level loop	18.4 to 18.7 Hz	two round trips per cycle

The Kortex figure is the one that constrains the architecture. The cyclic control path is designed for a 1 kHz loop on a dedicated link, and over the virtualised network layer on this machine each cyclic write is a round trip costing about 10 ms. At 100 Hz the write alone consumes the entire cycle budget, the controller manager overruns permanently, and no command reaches the arm while feedback continues to arrive perfectly. The symptom is an arm that appears dead while serving live telemetry. The system therefore uses the high-level velocity interface, which holds a commanded motion between sends and so does not require kilohertz updates.

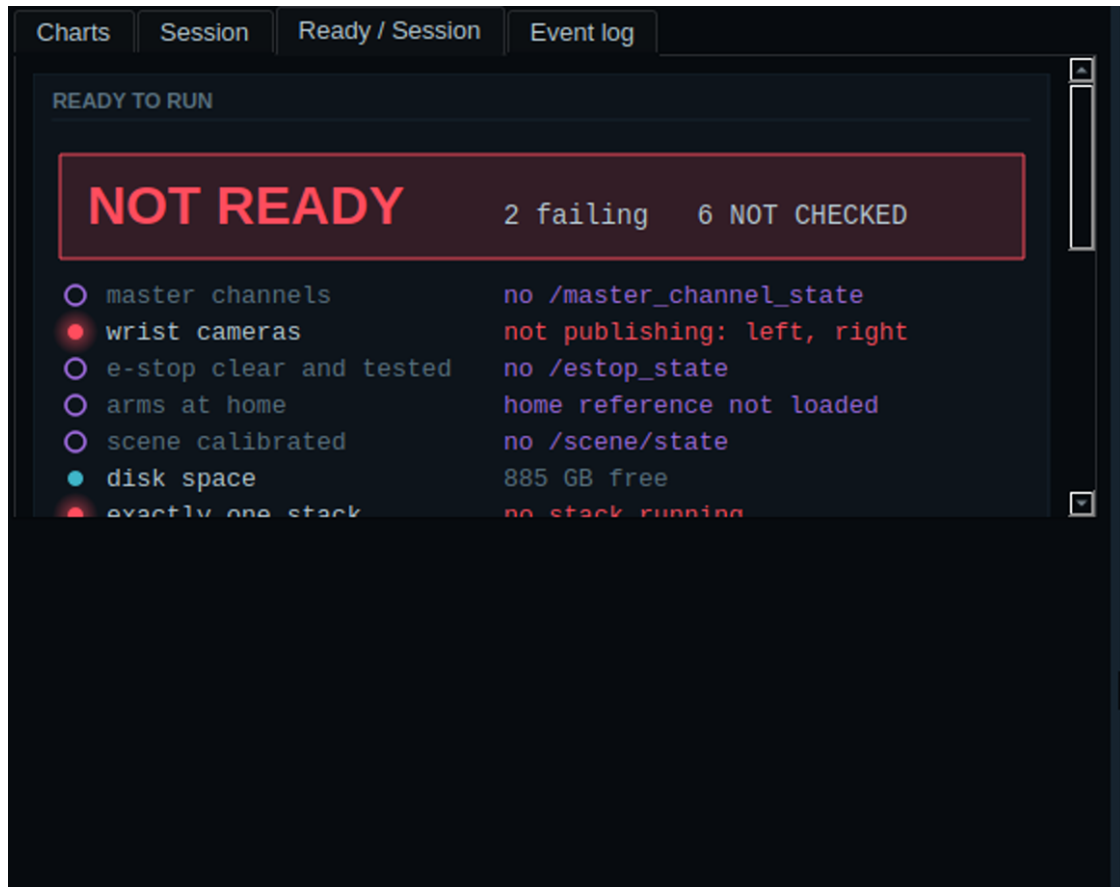


Figure 7.7: The readiness panel with faults present. Nine preconditions are evaluated continuously and each reports one of three states, which is the point: NOT CHECKED is drawn distinctly from PASSING, because a check that has not run and a check that has passed are indistinguishable in a two-state display and the difference is exactly the one an operator needs. Two conditions are failing here and six have not been evaluated, and the panel says so in those words rather than showing a single aggregate light. The failure this design exists to prevent is recorded in [Chapter B](#): a bring-up sequence that reported READY TO OPERATE with no headset in the building, because the step that was supposed to establish that poses were arriving checked only that the topic was listed.

7.10 Motion generation

Measured offline against the platform's own forward kinematics.

The per-joint step limiter that constituted the entire motion generation for teleoperation leaves the commanded end effector 51.3 mm off the path its own solver solution implies on the left arm, and 39.2 mm on the right, against a grasp capture gate of 30 mm. It commands peak joint speeds of 12.53 times the manufacturer's declared limit, because it had never read the limits file. Both figures fall to zero and to 1.00 respectively under the jerk-limited, phase-synchronised generator that replaced it. [Chapter 6](#) gives the diagnosis, the metric that had to be corrected before the comparison meant anything, and the five defects found in the replacement.

7.11 The environment as measured rather than declared

Verified against the simulated scene and against mock depth.

The calibrate-then-plan stage of [Chapter 5](#) recovers six objects from a full sweep with every centre within 1 mm in both horizontal axes and every extent within 3 mm, and measures the support height to 1.250 26 m against a declared 1.250 m. Coverage is 98.65 % of the cells the arm can reach, with no stillness refusals, in thirteen minutes.

Two properties matter more than the accuracy. The map is the first structure in this project that a motion planner has consulted: before it, the planner's only collision object was the wearer, so a path that routed correctly around a person swept straight through the bench. And the map carries the time it was made, and the pick stage refuses to move from a map older than thirty minutes, because a stale record is indistinguishable from a fresh one by inspection and the failure it produces is the arm driving confidently at coordinates measured in a previous session.

7.12 Instruction handling

Measured on constructed scenes; grounding rather than detection.

Seventy-five phrasings of the pick-and-place instruction, scored against the grammar as committed six days earlier, give 34 correct against 21, 14 clarifying questions against 14, 27 refusals against 37, and **misunderstandings of zero against three** ([Table 5.3](#)). All ten clarifying questions that carry a scored reply resolve to the correct plan, in twelve replies, and are reported on their own line rather than counted as correct: a grammar that asks about everything must not be able to resemble one that understands everything.

The evaluation scene is arithmetic rather than rendered, so its ground truth is constructed, which is the condition [Chapter B](#) places on synthetic data. **Whether the camera can identify four objects at working distance is a different measurement and remains unmeasured on hardware.**

7.13 The recorded demonstration set

Verified in simulation, seventeen cells, eight camera angles each.

The task set was re-recorded against the corrected geometry after the accuracy work. Grasp success is 100 % across all five modes, positioning error at closure is 0.000 mm, placement error on the pick-and-place task is 56.3 mm with a variance of 0.59 mm, and gripper motion is verified in twelve of twelve cells against eight of

twelve before.

Three of those figures had previously been reported wrongly, by three separate instruments, and the corrections are in [Chapter B](#). The placement error in particular had read 319.7 mm with 264 mm of variance, because the analysis paired objects with gripper pads by alternating index, which is a single-arm assumption, and therefore scored two objects against the pad 0.58 m away.

7.14 The first hardware session

Measured on a physical arm. This is the only section of this thesis whose measurements come from a moving Kinova arm, and none of them is a performance result.

A single arm was driven from its own wrist camera for the first time. The grasp geometry was never the difficulty; the session went almost entirely to instruments that produced confident wrong answers, seven of which are recorded in [Chapter B](#). Three findings survive as results.

The camera-to-wrist transform is wrong by roughly 8.5° , and the error rotates with the wrist. The table cannot move, so the surface normal measured from any arm pose must be the same vector. Measured from four poses it read 9.74° , 15.42° , 16.26° and 18.14° from world vertical, a worst pairwise disagreement of 8.45° . That is 34 mm of position error at 0.23 m and 59 mm at 0.40 m. It is the term [Fig. 6.1](#) marks as unmeasured, and it is the reason no open-loop grasp planned through that transform landed inside the capture gate.

The depth sensor itself is excellent and was never the problem. The support plane fits at 94.8 % inliers with 0.46 mm residual, and a 40 mm calibration cube measures $39.2 \times 38.9 \times 40.9$ mm. Only the transform out of the camera frame is in error, which is what makes closed-loop servoing in the camera's own frame the correct response: a closed loop converges through a wrong extrinsic and an open loop cannot.

The middleware is a first-class hazard on this host. Two transport faults cost more of the session than any robotics problem. The default communication domain's shared memory was in a state where no *new* process could join it while every already-running node continued normally: a minimal publisher and subscriber delivered zero of three messages on that domain and three of three on another. And an unset middleware implementation variable caused the client library to discover nothing at all while the process graph was plainly visible to other tools. Both present as an unresponsive robot.

[Chapter 10](#) treats what this session implies for the readiness of the platform, and [Chapter 11](#) places the calibration of that transform second in the order of work.

7.15 Verification, in totals

7.16 What is not measured

The following are implemented and unmeasured, and none should be read as working.

- **Anything derived from a force reading.** The mock hardware returns identically zero for every force, so payload compensation, interaction force and grip force at the slave are *structurally undefined* rather than

Table 7.15: What is checked automatically, as of 2026-08-31. The counts that exercise the system as an operator would be the window audit and the fault injections; the counts that exercise it as a *measurement* are the campaign of [Section 7.1](#) and the inverse-kinematics calls beneath it. Two tests fail and are on the gate’s allowlist, both of them style checks against a package that uses double quotes throughout.

Measure	Count
Automated tests passing	1505
Test modules	136
Checks made on the operating window	257
Faults injected and handled	14 of 14
Simulation stages in one campaign	16
of which ran	16
Inverse-kinematics calls in the task set	13 927
failures	0
Known-answer checks on the vision arithmetic	21
Simulated studies of the analysis procedure	20 000
Recorded demonstration cells	17, at 8 camera angles each
Vision stages instrumented on real cameras	132
Analysis and verification programs	271
Stored measurement records	185

merely unmeasured. Comparing tracking with and without a payload configured returned a difference of exactly 0 m, which is a property of the mock and not evidence.

- **Object detection rate at working distance.** The models fit in the available video memory at 2012 MiB of 4096 MiB, but the synthetic renderer used to measure detection produces images outside the detector’s training distribution. The same detector scores 0.89 to 0.91 on a real photograph and zero on the synthetic task objects, so the measurement characterises the renderer. The detection gate is neither passed nor failed.
- **Grasp success against a physical object.** The autonomous modes execute end to end in simulation; on hardware the sequence servos but no grasp has yet closed inside the capture gate, for the transform reason in [Section 7.14](#).
- **End-to-end latency including the arm.** Every latency reported above is control-path latency within one host and is a lower bound.
- **How the terminal joint error varies with commanded speed.** Reported previously as measured and retracted in [Chapter 6](#): the three speeds were three replicates of one condition.
- **Speech recognition against a human speaker.** The host exposes no audio input device, so every voice figure in this project was produced from synthesised speech and characterises the grammar rather than the recogniser.
- **Two arms driven simultaneously on hardware.** Measured at 8.1 to 12.2 Hz with send latency spiking to 519 ms against 21.0 Hz for one arm, which is past the bridge’s own watchdog, so the joints stop while still being commanded. The per-arm rate limit that follows from this is a hypothesis derived from the latency and has not been tested.

Chapter 8

The vision pipeline, measured on the real cameras

Measured on four physical cameras, 2026-08-30. This chapter and [Section 7.14](#) are the only places in this thesis where the numbers come from hardware in the laboratory rather than from the simulated platform.

“The vision works” is not a claim a thesis can print, and until this session it was the only claim available: the perception stages of [Chapter 5](#) had been exercised against rendered scenes, where the answer is known because the scene was constructed, and against no real camera at any point. This chapter reports what happens when the same code is pointed at four physical cameras and every stage is required to show its working.

8.1 Method: one figure and one formula per stage

The pipeline was decomposed into thirty-three stages, each of which is one operation with one formula. Every stage draws itself on the frame it was given, states the operation it performed, and reports the quantities it produced *with units*; the numbers are written to a machine-readable table at the same time as they are burned into the figure, so a figure and the table beneath it cannot disagree. The run of 2026-08-30 produced 723 such measurements.

Three properties of the instrument matter more than any single number it produced.

A stage that cannot run says so, and says what would fix it. There are three outcomes, not two. A stage *succeeds*; a stage *refuses*, and the refusal names its cause and its remedy; or a stage is *absent by design*, which is drawn differently again. The webcam has no depth sensor, so its fifteen depth stages are by-design absences and are not faults. Rendering a by-design absence identically to a defect is a failure mode this project has recorded before, and it costs a reader the ability to tell a limitation from a bug.

Nothing in the file can move the arm. The program contains no publisher, no service client, no Kortex session and no trajectory. Its only possible traffic is a subscription. This is a structural property rather than a policy, and it is what allows the pipeline to be exercised on live hardware without a safety case.

The arithmetic is checked against a constructed frame whose answer is known. A self-test runs every geometric stage against a synthetic depth image whose plane, object heights and object widths were chosen rather than measured, so that a stage returning the wrong answer fails before any real camera is involved. This is the standing rule of [Chapter B](#) applied to a new instrument at the point of building it rather than after it produced a surprising number.

8.1.1 The geometry, written out

Depth arrives as a range image and is lifted to a point cloud by the pinhole model, using intrinsics read from the device rather than assumed:

$$\mathbf{P}(u, v) = \left(\frac{u - c_x}{f_x} Z(u, v), \frac{v - c_y}{f_y} Z(u, v), Z(u, v) \right)^\top. \quad (8.1)$$

The support plane is fitted by random sample consensus. Three points are drawn, the candidate plane is $\mathbf{n} = (\mathbf{b} - \mathbf{a}) \times (\mathbf{c} - \mathbf{a})$ normalised, with offset $d = -\mathbf{n} \cdot \mathbf{a}$, and it is scored by the number of points within a tolerance $\tau = 6$ mm:

$$S(\mathbf{n}, d) = |\{ \mathbf{P} : |\mathbf{n} \cdot \mathbf{P} + d| < \tau \}|, \quad \text{best of 400 draws.} \quad (8.2)$$

Height above the fitted surface is then the signed residual, and it is what separates an object from the surface it stands on:

$$h(\mathbf{P}) = \mathbf{n} \cdot \mathbf{P} + d. \quad (8.3)$$

Two quantities are reported for every plane fit and both are needed. The residual root mean square says how flat the fitted set is; the inlier *fraction* says whether the fitted set is the scene. A plane fits three points perfectly, so a small residual on its own establishes nothing, and [Section 8.3](#) is a case where the two diverge sharply.

8.2 The orientation defect

The first thing the instrumentation found was in the instrumentation's own input, which is the outcome the standing rule predicts.

Three separate places in the software decided independently whether the scene depth camera is mounted upside down. Two rotated its frames by 180° and carried the comment that the camera is inverted; the third, the live node that publishes to the rest of the system, rotated nothing. Nobody had measured it, so the recorded figures and the live topics disagreed and neither side could tell.

It was settled by measurement rather than by inspection, using a reference that needs no convention: the HD webcam watches the same scene from the same side of the room and is rotated by nothing. Each frame is reduced to a 64×64 grey signature, standardised so that a brighter camera does not score differently from a darker one, and correlated against the reference in both orientations. Over four independent captures the un-rotated RealSense frame agrees with the webcam at $r = 0.66$ to 0.70 and the rotated one at $r = 0.27$ to 0.34 : the same verdict every time, a factor of 2.4 apart. **The camera is mounted the right way up**, and every scene figure recorded before the correction was inverted.

The correction is not a rotation of the image. A 180° rotation maps pixel (u, v) to $(W-1-u, H-1-v)$, and the principal point is a pixel, so it must move with it:

$$c'_x = W - 1 - c_x, \quad c'_y = H - 1 - c_y. \quad (8.4)$$

Rotating the image and leaving the intrinsics alone places every deprojected ray on the wrong side of the optical axis. On this device the principal point sits 8.85 px from the image centre, so the error is 17.7 px of

apparent principal-point displacement, or 29 mm at 1 m — small, real, and completely invisible in a picture. The orientation is now stored as a measured configuration with the evidence that set it, one function applies it to the image and the intrinsics together so that half a rotation cannot be applied, and all three consumers read the same file. Undoing the rotation restored the intrinsics to the device’s own reported values of $c_x = 318.87$, $c_y = 231.22$ exactly, which is an independent confirmation that Eq. (8.4) was applied in the right direction.

8.2.1 What the defect did and did not corrupt

Because the rotation was applied *consistently* to the image and the intrinsics, substituting Eq. (8.4) into Eq. (8.1) negates the X and Y components and leaves Z untouched. The recovered point cloud was therefore an exact reflection of the true one through the optical axis, and every distance, extent, angle and residual is preserved under a reflection. No metric quantity should have changed at all, and this was checked rather than assumed: the deprojected extents are exactly negated ($X \in [-3.477, 1.141]$ became $X \in [-1.141, 3.477]$) and the centroid components change sign and nothing else.

Two classes of stage changed anyway, and the reason each changed is worth recording.

The learned segmenter is not rotation-invariant. Given the same scene the right way up it returned 96 regions rather than 92, with a different largest region and a different median area. A convolutional segmenter has no reason to be invariant to a 180° rotation and was not trained to be, so this is expected behaviour and not a defect — but it means that a geometric pipeline can be immune to an orientation error while the learned component sitting beside it is not, and only the learned component will show it.

The plane fit changed because its support is weak, not because the geometry changed. Equation (8.2) draws candidate triples in raster order, and rotating the image reverses that order, so a different sequence of candidates is examined. On the wrist cameras, where the best plane holds 74.0 % of the points, this makes no difference. On the scene camera, where the best plane holds 3.8 %, it selects a different plane entirely: 2.68 m from the camera before the correction and 4.35 m after. That is not a consequence of the orientation bug. It is a consequence of pointing a plane-finder at a room, and the orientation change merely made it visible.

8.3 What the cameras measured

Table 8.1: Outcomes of 33 stages on each of four cameras. *By design* is the webcam’s depth stages, which are absent because the sensor is absent; they are counted separately from refusals so that a limitation is never reported as a fault. **No stage crashed on any camera.**

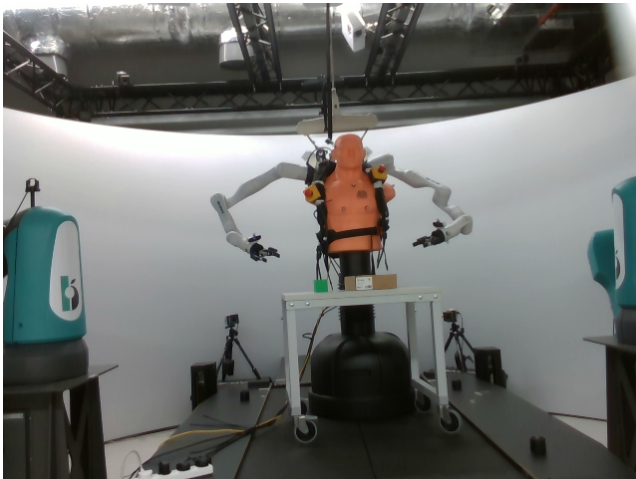
Camera	Result	Refused	By design	What it is
left wrist	28	5	0	Kinova vision module, RGB-D, on the arm
right wrist	29	4	0	Kinova vision module, RGB-D, on the arm
scene depth	27	6	0	RealSense D435i across the room
scene colour	12	8	13	HD webcam, colour only, never calibrated
total	96	23	13	of 132

Table 8.2 contains one result and one non-result, and it is worth being explicit about which is which.

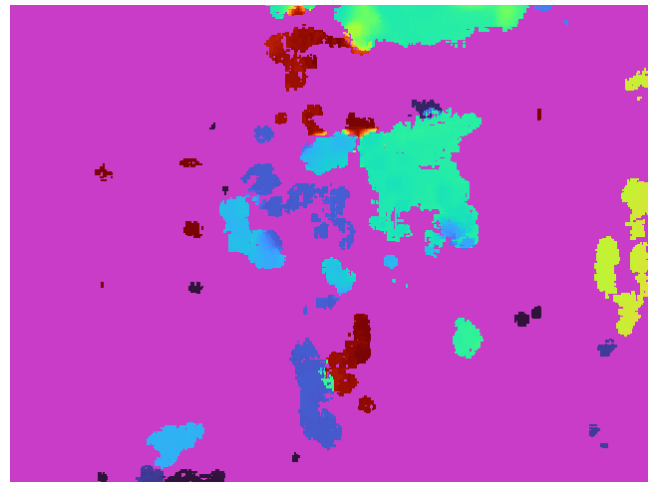
The wrist cameras measure a work surface to about 1.5 mm. A plane holding 74.0 % of the returned points at 1.43 mm root mean square, at a working distance of 0.278 m, is the quantity the autonomous pick path in Chapter 5 depends on, and it is close to the 0.54 mm this project recorded for the same sensor class against a

Table 8.2: Geometry recovered from each depth camera in one frame. The inlier fraction is the column that decides whether the row below it means anything: the wrist cameras identify a work surface and the scene camera does not, which is the honest consequence of pointing a plane-finder at a room rather than at a table.

	left wrist	right wrist	scene depth
pixels returning a depth	89.4 %	80.5 %	14.5 %
median range	2.222 m	1.838 m	4.706 m
points inside the working band	54 789	42 779	38 526
colour pixels given a depth	64.4 %	58.8 %	aligned in hardware
plane inliers	74.0 %	52.8 %	3.8 %
plane residual, RMS	1.43 mm	1.59 mm	3.41 mm
surface extent	0.579×0.423	0.428×0.467	1.012×1.762
surface distance	0.278 m	0.290 m	4.352 m
objects posed in three dimensions	5	12	12
nearest object	0.539 m	0.387 m	2.609 m
graspable within the jaw	0	0	0



(a) Colour, the right way up.



(b) The aligned depth of the same frame. Magenta is no return.

Figure 8.1: The scene camera after the correction of [Section 8.2](#). The depth panel is the reason the row for this camera in [Table 8.2](#) reads as it does: at a median range of 4.706 m only 14.5 % of pixels return anything, and what does return is the far wall and the floor rather than a work surface. A stereo depth camera across a room is not a small version of a wrist camera.

rendered scene. This is the first evidence in the project that the surface-fitting stage behaves on a real table as it does on a constructed one.

The scene camera does not identify a support plane, and the figure says so rather than reporting the number quietly. Its best plane holds 3.8 % of the points. Every quantity that depends on it — height above the surface, object heights, three-dimensional boxes — therefore rests on a plane that one point in twenty-six agrees with, and none of those numbers should be quoted. The instrument reports them because suppressing them would hide the inlier fraction that condemns them; the interpretation is stated here instead.

8.3.1 The two defects the first runs exposed

The first attempts at this session produced results that were wrong in ways that only a stage-by-stage instrument would have separated from a broken camera, and both were in the software.

A gate belongs to the thing being measured. The depth working band 0.08 to 1.5 m is the pick path's own, and is correct for a camera on the hand. Applied unchanged to a camera across the room, whose nearest return is 0.741 m and whose median is 3.84 m, it retained **0 of 85 061 returns**, and sixteen stages then refused for want of depth that the camera had in abundance. The band is now declared per source and every figure states the band it used; the same camera now yields 38 526 points.

Two stages disagreed about the same frame, and the disagreement was the evidence. The stage that asks what is standing on the surface reported nothing, while the stage that poses objects in three dimensions — on the same frame — reported six objects, the nearest at 0.536 m. Both cannot be true. The cause was that depth at 480×270 forward-projected into colour at 1280×720 lands one sample in roughly every seven colour pixels, so only 7.22 % of colour pixels carried a depth. The above-plane mask was a dot pattern: every connected component fell below the area floor, so the stage that needs connectivity saw nothing while the stage that works per mask was unaffected. Splatting each projected sample over the gap it spans, with the radius derived from the resolution ratio rather than chosen, raised coverage to 64.4 % and 58.8 % (Table 8.2). A contradiction between two stages was worth more than either stage's output.

8.3.2 What did not work, and why that is an answer

No object in any frame could be grasped. The parallel-jaw planner refused on every camera, and the refusal carries its arithmetic: the narrowest axis of the nearest object measures 144 mm against a usable jaw opening of 75 mm. That is a statement about the scene, not a failure of the planner, and it is the same geometric constraint that Section C.3.1 imposes on the study's carry tasks.

The open-vocabulary detector returned nothing on any camera. It was given the single class *green cube* and found no instance of it at a confidence floor of 0.05. It was not asked for people or furniture, so this is the measured behaviour of that model on this rig with that prompt rather than a misconfiguration; the classical colour and geometry stages found the same objects the model did not.

No fiducial marker decoded in any frame, because none was in the scene. A marker either decodes or it does not, which is the property that makes it a poor thing to depend on and a good thing to check.

The colour scene camera has never been calibrated, so eight of its stages refuse rather than assume that the principal point is the image centre. On the depth camera the principal point is 8.85 px from centre, which would be 14.7 mm of error at 1 m if assumed away. The refusal is the correct behaviour and the calibration is a half-hour task that has not been done.

The robot is not detected in any image, and no stage claims that it is. Nothing in this system finds an arm

in a picture. The arm's pose comes from its encoders through the kinematic description, and drawing it into a camera image requires the hand-eye transform that [Chapter 6](#) identifies as the largest unmeasured term in the positioning budget.

8.3.3 A caption that could not be written

The arms were to be parked at the pick pose for this session so that the figures could be captioned as such. The instrument measures the pose rather than assuming it, and found the left arm 10.67° and the right 13.53° from the reference configuration, against a tolerance of 2° . Earlier runs in the same session were 66 to 82° out.

None of these figures may therefore be captioned “at the pick pose”, and none is. This is the check working rather than failing: a figure captioned with a pose nobody measured is worth nothing, and nothing else in the session would have noticed. It is the same failure mode as [Section 8.2](#) in a different variable — a property everybody believed, that nobody had put a number on.

8.4 What this chapter changes

Three things are true after this session that were not true before it.

The perception geometry has been exercised on real sensors and the surface fit holds to 1.43 mm at working distance, which is the term the pick path depends on and which had previously only been characterised against a renderer.

The scene camera is a different instrument from the wrist cameras, not a distant one, and the numbers say by how much: an eighth of the depth returns, a twentieth of the plane support, and a scene with no dominant plane in it. Any autonomous behaviour that expects a support surface must take it from a wrist camera.

And a constant that three separate files had asserted and none had measured was wrong. It was found in the first hour of pointing an instrumented pipeline at the hardware, it was invisible to every metric stage by construction, and the only thing that would ever have shown it is a second camera looking at the same room. It is the twenty-sixth entry in [Chapter B](#), and the first whose mechanism is a constant asserted in three places and measured in none.

Chapter 9

Workspace characterisation and its consequences

This chapter is separated from [Chapter 7](#) because it is not a single measurement but a chain of them, and because its consequences determined the shape of the task set. It reports a negative finding together with a quantified and buildable mechanical fix; the finding alone would be a complaint rather than a contribution.

9.1 Continuous reachability from the home pose

Reachability was measured as a swept extent rather than as pointwise feasibility, because teleoperation requires the arm to travel to a pose from where it already is, without a branch change. Twenty-six directions were swept in 1 cm steps from the home configuration, each step seeded from the previous solution exactly as the follower does.

Repeating the sweep ten times gave identical quartiles: the inter-quartile range was 0.000 m in every direction for both arms. The measurement repeats. What did *not* repeat in earlier runs was the summary statistic: isotropy taken as the minimum over directions of a single sweep is dominated by rare early terminations and varied by a factor of two between runs. On medians it is stable, at 0.16 for the left arm and 0.10 for the right ([Fig. 9.1](#)).

The early terminations were traced to the measuring tool rather than to the robot. A sweep is a chain of up to ninety solver calls and any single failure ends that direction; the follower does not suffer this because it re-seeds the redundant joint and retries. Giving the sweep the same retry reduced early terminations from nine to zero on the left arm and three to zero on the right, while the medians barely moved. That the medians were already right is what establishes the outliers as chain compounding rather than geometry.

9.2 The two arms share no reachable workspace

Over a grid of sixty-three cells spanning the frontal working volume, with the arms verified at the home pose, **no cell was reachable by both arms**. The result survives the two obvious objections, as [Table 9.1](#) shows: freeing the wrist yaw raises each arm's individual reach from sixteen and eighteen cells to twenty-one and still yields no shared cell, and repeating the test with more redundancy retries changes nothing.

A fine sweep in 0.05 m steps across the plane on which the arms would have to meet locates the boundary precisely: the right arm reaches $x \leq -0.10$ m, the left reaches $x \geq 0.15$ m, and there is a 0.25 m dead band between them in which neither arm can place its end effector ([Fig. 9.3](#)).

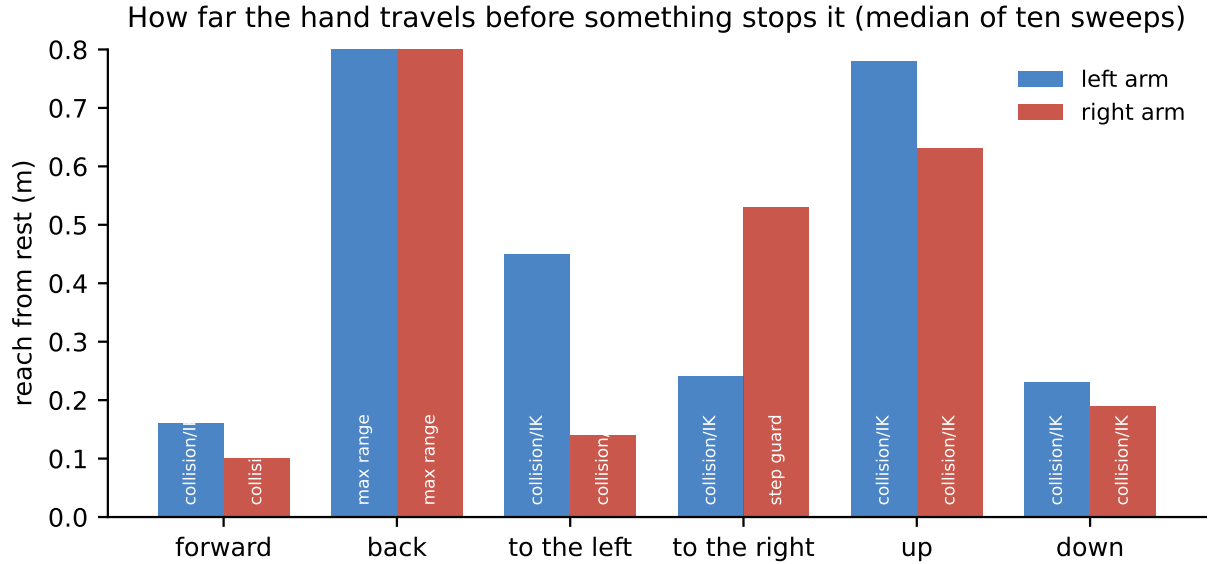


Figure 9.1: Continuous reachability from the home pose in the six principal directions, median of ten sweeps, with the binding constraint printed inside each bar. The envelope is markedly anisotropic: the median reach over all twenty-six directions is 0.34 to 0.40 m while the worst direction is 0.14 m on the left arm and 0.09 m on the right. *Max range* marks a direction in which the sweep exhausted its own travel before the arm did, so those two figures are lower bounds. Regenerated from `recordings/baselines/workspace_n10_20260806.json`.

Table 9.1: Both-arm overlap over the 63-cell frontal grid, arms verified at home. The null result survives both wrist freedom and additional inverse-kinematics retries.

Test condition	Both	Left	Right
Fixed orientation, 3 retries	0/63	16	18
Fixed orientation, 6 retries, 3 repeats	0/63	17	18
Yaw free, 6 retries	0/63	21	21

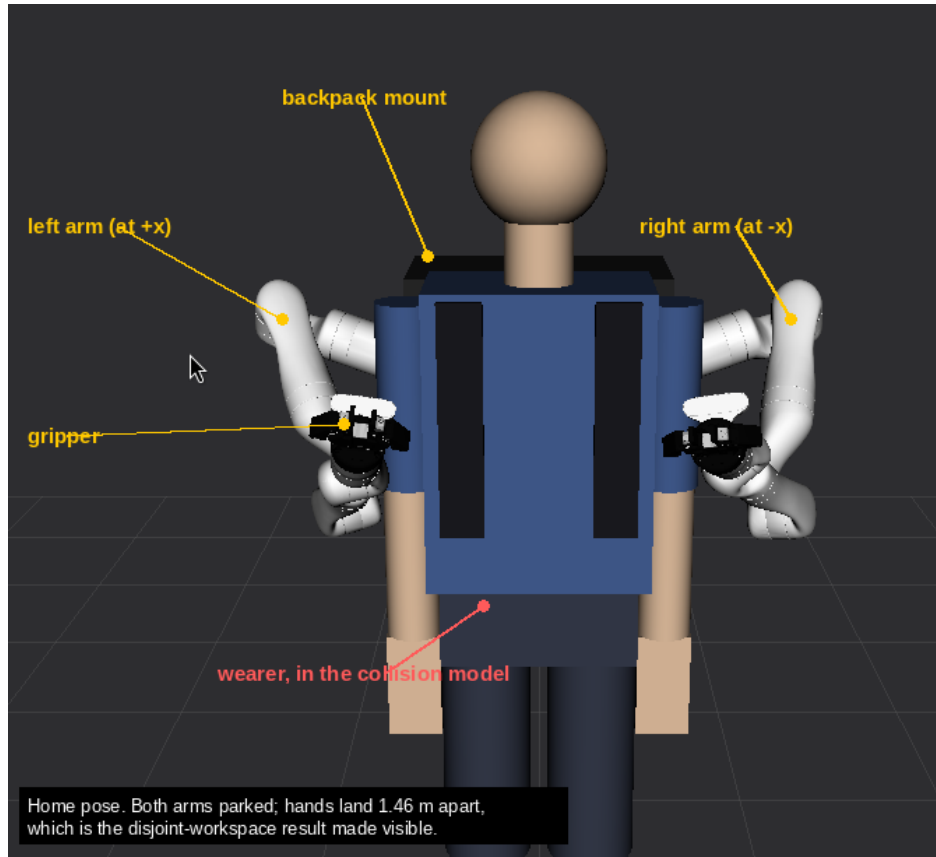


Figure 9.2: The simulated platform at the home joint angles, captured from the running system rather than drawn. Both arms are parked over the shoulders and both hands finish on the same side of the body, 1.46 m apart. That asymmetry is not a rendering artefact and not a mount error: it follows from the home joint angles, which are ground truth because the simulation-to-hardware bridge replays them onto the real arm. It is the disjoint-workspace result of this chapter made visible.

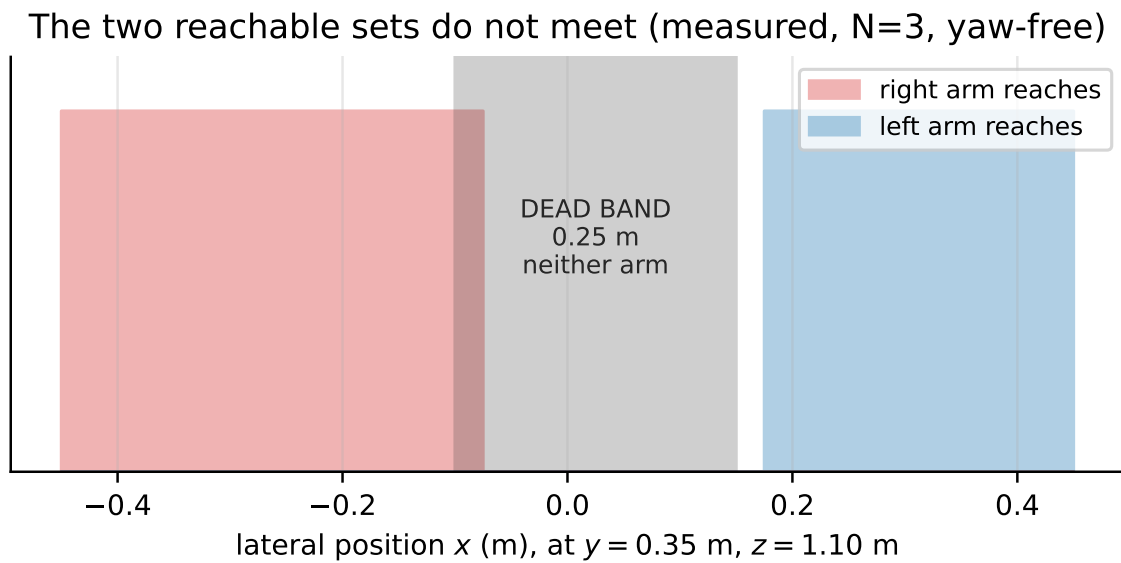


Figure 9.3: The two reachable sets do not meet. Measured at $y = 0.35$ m, $z = 1.10$ m with yaw free and three repeats per point.

9.2.1 What this does and does not establish

It establishes that, within $x \in [-0.6, 0.6]$, $y \in [0, 0.4]$, $z \in [0.90, 1.30]$ m, with the as-built mount and the legacy home joint angles, the two reachable sets do not intersect.

It does *not* establish behaviour outside that volume, nor does it generalise to other mounts. The correct claim is that the arms share no workspace *as mounted and as parked*, and the remedy is mechanical.

9.3 The fix is buildable

A parametric sweep over candidate mount placements shows the null result is not intrinsic. Moving the mounts inboard, forward and down, with additional forward tilt, raises the number of shared cells from three to **twenty-two** on the purely kinematic measure, within a buildable 0.20 m mount separation (Fig. 9.4).

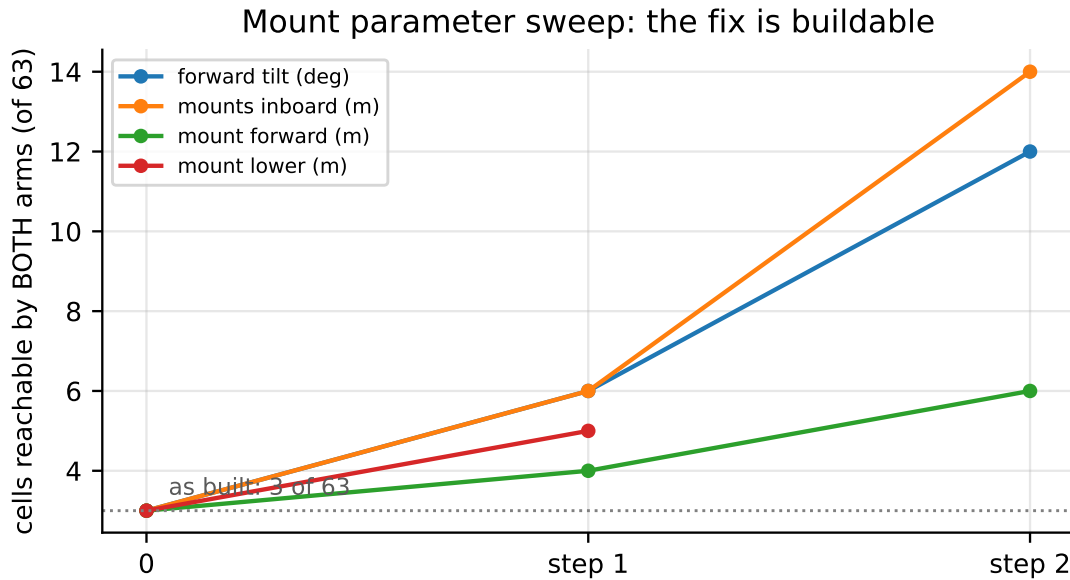


Figure 9.4: Single-parameter sweeps from the as-built mount. Forward tilt and inboard translation both help substantially; splaying the mounts outward *hurts*. The sweep is purely kinematic, moving the arms’ bases without moving the wearer, so these are upper bounds and any candidate requires a clearance check before adoption.

The sweep was recorded and deliberately not applied at the time, because changing the mount invalidates the home pose, the workspace anchor and every clearance figure reported in Chapter 7. It was applied later, as one deliberate pass with the whole task set re-measured afterwards, and Section 9.6 reports what it bought.

9.4 One boundary, two independent constraints

Two further measurements, taken for different reasons, land on the same lateral boundary and together make the case that the inboard band is unusable.

The first is wearer clearance. Every scenario in the chest-height band passes inverse kinematics with collision checking enabled, and the planner reports the poses valid, yet the arms pass within 48 to 67 mm of the wearer’s torso, inside the margin the real-robot configuration enforces, which was 120 mm at the time of that measurement and is 150 mm now. The reconciliation is that the two tests ask different questions: the planner’s check is binary

contact, whereas the floor is a margin, and a pose can satisfy one while being refused by the other. Sweeping the lateral half-span shows clearance rising monotonically and crossing the floor at 0.25 m.

The second is grasp feasibility. A top-down grasp requires the wrist to adopt an orientation with its approach axis vertical. Sweeping the same lateral coordinate gives a sharp cliff: zero of nine grasp candidates are solvable at $|x| = 0.25$ m and nine of nine at $|x| = 0.30$ m. The failure persists with collision checking disabled, so it is kinematic infeasibility of the wrist orientation rather than obstruction.

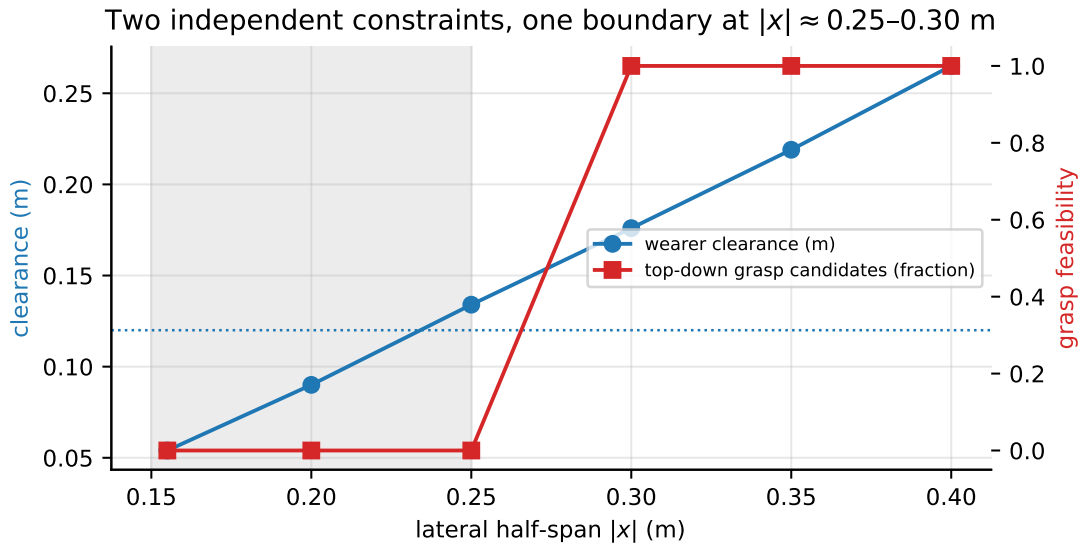


Figure 9.5: Wearer clearance and top-down grasp feasibility against lateral half-span. The two constraints were measured independently and for different purposes, and both place the usable boundary at $|x| \approx 0.25$ to 0.30 m.

9.5 The centre of the working space, asked four ways

The question the platform is most often asked is whether the arms can work in front of the wearer’s own chest, where the wearer’s own hands would be. It was asked four times, by four instruments built for different purposes, because the first answer was unwelcome enough to distrust.

Table 9.2: Four independent attempts to place usable work within 100 mm of the body centre line. All are at ten repeats over the full densified path with the wearer measured geometrically against the 150 mm floor.

Question	Answer
Innermost workable column at the working height	$ x = 0.325$ m on the left arm and 0.450 m on the right
The same, with the wearer’s arms deleted from the model entirely	0.300 m and 0.375 m. Every column from 0.125 m to 0.300 m then binds on the torso , and at $ x = 0.10$ m the clearance is -0.007 m and -0.013 m: the arm is inside the chest
Objects <i>resting</i> on a surface, sweeping surface height and distance together, 3360 configurations	no configuration places a graspable object within 100 mm of the centre line
The same, asked for a top-down approach instead of the side approach	0 of 840 cells, at any surface height from 0.70 m to 1.10 m, on either arm

Two conclusions follow, and the second is the operationally useful one.

The constraint is the **torso**, not the wearer's limbs. Making the wearer's arm posture a variable and re-running the sweep across five postures moves the boundary by at most 75 mm, and deleting the arms altogether leaves the centre shut. An instruction to the wearer is therefore worth something and is not a route to the centre. Two of the five postures are actively worse than the default: with a back-mounted rig, arms clasped behind the back and arms held out to the sides both place the wearer's own limbs inside the clearance floor at the home pose, before the robot has been asked to do anything.

Neither arm crosses the centre line at all. Measured at ten repeats with the wearer and the working surface present, every cross-side placement slot and every cross-side object returns zero solutions of ten, on both arms. Not marginal: nothing solves. This is what forces the randomised stage of the pick-and-place task to draw the *side* and let the colour follow, and the task builder refuses an object whose declared colour names the other side's target rather than emitting a path whose every waypoint fails for a reason nobody would connect to colour.

9.6 What the applied mount change bought

The mount modification was subsequently applied, as one pass with the task set re-measured afterwards: the mounts moved 150 mm outboard with 15° of outboard yaw, the working surface was raised, and its near edge was brought forward.

Table 9.3: What the applied mount change altered. Home clearance improves because the mount is no longer the nearest part of the robot to the wearer, which also raises the ceiling on every clearance figure the platform can report.

Quantity	Before	After
Clearance at the home pose	0.1610 m	0.2202 m
Nearest robot part to the wearer at home	the mount, which no joint moves	the moving chain
Task objects resting on the surface	not achievable	achieved
Pick-and-place placement targets	0.530 m and 0.350 m off centre	0.290 m either side

The cost is that every quantity derived from the previous mount had to be re-derived, which is the reason the change was deferred rather than taken early: the approach anchor, the pad offset, the home end-effector position and every task coordinate computed through them. That is a session's work and it is the correct way to spend it, but it cannot be done twice.

9.7 Clearance along the task paths

Reachability is not the binding property for a machine mounted on a person. [Figure 9.6](#) gives the closest approach to the wearer over the whole of each task's path, which is the quantity the safety case turns on.

The breaches are reported rather than corrected. Each is a layout change with its own re-verification, and moving a task's coordinates to satisfy the floor without walking the whole path again would replace a known breach with an unknown one.

9.8 Consequences for the task set

Three consequences follow directly, and each is visible in the task definitions of [Chapter 7](#).

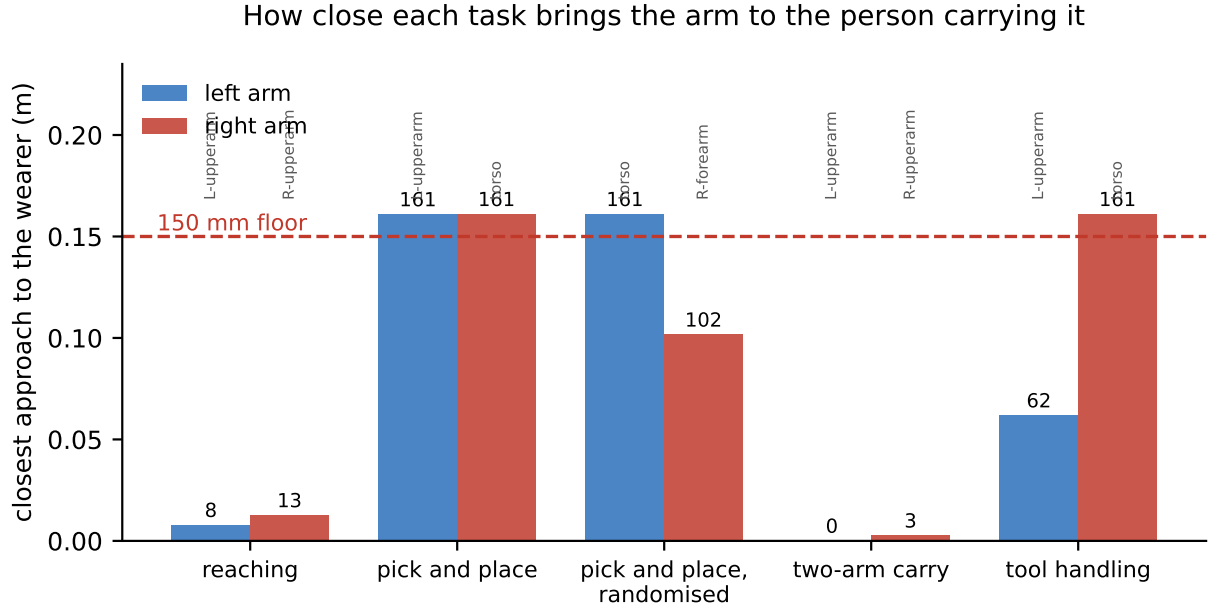


Figure 9.6: Closest approach to the wearer over each task’s full commanded path, ten repeats, with the body measured geometrically and the nearest body part printed above each bar. Only the pick-and-place task clears the floor on both arms. The three bars at 161 mm are the mount before the change of [Section 9.6](#), which no joint moves and which therefore sets the ceiling. Regenerated from `recordings/baselines/home_change_applied.json`.

Inter-arm handover is impossible. It requires both arms at one point, and no such point exists. This removes an entire task family, and no control, planning or autonomy change can restore it.

A bimanual task must either span the gap or exploit simultaneity. A rigid object held at two points can bridge the dead band; two targets in the two disjoint sets can require simultaneous satisfaction. Both routes are used.

The coupled objects had to be re-specified. The original 310 mm tray placed each grip at $|x| = 0.155$ m, inside the dead band, and ten of twenty-five waypoints failed once the arms were verified at home. Widening to 500 mm places each grip at $|x| = 0.25$ m, which satisfies the clearance boundary and the reachability boundary simultaneously. The compliant carrier had to lengthen with it: retention of a ball of radius r requires sag of at least $2r$, so

$$\text{sag}(s) = \sqrt{\left(\frac{L}{2}\right)^2 - \left(\frac{s}{2}\right)^2}, \quad s_{\max} = 2\sqrt{\left(\frac{L}{2}\right)^2 - (2r)^2}, \quad (9.1)$$

where s is the gripper separation, L the carrier length and r the ball radius. For $L = 540$ mm and $r = 20$ mm this gives $s_{\max} = 534$ mm, leaving a 34 mm margin at the nominal 500 mm separation ([Fig. 9.7](#)). The length is chosen so that the failure threshold sits *inside* the reachable band: a longer carrier could never fail and would measure nothing.

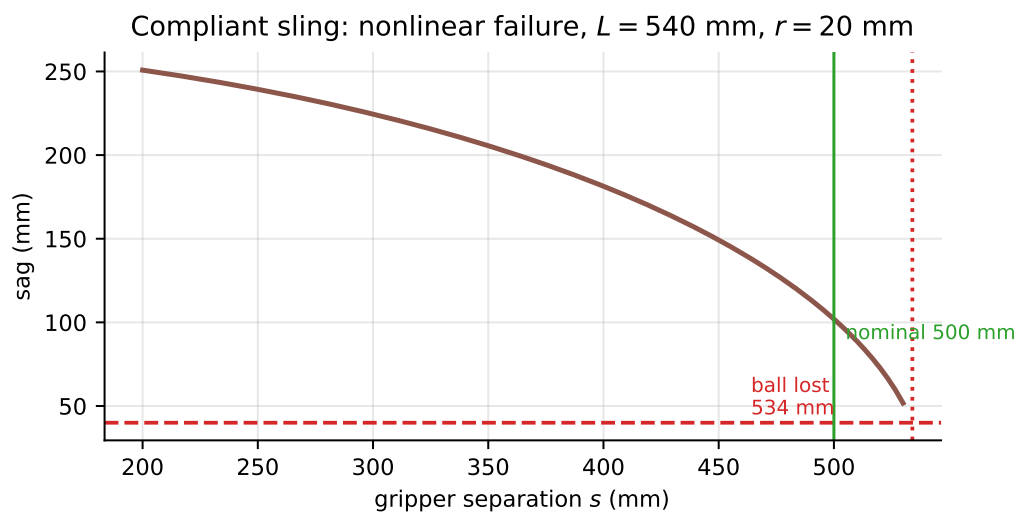


Figure 9.7: Sag against gripper separation for the re-specified compliant carrier. The nonlinearity is the point: sag falls slowly, then the ball is lost abruptly at 534 mm.

Chapter 10

Discussion

10.1 What the degradation architecture bought, and what it did not

The premise in [Chapter 1](#) was that a body-worn master will lose channels, so losing them must be survivable. The two dated captures support the premise empirically: the failed population grew between them ([Section 7.2](#)). The architecture's value can be stated precisely from the measurements.

It kept the system running. With eight of fourteen channels failing the health criterion, the right arm retains full position sensing at $R^2 = 0.986$ and the left arm retains direction. A joint-space mapping would have been inoperable on either arm from the first failed channel, because it has no reconstruction path.

It kept the failures visible. The incoherent verdict exists because a failing potentiometer does not go quiet, and range alone calls a wide-spanning nonsense signal healthy. Without that verdict the system would have kept following 54.5 mm of commanded motion the operator never asked for, on a master whose whole reach is 272 mm.

What it did not buy is capability. Freezing the left arm's bend joints removes its radial dimension entirely, collapsing its commanded set from a shell to a surface, and the wrist-roll fallback restores an axis rather than restoring the measurement. The architecture converts a fault into a known, bounded, announced reduction. It does not convert it into working hardware, and [Section 7.2](#) names the two channels whose repair would.

10.2 The reduced-degree-of-freedom premise, assessed

The second premise was that commanding fourteen joints through a body-worn master is a usability problem, and that the answer is to take each component from the sensor that observes it best. The evidence is mixed, and it divides cleanly by component.

Elevation from gravity works. It is mount-independent, drift-free, integrates nothing, and survived every reformulation. Reach from the forward-kinematic magnitude works, because a magnitude depends on the bend joints and tolerates a dead roll passed through as zero. Both vertical and fore-and-aft motion track correctly on both arms.

Azimuth from a single joint does not work, and the reason is instructive rather than incidental. The spherical model assumes azimuth and elevation are independent. They are not, because joint one is a roll about the arm's own axis, so its effect on the tip scales with how bent the arm is. The 137.7° measured between two directions intended to be 90° apart is not a calibration error that a better axis map or a better zero could remove; it is the model being wrong about the geometry.

The honest reading is that the decomposition succeeded for the two components where a single sensor observes

the quantity directly, and failed for the one where it does not. That is a usable design rule for a successor: decompose along the axes your sensors actually observe, not along the axes that make the mathematics tidy.

10.3 The workspace result and what it costs the study

Chapter 9 reports that the two arms share no reachable workspace under the as-built mount. The consequence for the experimental programme is severe and worth stating plainly rather than absorbing into a limitations list.

Inter-arm handover is impossible. Not difficult, not unreliable: there is no point in space both arms can reach, so no control strategy, no planner and no orientation mode can produce one. The task that would have demonstrated autonomy achieving something teleoperation cannot was blocked by geometry before it was blocked by anything else, and the finding is the reason it is not in the task set.

The cause is not the mount rotation, which was proved to leave the mirror residual invariant. It is that the two arms are parked asymmetrically at their home joint angles, and those angles are ground truth because the simulation to hardware bridge replays them onto the real arm. Changing them in software would command the difference as a jump. Fixing this requires re-parking the right arm in hardware.

Two of the five delivered tasks are the same bimanual mechanism with different objects, and only one task uses the second mechanism. There are two genuinely bimanual mechanisms available on this platform, not three. That is a real narrowing of what the platform can be used to ask.

10.4 Comparison against the planning report's targets

The delivered system diverges substantially from the plan, and the divergences fall into three groups.

Superseded by measurement. The planning report specified physical springs at the master for force feedback and virtual spring impedance at the slave. The impedance target is unreachable on this platform for a reason discovered during the work: the arms' cyclic control interface is unusable over the available network path, so the system commands through a high-level velocity interface at 18.4 to 18.7 Hz. An impedance law at 18 Hz is not an impedance law. The compliance the plan sought would have to come from the mechanism instead, through series elastic actuation at the joint [34, 19], or from a current-based formulation that does not require a high-rate torque loop [6]. Both are hardware changes.

Deferred and still open. Coordinated pick-and-place on the real platform was not demonstrated. The task set is specified and verified geometrically, and the runner exists, but the arms were not available for the period in which it would have been run.

Added. The safety architecture, the degradation architecture and the workspace characterisation were not in the plan. The first two are the direct consequence of discovering that sensor faults dominate; the third is the consequence of asking whether the task set was reachable before running it.

10.5 The safety layer is thorough about distance and silent about time

The safety architecture is careful about the quantity it was designed around. The margin is measured geometrically against the parts of the arm that actually pass the wearer rather than through the semantic collision description; fourteen injected faults are caught; every mechanism that can stop the arm announces itself by name; and a blocker that cannot state how it clears is refused at registration. Most of that was built after a specific failure demonstrated

the need for it.

None of it addresses the fact that the margin is a *distance* and the wearer can move. [Table 6.3](#) gives the arithmetic: in five of six modelled limb motions the limb travels further than the entire 150 mm margin before the guard is aware anything has changed. The system protects a stationary person from a moving robot, and it is deployed on an arrangement that also contains a moving person.

Three responses are available and none is implemented. Inflating the floor by the perception latency multiplied by an assumed limb speed would at least make the margin mean what it claims, and it is a one-parameter change. Cutting the latency is largely available, since most of the 562 ms spread is a configured publication rate limit rather than the estimator, which runs at 28 to 34 ms per frame. And scaling commanded speed with measured clearance would couple two quantities the system already computes and currently keeps apart. The first is the one to do first, because it changes what the margin asserts rather than merely improving it.

10.5.1 The fallback restores an assumption that is optimistic by 224 mm

Admitting a camera into the clearance check is governed by the monotonicity rule of [Section 5.6](#), and that rule is correct about its own scope: a tracking result that would place the wearer further away changes nothing. The gap is on the other side. When the estimate is stale, silent or gated out, the guard falls back to the mannequin, whose arms are down.

Measured through the guard's own geometry, a wearer holding their arms out to the sides has 0.0933 m of clearance at the home pose, where the fallback believes there is 0.3170 m. The system's present answer to this is an instruction to the wearer, recorded as a standing operating rule. That is a genuine mitigation and it is not a control system, and it should be read alongside the reaction budget above rather than separately: markerless tracking on this platform is load bearing, and its failure mode is to restore an assumption quietly.

10.5.2 And the clearance model itself is not yet trustworthy

The most uncomfortable single finding of the hardware session belongs here rather than in the results. Asked about a pose in which the arm was *physically against the mannequin*, the clearance model returned 367.0 mm, which is the same value it returns for a visibly clear pose. It samples link *origins* rather than link geometry, so it cannot see the arm between joints.

Every clearance figure in this thesis that was computed through the guard's capsule model rather than through that link-origin model stands, and they are the majority. But the check that runs on the physical arm during homing was the link-origin one, which means the standing constraint that the wearer's margin is enforced at all times is, at the time of writing, **not enforced in code on hardware**. [Chapter 11](#) places its repair third in the order of work, above the mount change and above the study.

10.6 What the perception stage did and did not change

Replacing declared coordinates with measured ones removes a specific and serious failure mode: an arm acting with complete confidence on a number in a file that no longer describes the room. The map is accurate enough for the task, at every object centre within 1 mm, and it is the first structure in this project a planner has consulted, which closes the gap in which a path could route correctly around the wearer and sweep through the bench.

It does not make the platform able to pick things up. The hardware session in [Section 7.14](#) found that the transform from the camera frame to the arm is wrong by roughly 8.5° and rotates with the wrist, which is larger

than the entire capture gate at working distance. A perfect map expressed through a wrong extrinsic yields a wrong grasp. The correct reading is that the perception work moved the bottleneck from *not knowing where things are* to *not knowing where the camera is*, and the second is a calibration with a known method, which is a better position to be in.

10.6.1 What the real cameras changed about that assessment

[Chapter 8](#) is the first evidence that the geometry behaves on real sensors, and it changes the assessment in three specific ways.

The surface-fitting stage transfers. A support plane at 1.43 mm root mean square with 74 % inlier support at a working distance of 0.278 m, on a real table under laboratory lighting, is within a factor of three of what the same code achieves against a renderer. That is the one component of the perception chain that can now be said to work outside simulation, and it is the component the pick path leans on hardest.

The scene camera is a different instrument and should not be used as a work-surface sensor. At a median range of 4.7 m it returns depth on 14.5 % of its pixels and its best-supported plane holds 3.8 % of the points it does return. The design documents treated it as a wide-angle equivalent of a wrist camera; it is not, and every quantity that depends on a support plane must come from a wrist camera. This is a limitation of the arrangement rather than of the device: there is no dominant plane in a room.

The learned components contributed nothing that the classical ones did not. The open-vocabulary detector returned no instance of the prompt it was given on any of the four cameras, while the colour-and-geometry path found the objects and measured them. This is a single scene and a single prompt, so it is not a general claim about the models; but it does mean that no result in this thesis rests on a learned detector, which is worth knowing given that [Chapter 4](#) argues the models were adopted for open vocabulary rather than for accuracy.

The uncomfortable part of [Chapter 8](#) is not any of those. It is that pointing an instrumented pipeline at the hardware for the first time immediately found a constant that three files asserted, none measured, and which had silently inverted every scene figure the project had recorded. The defect was undetectable by any metric the pipeline computed, because a consistently applied rotation preserves every distance. It was detectable only by comparing one camera's picture against another's. The general lesson is the one [Chapter B](#) has been accumulating: the quantities most likely to be wrong are the ones nobody thought were quantities.

10.7 Validity of the measurements

10.7.1 The base rate of instrument error, and what caught it

Twenty-six times during this project a measurement was wrong and the system was not. That is a high enough base rate to change how the rest of this chapter should be read, so it is stated as a result rather than left in a methods note.

The pattern across the twenty-six is more useful than any individual case. Sorted by mechanism they fall into five groups: an absence read as a value; a test harness that constructs the condition it is testing; state left over from a previous run; the order of operations inside the measurement itself; and — added by this session — a constant asserted in more than one place and measured in none. None is specific to robotics.

Two numbers carry the argument. **Fifteen of the twenty-six were caught by an independently known quantity contradicting the reading**; five by inspecting the code, and six by looking at the raw artefact, usually an

image. Care was not the defence that worked. Redundancy was. And **fourteen of the twenty-six were already answerable from something in hand** — a published stroke, an arithmetic prediction, a quantity that is exact by construction — which is the recommendation this project’s experience supports: build the cross-check in rather than trying harder.

The consequence for this thesis is specific and is applied throughout. Every reachability figure is reported with its repeat count because one solver call is one coin flip. Every stage of the simulation campaign of [Section 7.1](#) carries controls that must *fail*, and [Section 7.1.2](#) reports those alongside the results they license. Where a quantity has never been measured, the text says so rather than offering an estimate — the camera mounting transform in [Fig. 6.1](#) is drawn with no bar for exactly that reason. And any unreproduced single measurement in this document should be read as provisional.

[Chapter B](#) is the full record: all twenty-six cases, what each actually was, and what revealed it.

10.7.2 Simulation is not the arm

Every quantitative result in this thesis except the channel measurements comes from simulation with mock hardware. The mock echoes commanded positions with no dynamics, which has one consequence that must not be glossed: every force reading is identically zero, so any quantity derived from a force is structurally undefined rather than approximated. The measurement that payload configuration made exactly 0 m of difference to tracking is a statement about the mock.

10.7.3 Latency figures are lower bounds

Every latency reported is control-path latency, measured between software stages. It excludes the operator’s reaction, the display, and the physical arm’s response. Any end-to-end figure will be larger, and the reported values should be read as a floor.

10.7.4 The one-stack constraint

Two instances of the master node split the serial stream between them and invalidated a full day of measurements before the cause was found. This is now prevented at the file-descriptor level, but it means that any measurement taken before that prevention was added carries a risk that cannot be retrospectively excluded.

10.8 The study the platform is not ready for

The two-person study this architecture was built to enable was designed in full and was not run. Both halves of that sentence are results, and they are reported here rather than as a chapter of their own, because a protocol that has not been executed is not a finding about supernumerary limbs.

What was produced is a complete protocol ([Chapter C](#)) and a measured property of the analysis that would evaluate it ([Chapter D](#)). The second is worth stating in the body. Applied to 20 000 constructed studies at the planned sixteen dyads, the pre-registered procedure recovers the effect the design was powered for on 97.3 % of them family-wise, but on only 63.8 % for any *named* task once the correction is applied, while rejecting a true null at exactly the nominal rate. The design’s own power calculation reports neither figure. So the study as designed is well powered for the question it asks — whether shared autonomy changes performance on this platform —

and under-powered for a per-task claim, and a per-task null at $n = 16$ would have to be reported as an interval rather than as an absence.

It was not run for reasons that are specific rather than administrative.

1. **Two master channels.** Left j2 and left j4. With both frozen the left arm has no radial dimension, so the direct teleoperation condition, which is the baseline the whole comparison rests on, is not representative of a working master.
2. **The right arm's home pose.** It has never been read from hardware, and the workspace geometry depends on it.
3. **Hardware validation of the safety layer.** Nothing in the safety architecture has run against a real arm. The Kortex session recovery path is the riskiest part, because the arm permits exactly one session and a leaked one refuses the next connection.
4. **Ethics approval for a two-person protocol**, which differs from a single-participant protocol in that one participant bears a physical risk arising from another participant's actions.
5. **A detection rate measured on real images.** The synthetic figure characterises the renderer.
6. **A hand-eye calibration for the wrist camera.** The transform is wrong by roughly 8.5° and is the largest term in the positioning budget. It needs the hardware and a calibration target, and until it exists no open-loop grasp will land inside the capture gate.
7. **A clearance check that measures link geometry rather than link origins**, before anybody wears the rig. This is the one blocker on the list that is a safety defect rather than an incompleteness.

None of these is a research question. All are execution, and the first is two soldered joints.

Chapter 11

Conclusion

This project built a teleoperation system for two seven-degree-of-freedom arms mounted on a worn backpack frame, commanded from an instrumented mannequin master, with a safety architecture appropriate to a manipulator standing beside a person and a shared-autonomy layer above it.

11.1 Against the objectives

A master, built and characterised. Fourteen potentiometer channels, two inertial sensors, two force sensors and two buttons on a single microcontroller, emitting a text frame at 100 Hz. Characterisation found six of fourteen channels usable and reversed two long-standing verdicts, both of which had come from computing statistics across transmitted rows rather than distinct sensor updates.

A mapping that degrades predictably. Position in spherical form with each component taken from the sensor that observes it best. Vertical and fore-and-aft motion track correctly on both arms. Lateral does not, and the measurements show why no axis map or recalibration can fix it. The cost of each channel reduction is quantified on 20 430 recorded frames, and the endpoint of the ladder reports position unavailable rather than inventing one.

A safety architecture validated by injection. Fourteen of fourteen injected faults handled. The two that took several attempts both failed for the same underlying reason, which is that a mechanism was keyed on the wrong observable: a dead-man on message arrival cannot detect a source that keeps publishing stale data.

A workspace characterisation. The two arms share no reachable workspace under the as-built mount, which blocks inter-arm handover for geometric reasons no software change can address. Asked four separate ways, the working space in front of the wearer's own chest is unavailable at any surface height or distance, and deleting the wearer's arms from the model moves the boundary by at most 75 mm, which identifies the torso rather than the limbs as the constraint. The quantified mount modification was subsequently applied as one deliberate pass, raising the clearance ceiling from 0.1610 m to 0.2202 m and bringing the task's placement targets from 0.530 m to 0.290 m off the centre line.

A verification methodology. Ten repeats over densified paths rather than single calls at endpoints, and an instrument validated against a known answer before a surprising measurement taken with it is reported. The five-task verification reproduced exactly across runs, which is the first reachability figure in this project to do so.

Motion generation and transfer. The per-joint step limiter that constituted the entire motion generation was replaced with a jerk-limited, phase-synchronised generator, removing 51.3 mm of unrequested Cartesian excursion on a platform whose capture gate is 30 mm, and bringing peak commanded joint speed within the manufacturer's declared limit for the first time. The simulation-to-hardware gap was measured rather than assumed: every joint parks 0.305° short on the side it approached from, which one scalar per arm removes three quarters of.

Measurement in place of declaration. A calibration stage measures the working surface and the objects on it, recovering six objects with every centre within 1 mm, and hands the result to a planner that had previously known about nothing except the wearer.

Identification of what blocks progress. Seven items, listed in [Chapter 10](#) in the order they should be addressed. The first is two soldered joints.

11.2 What was not achieved

No result in this thesis was validated on a physical Kinova arm carrying a load. One session drove a single arm from its own camera and produced three findings, all of them about calibration and none of them a performance result ([Section 7.14](#)). The task set is verified geometrically and has never been performed with a wearer in the harness. The two-person study the architecture was designed around was not run, and could not have been: the direct teleoperation baseline depends on channels that are currently incoherent.

The planning report's haptic objective was not delivered and, as [Chapter 10](#) argues, is not deliverable on this platform through the available control interface.

11.3 Further work

Three of the findings above point at work that is cheap relative to what it would unblock, and they are worth separating from the general list because each of them is a measurement rather than a build.

Two soldered joints. Left j2 and j4 are named by the observability regression rather than chosen: with both frozen the left arm retains no observable radial dimension at all, and repairing any other channel does not restore one. They are the reason the direct-teleoperation baseline of [Chapter C](#) would not be representative of a working master, and therefore the reason the study's central comparison could not be made.

One hand-eye calibration. The transform between the wrist camera and the arm is wrong by roughly 8.5° in a way that rotates with the wrist, which is larger than the entire capture gate at working distance ([Fig. 6.1](#)). It is the largest unmeasured term in the positioning budget and the reason no open-loop grasp planned through it has landed inside that gate. It needs the hardware and a printed target.

One clearance check that measures link geometry rather than link origins. At present the check samples link origins, so it returns a comfortable margin for a pose in which the arm is against the mannequin ([Section 10.7](#)). This is the only item on the list that is a safety defect rather than an incompleteness, and it should be closed before anybody wears the rig.

Beyond those, the substantive open questions are hardware ones. The right arm's home joint angles have never been read from the machine, and the workspace geometry depends on them; making the two arms' reachable sets meet requires re-parking that arm mechanically, because [Chapter 9](#) establishes that no control or planning change can do it. The safety layer has never run against a real arm, and the Kortex session-recovery path is the part a real session meets first. And the exchange rate between operator performance and wearer trust — the question this architecture was built to ask — remains open, because it needs two people and an approved protocol rather than more engineering.

11.4 Closing

The most transferable outcome of this work is not the mapping or the safety architecture. It is the discipline that produced them. Twenty-six times a result that looked like a system failure turned out to be a defect in the measuring tool, and fifteen of those were caught only because something else already had the answer. On a robot that stands beside a person, the difference between a measurement and an artefact is not an academic distinction, and the habit of establishing which one is in hand before acting on it is the part of this project most worth carrying forward.

Chapter A

The route the design took

This appendix records how the control approach reached the form reported in [Chapter 4](#), including the routes that were tried and abandoned. It is a record of process rather than of result, which is why it is here and not in the body.

The system described in [Chapter 4](#) is not the one the planning report proposed, and the route between them is part of the result. This chapter records the sequence, the alternatives that were considered at each branch, and what each abandoned approach contributed to the one that shipped. The divergence from the planning report is analysed separately in `thesis_report/DIVERGENCE.md`.

The account below is no longer reconstructed. The prior programs were recovered and are archived at `thesis_report/_source/prior_work/`; file dates give the calendar directly, running from 7 June to 27 July 2026.

A.1 Phase 1: joint-space control in simulation

The planning report proposed validating the master and slave mapping in Isaac Lab or MuJoCo before touching hardware. MuJoCo [32] was chosen over Isaac Lab [24]. It starts in under a second, its model format is a single readable file, and for a kinematic mapping question its contact solver is irrelevant, so the heavier alternative bought nothing.

The first mapping was the obvious one. The master reproduces the Gen3’s joint sequence, so each master potentiometer angle was sent to the corresponding slave joint after zero-referencing. Joint one to joint one, and so on through seven.

Three things were learned, and all three survived into the delivered system.

The mapping is dominated by convention handling, not by control. Almost all of the difficulty was in what a joint angle *means*: which direction is positive, where zero sits, whether the value wraps, and over what interval. This is where the degree and radian conversion described in [Chapter 1](#) was first written, and it is the reason it later became an isolated, unit-tested module rather than an inline expression. The lesson transferred directly: the arithmetic that converts between conventions deserves more test coverage than the control law it feeds.

An unconstrained joint mapping will drive the arm into things. In simulation this is a visual annoyance. On the delivered platform, where the arm is mounted beside a person’s head, it is the entire safety problem. A joint-space mapping has no representation of the wearer, so nothing in the formulation can express the constraint that matters most. That observation is what motivated the move to a framework with a collision model in it.

Sensor faults are the dominant failure mode, not control faults. Injecting a stuck or noisy channel showed the arm following it faithfully, because a direct joint mapping has no notion of a reading being implausible. The channel-health architecture in [Chapter 4](#) is a direct response.

A.1.1 What the prior programs actually did

Five of the recovered programs import MuJoCo and none contains an inverse kinematics call of any kind. The mapping is written literally as a joint-space assignment: `data.qpos[k1_qpos[j]] = k1_target[j]` in `two_armsbackpack.py`, and the same pattern by way of degrees in `srl_teleop.py`. Searching the whole set for *inverse* or *jacobian* returns nothing. The phase was therefore joint-space throughout, exactly as described, and the absence of any solver is what makes the later move to a task-space command a change of kind rather than of degree.

Two facts recovered from the source correct the account given previously.

The prior system already drove the real arm. `srl_teleop.py` imports the manufacturer’s Python API directly and calls `SendJointSpeedsCommand`, gated behind a `USE_REAL` mode. It is the *same high-level velocity interface* the delivered system uses, arrived at independently and about two months earlier. So the move to ROS 2 was not driven by a need for vendor drivers, as an earlier draft of this chapter supposed: the arm was already reachable. It was driven by the collision model, the solver and the controller lifecycle, which is a narrower and more honest claim.

The link lengths have a documented origin. The seven values the forward kinematics uses were not first written in Python. They appear in `basic_control/kinematics.h` as C++ constants, with the comment “physical link lengths (metres), shaft-center to shaft-center, measured directly on the physical master arm”, alongside the joint pattern `J1 roll, J2 bend, ... J7 roll`.

That provenance settles what the CAD comparison in [Chapter 7](#) can claim. *Shaft-centre to shaft-centre* is precisely the joint-module pitch measured off the STEP assembly, so the two quantities are commensurable, and their agreement to 1.2 % is a genuine cross-check rather than a coincidence of similar numbers. It also explains why the individual 43 mm and 37 mm figures cannot be separated from the CAD: neither the header nor the geometry distinguishes them, only their sum.

A.1.2 Why joint-space control was abandoned

Joint-space correspondence has a genuine advantage that should be recorded before the case against it: it fixes the redundant degree of freedom for free. The Gen3 has seven joints for a six-dimensional task, so a task-space command leaves one dimension unspecified and something has to choose it. Under a joint mapping the operator chooses it implicitly, by the posture they put the master in.

It was abandoned for two reasons that are both about the master rather than the method.

The first is scale. The master’s links sum to 0.272 m against the Gen3’s 0.902 m reach. Equal joint angles therefore produce tip displacements differing by a factor of about three, so the operator’s motion and the robot’s motion are related by a gain that varies with posture. Working in task space makes that gain an explicit, tunable scalar instead.

The second is decisive. A joint mapping requires *every* joint to be measured, because each one contributes directly to the commanded configuration. It has no graceful degradation at all: a single failed channel sends a

wrong angle to the corresponding slave joint, and there is no reconstruction available because nothing else in the mapping observes that joint. Given the argument in [Chapter 1](#) that a body-worn master will lose channels, a formulation that cannot survive losing one was not viable.

A.2 Phase 2: the move to ROS 2

MuJoCo answers questions about dynamics. It does not supply a collision-aware inverse kinematics service, a controller stack, or drivers for the actual arms, and by the end of phase 1 all three were needed. The system moved to ROS 2 Jazzy [\[22\]](#).

Four capabilities came with the move, and each closed a problem left open by phase 1.

- **A collision model containing the wearer.** MoveIt checks candidate configurations against a scene, so the wearer’s body can be part of the model. This is the capability a joint mapping could not express at all.
- **Inverse kinematics with redundancy resolution.** TRAC-IK solves the seven-joint problem with random restarts and a distance objective [\[1\]](#), which makes task-space commanding practical and lets the redundant degree of freedom be re-seeded on failure.
- **A controller stack with a hardware abstraction.** The same trajectory interface addresses a mock system and the real driver, so everything upstream is developed and tested unchanged and the substitution is one launch argument.
- **Vendor drivers.** The manufacturer supplies a ROS 2 driver, so the alternative was writing one.

The cost was real. ROS 2 introduced a discovery layer, a middleware configuration, and a process lifecycle, and a substantial fraction of the faults recorded in this project are in that layer rather than in the robot. [Chapter 7](#) reports them, because they are the ones a successor will meet first.

A.2.1 What was kept from simulation

Simulation did not stop being useful when hardware became available. It became the place where anything that could not be attempted on a person is attempted. Every workspace measurement in [Chapter 9](#), every fault injection, and every clearance figure is simulated, and that is a deliberate allocation rather than a fallback. The mock hardware interface is what makes it possible to exercise the entire stack, including the trajectory controllers and the emergency stop, without an arm attached.

The limit of that substitution is stated where it binds. The mock returns identically zero for every force, so anything derived from a force reading is structurally undefined rather than merely unmeasured.

A.3 Phase 3: from Cartesian control to a reduced-degree-of-freedom command

Task-space control was not the end of the sequence. The first Cartesian implementation computed the master’s tip pose by forward kinematics from all seven potentiometer angles and commanded the end effector to the scaled displacement. That is the textbook formulation and it failed on this hardware.

The failure was directional rather than noisy. Decomposing the commanded positions from five distinct gestures put 93.4% of the variance on a single axis, and placed the pose commanded by raising the master *below* the pose commanded by lowering it. A mapping that inverts up and down is not a mapping with a gain error.

The cause is that forward kinematics compounds every joint angle in the chain. An error in joint three propagates through joints four to seven, so the tip direction inherits the sum of seven absolute-angle calibration errors, and the inertial mount calibration had already shown those errors to be large enough to be physically impossible in places.

The response was to stop computing direction from the chain. Position is now built in spherical form about the master's shoulder, taking each component from whichever sensor observes it most directly: elevation from the gravity vector, azimuth from the single shoulder-roll potentiometer, and radius from the forward-kinematic magnitude, which depends only on the bend joints and is insensitive to roll errors. [Chapter 4](#) gives the formulation and [Chapter 7](#) the measured consequences, including the one axis that this does not fix.

A.3.1 Alternatives considered at this branch

Recalibrate the potentiometers and keep full forward kinematics. Rejected on cost and on durability. It requires an external angular reference per joint, and it has to be redone whenever a joint is reassembled. It also fixes nothing about channel loss, since forward kinematics still needs every joint.

Take azimuth from forward kinematics instead of from joint one. Built and measured, then rejected. It made direction worse on both arms, moving the mean reversal error from 12.6° to 33.3° on the left arm and from 26.2° to 39.5° on the right. It fails for the same reason the full chain fails: it compounds four suspect angles where the single-joint estimate compounds one.

Integrate the gyroscope for azimuth. Implemented, measured, and left disabled by default. It observes rotation about vertical directly and so avoids both error sources, and it halves the residual on the left arm. It is not enabled because one of the two sensors drifts at $44^\circ/\text{min}$ while stationary, and because the capture that would validate it needs a moving master and does not yet exist.

Add a second inertial sensor on the upper arm. Not attempted, for the hardware reason in [Chapter 3](#): both available addresses are occupied. It remains the most promising route, and [Chapter 7](#) quantifies what it would buy.

A.4 Summary of the sequence

Table A.1: Development sequence, with the reason each stage ended.

Stage	Mapping	Why it ended
MuJoCo, phase 1	joint to joint	No collision model, so the wearer could not be represented. No degradation path: one failed channel corrupts one slave joint irrecoverably.
ROS 2, first Cartesian	tip pose by full forward kinematics	Direction collapsed. 93.4 % of variance on one axis, and raising the master lowered the arm. Seven compounded calibration errors.
ROS 2, delivered	spherical, per-component sensing	Current. Elevation and reach are correct; azimuth remains limited by the lever-arm coupling described in Chapter 7 .

The through-line is that each stage failed because of what it assumed about the *sensors*, not because of what it assumed about the control. Joint mapping assumed every channel works. Full forward kinematics assumed every angle is accurate. The delivered mapping assumes only that gravity points down and that the best-conditioned joints are identifiable at run time, which are assumptions the hardware can actually support.

Chapter B

Measurement validity: the full record

Section 10.7 states the validity argument and the two numbers that carry it. This appendix is the evidence behind it: all twenty-six occasions on which a measurement in this project was wrong and the system was not, what each actually was, and what revealed it.

Twenty-six times during this project a measurement was wrong and the system was not. This chapter reports them as a body of evidence rather than as a defect list, because the pattern across them is the most transferable outcome of the work.

Most engineering write-ups treat measurement as transparent: the apparatus reports, and the report is the finding. That assumption failed here often enough to be worth quantifying. In every case the wrong reading was *plausible*. None announced itself. Several would have entered this thesis as results had a second check not contradicted them, and two would have removed findings that are now central to it.

B.1 The rule

A surprising failure is evidence about the instrument until the instrument has been cleared.

Four corollaries, each learned from a specific case below.

1. Every analysis script gets a **known-answer test**: an input whose answer is known independently, checked before the script's output is believed.
2. Prefer **real recorded data** whose properties were established separately.
3. Use synthetic data only where the ground truth is **constructed** (arithmetic, geometry), never where it is **rendered** (images, physics). Two points 300 mm apart really are that far apart; a rendered image is only as good as the renderer.
4. A result that **contradicts an earlier measurement** is an instrument check, not a finding, until the two are reconciled.

B.2 The cases

[Table B.1](#) lists the sixteen recorded during the interface work and [Table B.2](#) the ten recorded afterwards, during the perception, motion generation and hardware work. The split is itself informative: the rate did not fall as the

project matured, because each new capability brought a new instrument with it, and the later cases are on the whole more expensive than the earlier ones.

B.3 Four mechanisms

Sorted by mechanism rather than by symptom, the twenty-six fall into four groups. None is specific to robotics.

Absence read as a value. A known-answer check that passed on zero parsed rows; a scene sweep that received no messages and concluded every object had gone; a noise floor of zero produced by a recorder outrunning its sensor. In each case code conflated *no data* with *a measurement of zero*. This is the most dangerous group, because absence is silent by definition and the resulting value is usually the most reassuring one available.

The harness constructs the condition it tests. A GUI test that allocated the terminal in which the bug cannot occur; a detection rate taken through a renderer the detector was never trained on; a fault injected at 30° against a 60° threshold. A test written from the same understanding as the code inherits that understanding's errors.

State left over from a previous run. An anchor read while the arms were still parked where the last run left them; a reachability sweep taken with the arms 155 to 166° from home; a boundary probe firing from the previous run's end point. Scripts are written as though they begin from nothing, and they do not.

Order of operations inside the measurement. Reading a reference after perturbing it, and screenshotting after the window closed. Both are easy to do and invisible in the output afterwards.

A constant asserted in more than one place. The twenty-sixth case belongs to none of the four and is worth naming separately, because it is the one this project is most likely to repeat. Three files each carried their own answer to "which way up is the camera", two of them agreeing with each other and none of them measured. There was no single place to be wrong, so there was no single place to check, and the disagreement between the files was silent because no artefact was ever compared across them. The remedy is structural rather than diligent: one owner for the fact, one function that applies it to the image and the intrinsics together so that half of it cannot be applied, and a stored record of the measurement that set it.

B.4 What actually caught them

The defence that worked was not care. It was **redundancy**: having an independently known quantity to check against.

Fourteen of twenty-six were caught because something else already had an answer: a vendor's published stroke, a published detection score, an earlier incompatible result, an arithmetic prediction, a declared quantity that is exact by construction. Fifteen were caught by an independently known quantity contradicting the reading; only five were caught by inspection, and those were the shallowest.

This has a direct design consequence, and it is the recommendation this chapter exists to make. **Build the cross-check in rather than trying harder.** A measurement with no independent reference is not merely unvalidated; it is structurally unable to be validated, and no amount of diligence changes that. Where a project cannot obtain

Table B.1: Every occasion on which a measurement was wrong and the system was not. *Caught by* is the thing that actually revealed it, not the thing that should have.

Looked like	Actually was	Caught by
Noise floor exactly zero on all 14 channels	The recorder sampled a 14.7 to 17 Hz sensor at 50 Hz, so 82 to 88 % of adjacent rows were identical	The figure being <i>too</i> clean; over distinct updates the real floor is 0.2 to 5.4°
Isotropy swinging by a factor of two between runs	The statistic, not the measurement: min-over-directions is maximally sensitive to one tail failure	Repeating to $N = 10$, where the inter-quartile range is 0.000 m
Object detection at 0 to 4 %	The renderer: flat-shaded primitives are outside the detector’s training distribution	The same model scoring 0.89 to 0.91 on a real photograph
A carry path failing 0 of 19 placements	The path template carried toward the wearer, the one direction leaving the feasible band	Contradiction with an earlier result showing 19 working grip pairs
An AprilTag renderer detecting nothing	A mirrored tag: OpenCV’s camera has $+y$ down, reversing the winding	A vertical flip of the same crop decoding immediately
A flat 22° pose error	The ground-truth rotation applied in the wrong frame; the two forms <i>commute</i> about x and y	Testing a general in-plane axis: 21.79° against 1.06°
2–3 processes surviving a clean shutdown	The check grepped bare names and matched its own command line	Matching installed executable paths, which a command line cannot contain
A terminal GUI launching correctly	The test built the one environment where the bug cannot occur, and checked only for captions	Running the real entry point and asserting on exit status
The gripper model wrong by 52 mm	The measurement took the gap between link <i>origins</i> , which sit behind the pad faces	The vendor’s published 85 mm stroke: corrected, it lands at 84.84 mm
A wrist rotation of 0°, twice	First reading the anchor after the arm had moved onto the grasp pose; then reading it when a <i>previous run</i> had left it there	Contradiction with an independently known 169.7°
A known-answer check passing	It parsed zero rows, and <code>all()</code> over an empty collection is true	The output table being empty while the check said pass
A health classifier missing an injected fault	The injected fault was a 30° <i>circular</i> jump, correctly below the 60° threshold	Asking why only one of two injected faults was detected
RViz failing to render inside a Qt container	The capture fired after the probe window had closed	Re-capturing during the window’s lifetime
A scene fingerprint reporting every object vanished	The sweep began before discovery matched the detector and received <i>zero</i> messages	Running it twice on an unchanged scene
A boundary probe reporting a wall that was not there	The test commanded an orientation unreachable at 0 of 6 points on that line	Instrumenting the node’s own probe trace
Nine sling clips with no object	Two faults at once: a stale 350 mm length making the sling geometrically impossible, then floors calibrated at the wrong image scale	Looking at a frame, and the overlay reading <code>sag=0mm</code>

Table B.2: The nine further cases, recorded during the perception, motion generation and hardware work. Four of them produced a *passing* result on a broken system, which is the more dangerous direction.

Looked like	Actually was	Caught by
Every declared coordinate of a task unreachable	Every reachability sweep in the project solved at the <i>home</i> wrist while the task layer commands a stored anchor, 42.9° away	A known-good pick reading 10 of 10 at the anchor and 0 of 10 at the home wrist
A working surface 5.2 mm below where it was placed	The reported height was the fitted plane’s intercept <i>at the origin</i> , a point inside the wearer, rather than its height over the working area	Contradiction with the declared surface height, which is exact by construction
367.0 mm of clearance	The arm was physically against the mannequin. The model samples link <i>origins</i> , so it cannot see the arm between joints, and it returns the same constant for a clear pose	Looking at the arm
An arm reporting the saved pose to 0.144°	A leftover node republishing a frozen snapshot with fresh timestamps after its own input had died	The agreement being <i>too</i> exact; distinct source stamps then settled it
A test suite reporting a failing test named =	The style checker echoes the source lines it objects to, and one file contained the token FAILED inside a string literal, which the gate’s pattern matched	Reading the name of the failing test
A scene rendering correctly at 18 % ink coverage	The scene was empty. The wearer alone occupies 18 % of the frame; a populated scene reads 46 to 50 %	Looking at the image
One task’s geometry verified under another task’s name	A conditional expression with the same operand on both branches, so the check read the first task’s objects whichever task it was asked about	Reading the expression
A jerk-limited generator 0.0509 mm off its own path	The reference curve was sampled on a 0.1 mm grid, so nearest-sample distance could not go below half a grid step	The generator being on the line to 2×10^{-16} rad by construction
An arm stalled 360.076° from its target	Three joints are continuous, so −179.05° and 181.02° are the same configuration. The move had completed	The commanded and achieved poses being physically identical
Every scene-camera figure recorded upside down	Three files decided independently whether the camera is inverted, two rotating and one not. None had measured it; the camera is the right way up (Section 8.2)	A second camera watching the same room, correlated against the first in both orientations

Table B.3: What revealed each of the twenty-six.

Revealed by	Cases
An independently known quantity contradicting the reading	15
Inspection of the code or the method	5
Looking at the raw artefact, usually an image	6

a second reference for a quantity, the honest response is to mark it unverified rather than to raise confidence in the single measurement.

B.5 Consequences for this thesis

Every quantitative claim in this document carries a maturity label and a traceable measurement, and the audit records for each whether its instrument was validated. Of thirty-four figures audited, fifteen are validated against an independently known answer, fifteen are arithmetic or graph inspection with no instrument to mis-calibrate, three are unverified and are not reported as results, and one is structurally undefined in the available environment.

Two practices in the later chapters follow directly from this chapter and are worth naming, because both were adopted after a case above rather than in advance. extbfA check is required to be capable of failing on a deliberately broken input: the on-surface geometry check, for instance, takes an injection argument that displaces the scene, and one instrument then returns three different verdicts about one scene, including a pass. And extbfa refusal is required to name what it refused on, which is what exposed a wearer-tracking gate that had never executed while its enclosing test reported seven of seven.

One capability was **cut rather than shipped** on this basis. A boundary-warning node produced a correct 88 mm lead in three of five runs and a false alarm in the other two, and the cause survived elimination of every candidate mechanism. A warning that is wrong teaches the operator to ignore it, which spends the credibility on which every other alarm in the system depends. Suppressing the symptom without knowing the cause was available and was refused.

This chapter is placed in the main body deliberately. It is a methods contribution rather than a result, and the twenty-six cases it reports are the reason several numbers elsewhere in this thesis are stated with the qualifications they carry. Read as an appendix it would be a list of bugs; read in place it is the argument for why the measurements that survived are worth believing.

Chapter C

Protocol for the two-person study

The study specified here was not run, for the reasons given in [Section 10.8](#). The protocol is recorded in full because it is a deliverable of the project in its own right — the design decisions are costed against measurements reported in [Chapter 7](#) — and because a protocol that exists is the difference between a study that can be run next and one that has to be designed first.

The system was built to make one measurement possible. This chapter specifies that measurement completely: the question, the hypotheses, the design, the instruments, the procedure and the analysis. None of it has been executed. Setting it out in full is what makes [Chapter D](#) an empty container rather than an omission, and [Chapter 10](#) states in priority order what must be resolved before it can be run.

C.1 The question

Does supplying part of the operator's control with autonomy improve performance, and what does it cost the person carrying the arms?

The second clause is the contribution. The shared-autonomy literature measures the operator [[7](#), [16](#)]; the near-body interaction literature measures the wearer [[40](#)]; no study measures both under the same autonomy in the same trial, which is the gap identified in [Chapter 2](#). Until that is done the exchange rate between them is unknown, and an improvement of twenty per cent in completion time at the cost of a point of wearer trust cannot be evaluated as a good trade or a bad one.

C.2 Hypotheses

Stated with direction, dependent variable and test before any data exists.

- H1.** Task completion time is shorter under shared autonomy than under direct teleoperation. *Dependent variable:* completion time per trial. *Test:* repeated-measures analysis of variance across mode within dyad, with a rank-based equivalent if the residuals fail inspection.
- H2.** Operator workload is lower under shared autonomy, on the task-load index total and on the mental-demand subscale specifically [[11](#)].
- H3.** **Wearer-reported trust is lower under shared autonomy**, against the direction of H1 and H2. This is the directional prediction taken from the concealed-operator study [[40](#)], tested here with autonomy that is genuinely a program and a wearer who has a concurrent task of their own.

- H4.** The wearer’s sense of agency is lower under shared autonomy while the sense of ownership is unchanged, on the grounds that the two dissociate [20] and only one of them has a functional referent when the operator is a different person.
- H5.** The cross-arm interference coefficient in the dual-pursuit task is lower under shared autonomy, and the accompanying phase lag is not, which would identify the benefit as attentional rather than as a change in tracking delay.

H3 and H4 are the hypotheses worth running the study for. H1 and H2 are close to assumed and exist principally to establish that the platform can show an effect at all, which is what makes a null result on H3 or H4 interpretable rather than merely quiet.

C.3 Design

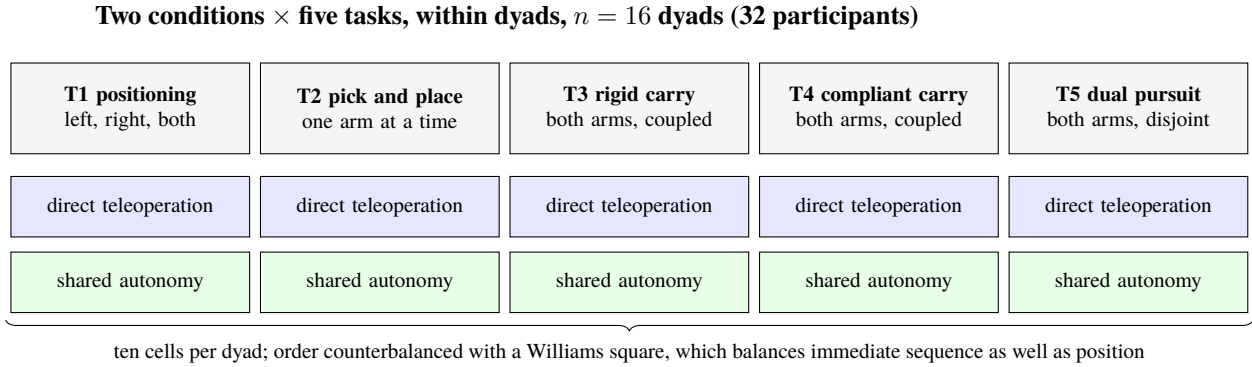


Figure C.1: Conditions crossed with tasks, together with the design options that were costed and rejected. Within-dyad throughout: between-subject variance on this task is larger than the effects sought.

Two modes, direct teleoperation and shared autonomy, crossed with five tasks, within subjects, at sixteen dyads. Roles are fixed for the whole session: swapping them mid-session would halve the effective sample and confound role with fatigue, and the wearer’s role involves carrying 17 kg, which nobody should do twice in one visit.

Two modes rather than four is a costed decision, not a convenience. Four modes crossed with five tasks is 260 to 320 min of task time against a 5.5 to 6.5 h session, which exceeds by a wide margin how long anybody should carry 17 kg. Three alternatives were priced: two modes across five tasks at ten conditions and 167 min; four modes on a single task at eight conditions and 169 min, which still breaks the exposure cap; and a between-subjects design across modes, which needs ninety-six participants to reach the same sensitivity because between-subject variance on this task is larger than the effect sought. The two-mode design is the only one of the four that fits inside the wearer’s exposure limit while retaining the task contrast, and dropping the controller and full-autonomy modes is recorded as a real loss of scope rather than absorbed silently.

Mode order is counterbalanced with Williams squares rather than a Latin square, because a Latin square balances position but not immediate sequence, and on a task with a learning curve the sequence effect is the one that bites. At $n = 16$ the design achieves zero position imbalance and zero sequence imbalance, verified by the counterbalancing module rather than assumed.

Table C.1: The three designs that were costed before this one was chosen. Session length is task time plus fitting, breaks and questionnaires; the exposure cap is the 48 min for which a person should carry 17 kg in one continuous block, and the pack-on total is what the design actually imposes. Detectable effect is at 80 % power, $\alpha = 0.05$.

Design	Cells	Session	Exposure	Detectable effect
Mode between subjects	5	127 min	within cap	$f = 0.40$, needs $n = 96$
Four modes, one task	8	169 min	exceeds cap	$d = 0.73$, $n = 32$
Two modes, five tasks	10	167 min	within cap	$d = 0.73$, $n = 32$

C.3.1 Which cells exist

One row of the design is unavailable, and it is a measured property of the platform rather than a design decision.

Grasping under direct teleoperation is not achievable. The default orientation mode pins the commanded wrist to the anchor captured at the home pose, the task-set block requires a rotation of 169.7° from that anchor, and there is no master channel able to command it (Chapter 7). The recorded grasp demonstrations bypass the teleoperation orientation lock by calling the solver directly, which establishes that the *robot* can execute an aligned grasp and not that an *operator* can command one.

T2 therefore carries no mode contrast and must be analysed as a single condition. It is retained because grasp success and placement accuracy under autonomy are worth measuring in their own right, and because it is the one task where the categorical rather than quantitative advantage of autonomy on this platform can be stated: the robot can execute an aligned grasp and an operator cannot command one.

The coupled-carry tasks carry a second constraint that is geometric rather than sensory. The object presents 113.9 mm to the right gripper along its closing axis against an 85 mm jaw opening (Chapter 7), so the carrier must be re-specified to a grip the hand can close on before those two rows can be run. The re-specification is derived in Chapter 9 and the geometry is fixed; what remains is to build it.

Both are stated in the design rather than in the limitations, so that the empty cells in Chapter D are not read as data that was lost.

C.4 Tasks

The first task is a positioning task with three target widths at each amplitude, which characterises the interface against the speed-accuracy relation [9] and supplies the single-arm against both-arms contrast that measures the attention bottleneck directly. It is the only task available in every condition and therefore carries the main comparison.

C.5 Measures

Exertion is taken from the wearer between every block rather than once at the end. Fatigue under a carried load rises monotonically, so a single measurement at the end converts a covariate into a guess. The task load index is administered to *both* participants, because the two roles do not load the same way and the comparison between the two columns is the object of the study.

The cross-arm interference coefficient requires a caveat that travels with it everywhere it is reported. It is the total sensitivity of one arm's tracking error to the other arm's target speed, and tracking error contains both an attentional amplitude term and a lag term, since a fixed reaction delay L becomes a positional error vL . The two

Table C.2: The task set, with what each measures and its current availability. Availability is a measured property of the platform, reported in [Chapter 7](#) and [Chapter 9](#).

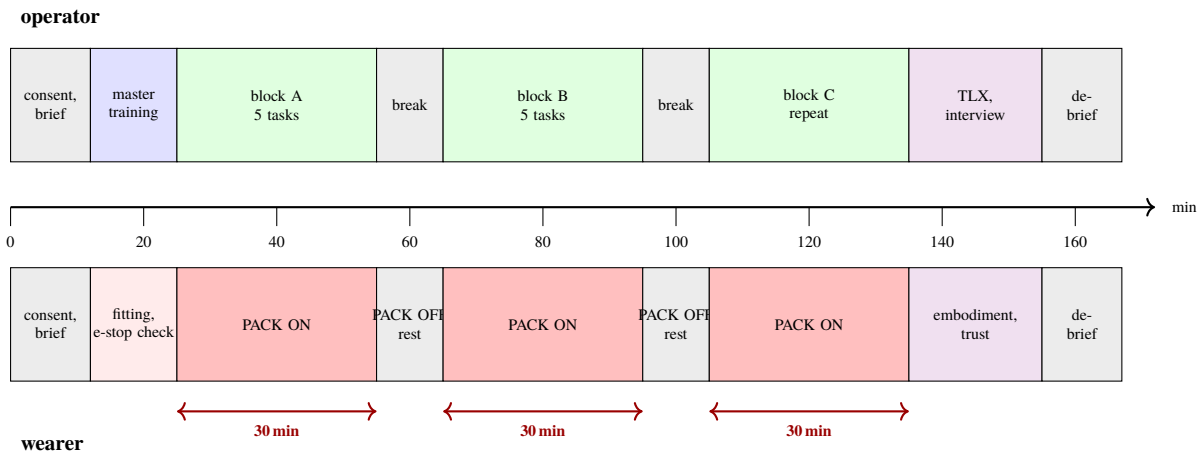
Task	Measures	Mode contrast available
T1 positioning, left, right and both arms	Throughput against index of difficulty [9]; the single-arm against both-arms decrement, which measures the attention bottleneck directly	both modes
T2 pick and place, one arm at a time	Grasp success, placement accuracy, recovery from a missed grasp	shared autonomy only: see below
T3 rigid carry, both arms coupled	Coordination error as object tilt, logged continuously through the carry rather than scored at the end	both modes
T4 compliant carry, both arms coupled	Coordination error as gripper separation against the carrier’s retention threshold	both modes
T5 dual pursuit, both arms disjoint	Cross-arm interference under divided attention	both modes

Table C.3: Every quantity recorded, its source and its cadence. Automatic measures are written by the trial logger, one file per trial, flushed per sample.

Measure	Source	Cadence
Completion time	automatic	every trial
Throughput, index of difficulty	automatic	every trial, positioning task
Path efficiency	automatic	every trial
Success or failure	automatic	every trial
Clutch re-engagements	automatic	every trial
Cross-arm interference coefficient	automatic	dual pursuit
Minimum wearer clearance	automatic	every trial
Emergency stops, with cause	automatic	every trial
Task load index [11]	both participants	after each block
Physical exertion [3]	wearer	after each block
Trust	both participants	after each block
Ownership and agency [20]	wearer	end of session
Free comment	both participants	debrief

are not separable from root-mean-square error alone. The measured phase lag is therefore reported beside it: a large coefficient with a flat phase lag is attentional, and both rising together is not.

C.6 Procedure



Pack-on time totals 90 min and never exceeds 30 min continuously. Two fatigue processes run in this session and they are not interchangeable. The operator fatigues from holding an uncompensated master arm against gravity, which a Borg CR10 probe between blocks converts from an unmeasured confound into a covariate. The wearer fatigues from carrying 17 kg while doing nothing, which no counterbalancing can remove because it is monotonic in time. The pack-off rests exist for the wearer, not for the operator.

Figure C.2: Session structure for two people. The wearer's exposure to the carried load is the binding constraint and is drawn as its own track.

Three people are present: the two participants and a safety observer whose only task is to hold an emergency stop and watch the wearer rather than the screen. That role exists because the operator is attending to the task and the wearer cannot see behind their own shoulders. The observer's presence is enforced in software as well as procedurally: the safety node treats two seconds of silence from the observer's heartbeat as withdrawal of consent to move, which is described in [Chapter 4](#).

Consent is taken separately and in separate rooms. The two participants may know one another, and a wearer who feels observed by the person who recruited them is not consenting freely.

The emergency stop is demonstrated and exercised with both participants watching before any task begins, and the wearer is shown a stop within their own reach.

C.7 Analysis

Comparisons are within dyad across mode. Where residuals permit, a repeated-measures analysis of variance; otherwise its rank-based equivalent. Post-hoc comparisons are Holm-corrected, and raw and adjusted values are reported side by side so that the effect of the correction is visible.

Effect sizes are reported with confidence intervals, and the intervals rather than the significance carry the interpretation. At sixteen dyads the design is powered for a large effect on completion time and is not powered for a small one; that is stated in advance rather than discovered afterwards. The effect sizes used in the power calculation are estimates and are labelled as such: they must be replaced with pilot values before the design is

locked.

Trial validity is defined in one module shared by every analysis (`srl_experiments/validity.py`) so that the definition cannot drift between experiments. It excludes on the recorded validity flag, prints every exclusion cause, and **exits non-zero when no trials survive or when more than half were excluded**. Before that existed, a session in which every trial had been invalidated analysed cleanly to zero valid trials followed by empty means and undefined intervals, which is indistinguishable from a null result to anyone reading the output.

C.8 Ethics and risk

The application is drafted and has not been submitted. A study cannot honestly be submitted for approval while its baseline condition depends on sensor channels that do not work.

The dominant risk is stated first: a 17 kg manipulator pair mounted behind a seated person, moving under partly autonomous control, within reach of their head and torso. Everything below follows from it.

Table C.4: Principal risks to participants and the measures in place. Software mitigations are those described in [Chapter 4](#); the remainder are procedural.

Risk	Mitigation
Contact with the wearer	A 150 mm margin enforced over the whole path, measured geometrically against the fused body rather than through the semantic collision description, with the arm holding rather than slowing at the limit
The wearer cannot stop the arms	A stop within the wearer's own reach and a second held by the safety observer, whose heartbeat gates motion
Carried load	Twenty minutes of exposure at a time, seated where the task permits, exertion rated between blocks, session ended on request without explanation
Fatigue confounding condition	Order counterbalanced; exertion recorded as a covariate rather than assumed constant
Perceived inability to withdraw	Consent taken separately; the right to stop restated at every break
Identifying data	Refused in code: <code>write_manifest()</code> raises on name, email address, date of birth, address or telephone number

Two notes on that table. The anonymity constraint is enforced in the program rather than in a policy document, because a policy document does not stop a keystroke. And video recording, which would be analytically useful, is not part of the protocol: a video of a participant wearing this rig is not anonymous, and no measure in [Table C.3](#) requires one.

Chapter D

The analysis procedure, and its behaviour on constructed data

The analysis that [Chapter C](#) would be evaluated with, written out, and measured against data whose answer was chosen. No participant data exists; what is reported here is a property of the procedure.

No participant data has been collected.

The study of [Chapter C](#) has not been run. This chapter therefore reports what *can* be established without participants, which is more than nothing and less than a result: the analysis that will be applied, written out in full, and a measurement of how that analysis behaves on data whose answer is known. The reason no session has been run is not administrative and is stated in [Section D.5](#).

An analysis specified but never executed is an assumption. The pre-registered plan for this study carries one number — 80 % power to detect $d = 0.73$ at sixteen dyads — and that number came from a power formula for a single paired comparison, while the analysis that will actually be run applies a family-wise correction across five of them. Those are different quantities, and the difference is not small. This chapter measures it.

D.1 The analysis, written out

The design is within dyads: every dyad performs every condition, so the unit of analysis is the paired difference and not the raw score. For task j , with y_{ij}^{DT} and y_{ij}^{SA} the outcome for dyad i under direct teleoperation and shared autonomy,

$$d_{ij} = y_{ij}^{\text{SA}} - y_{ij}^{\text{DT}}, \quad t_j = \frac{\bar{d}_{\cdot j}}{s_{d,j}/\sqrt{n}}, \quad \nu = n - 1. \quad (\text{D.1})$$

The effect size reported alongside is the standardised mean of those differences, which is the quantity the design was powered for and is not the same as the standardised difference of the two means:

$$d_z = \frac{\bar{d}_{\cdot j}}{s_{d,j}}. \quad (\text{D.2})$$

The distinction is worth stating because conflating the two deflates every effect by a factor of $\sqrt{2}$ when the two conditions are uncorrelated, and by more when they are positively correlated, which within a dyad they are.

Five tasks give a family of five comparisons. The pre-registered correction is Holm’s step-down procedure: with the raw values sorted ascending as $p_{(1)} \leq \dots \leq p_{(m)}$,

$$\tilde{p}_{(k)} = \max_{l \leq k} \min \{ (m - l + 1) p_{(l)}, 1 \}, \quad (\text{D.3})$$

the outer maximum enforcing monotonicity so that a comparison cannot be declared significant while a stronger one is not. Holm controls the family-wise error rate at α under any dependence structure and is uniformly more powerful than the plain Bonferroni bound it refines, so there is no configuration in which Bonferroni would be preferred.

D.2 What the analysis does to data whose answer is known

Equations (D.1) to (D.3) were applied to constructed data: 20 000 simulated studies at the planned sixteen dyads, each with five contrasts, in two conditions. In the first, a true effect of exactly $d_z = 0.73$ was added to every contrast. In the second, no effect was added at all. Both were pushed through the same implementation the study will use — `srl_experiments/posthoc.py`, unmodified — by `scripts/validate_analysis_pipeline.py`.

This is the one place in this thesis where synthetic data is admissible, and it is admissible for a specific reason: the ground truth is *constructed* — a constant added to a sample — rather than *rendered*. Nothing here is a claim about the platform. It is a measurement of the analysis.

Table D.1: The pre-registered analysis applied to 20 000 simulated studies per row at $n = 16$ dyads, $m = 5$ contrasts, $\alpha = 0.05$. The null row is the check that can fail: a procedure that rejects everything has perfect power and is worthless, so the false-positive rate is reported beside the power and must come out at α . It does, to three decimal places, which is what licenses reading the row above it.

Constructed truth	probability of rejecting H_0		
	one contrast, uncorrected	one contrast, Holm-corrected	some contrast in the family
$d_z = 0.73$ on all five	0.777	0.638	0.973
no effect at all	0.052	0.010	0.050

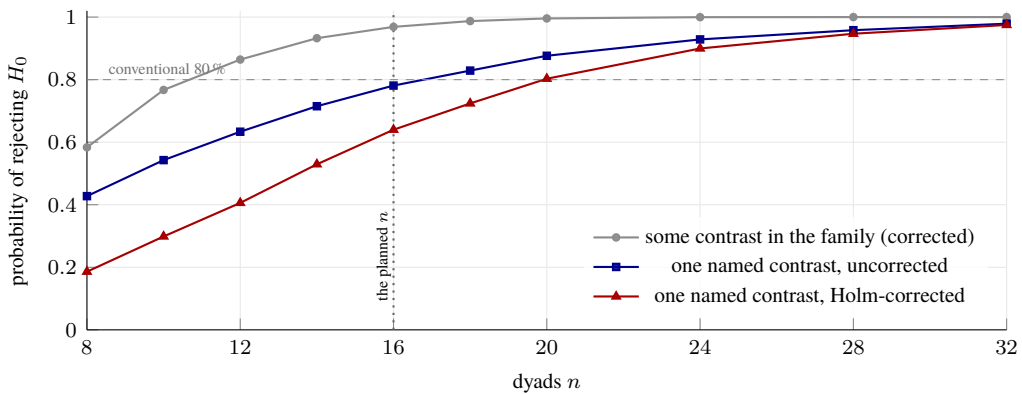


Figure D.1: Power against the number of dyads at $d_z = 0.73$, from 6000 simulated studies per point. The design’s stated 80 % is the middle curve read at $n = 16$, and it is met. The lower curve is the same contrast after the correction the analysis actually applies, and the upper curve is the probability that *some* contrast in the family survives. The three answer three different questions and the design records only the middle one.

Three things follow, and none of them was in the design.

The design’s own number is confirmed, and it is the wrong number to plan with. An uncorrected paired contrast at sixteen dyads recovers a $d_z = 0.73$ effect 77.7 % of the time, which reproduces the planning calculation. But no contrast will be reported uncorrected, because there are five of them.

The correction costs fourteen percentage points. After Holm, the probability of recovering the effect on a *named* task falls to 63.8 %. [Figure D.1](#) shows that restoring 80 % on a named task needs twenty dyads rather than sixteen — eight additional participants, or roughly two additional laboratory days at the session length of [Fig. C.2](#).

The study is nevertheless well powered for the claim it is making. The probability that at least one contrast in the family survives correction is 97.3 %. The design’s question is whether shared autonomy changes performance on this platform, not whether it changes performance on task T3 specifically, and for that question sixteen dyads is ample. The consequence for reporting is precise: a per-task null result at $n = 16$ is not evidence of no per-task effect, and must be reported as an interval rather than as an absence.

The null row of [Table D.1](#) is the half of this measurement that could have failed. A family-wise false-positive rate of 0.050 against a nominal $\alpha = 0.05$ establishes that the correction is implemented as specified rather than merely called; the per-contrast rate of 0.010 is the α/m the step-down procedure is expected to leave at the first step.

D.3 What the session will report

The outputs below are fixed in advance so that the choice of what to report cannot be made after seeing the data. Each names the quantity, the test, and the reason the decomposition is what it is; none is drawn here, because a figure of no data is not a figure.

Table D.2: Every output the session produces, fixed before data collection. Where a measure is reported separately by role, the reason is that the two roles load differently and pooling them would average away the contrast the two-person design exists to expose: only the wearer carries 17 kg, and only the operator holds an uncompensated master arm against gravity.

Quantity	Test	Decomposition, and why
Completion time	paired contrast per task, Holm across five	per task; T2 has no mode contrast (Section C.3.1) and is reported as a single condition
Coordination error	threshold crossing rate, T3 and T4	against the measured failure thresholds, 6.8° of tray tilt and 534 mm of gripper separation
Cross-arm interference	regression of one arm’s tracking error on the other arm’s target speed, T5	the slope reported beside the phase lag: a large slope with a flat lag is attentional, both rising together is not
Grasp success	proportion, autonomy conditions only	no teleoperation row exists and none should be added (Section C.3.1)
NASA-TLX [11]	six subscales and the weighted total	operator and wearer separately
Borg CR10 [3]	slope across blocks, not endpoint	fatigue is monotonic in time, so the slope is the quantity and the endpoint is a confound
Trust	by autonomy level, wearer only	against the prediction that higher autonomy <i>lowers</i> wearer trust [40]
Embodiment	ownership, agency and location separately	the components move independently [20] and a single score hides that
Proprioceptive drift	paired, pre against post	reported per participant against the unity line; two means discard the pairing the design bought

Two constraints on reporting follow from [Section D.2](#) and are stated here rather than discovered later. Effects are reported as estimates with intervals whether or not they cross the threshold, because at $n = 16$ a per-task null is uninformative on its own. And the exchange rate between operator performance and wearer trust — the quantity the two-person framing makes interesting — is a correlation *across* roles, which this design was not powered for; it is reported descriptively with an interval and no test.

D.4 Safety events

Table D.3: Everything the safety layer does during a session, recorded automatically rather than by an observer. This table is reported whether or not anything happens: a table of zeros is a result, and its absence is indistinguishable from a table that was never produced. No field in it identifies a participant; `write_manifest()` raises on name, electronic mail address, date of birth, address and telephone number, so anonymity is enforced by the code that writes the record rather than by the person writing it up.

Event	Direct teleoperation	Shared autonomy
Emergency stops, and by whom	—	—
Clearance holds	—	—
Minimum recorded clearance	—	—
Trials invalidated by a fault, with cause	—	—
Sessions ended at a participant's request	—	—

D.5 Why no session has been run

Two independent blocks, neither of which is a matter of scheduling.

The first is technical and is measured in [Chapter 7](#): the direct teleoperation condition is the baseline of every comparison in [Table D.2](#), and it depends on master channels of which six of fourteen currently meet the coherence criterion. The failing channels are a soldering problem at two identified joints, and until they are repaired the baseline condition would be measuring the master arm's wiring rather than the operator.

The second is ethical and is not resolved by repairing anything. The wearer and the operator are different people who consent separately, and worn operation places more than 17 kg on someone who is not commanding the motion and has no gravity compensation. [Chapter 10](#) sets out, in priority order, what stands between the system as built and a session with two people in it. The first item on that list is two soldered joints; the second is an approved protocol.

Chapter E

Master arm firmware

The complete sketch is at `firmware/master_arm/teensy_final.ino`. The superseded revision, which carried the pin conflict described in [Chapter 3](#), is kept at `firmware/master_arm/teensy_merged_prior.ino` so the change can be diffed.

The inertial-sensor driver header is included by the sketch but was not among the files handed over, and is not in the repository. It is therefore not listed below. Every call site that uses it appears in the listings that follow, so the interface the system depends on is fully specified here even though the implementation is not.

E.1 Pin assignment as declared

Listing E.1: Pin arrays. A4 and A5 appear in neither array, which is the fix described in [Chapter 3](#).

```
1  const int potPinsK1[JOINTS] =
2  {
3    A0, A1, A2, A3, A6, A7, A8      // K1 (left arm) -- 7 channels, no I2C pins
4  };
5  const int potPinsK2[JOINTS] =
6  {
7    A9, A10, A11, A12, A13, A16, A17 // K2 (right arm) -- 7 channels, no I2C pins
8  };
9
10 const int FSR1_PIN = A14;
11 const int FSR2_PIN = A15;
12
13 const int BUTTON1_PIN = 2;
14 const int BUTTON2_PIN = 3;
```

E.2 Signal conditioning as implemented

Listing E.2: The potentiometer filter and the count-to-degree transform. The filter operates in raw ADC units; note that `setup()` seeds the accumulator with `analogRead(pin) << 8`, which is inconsistent with this recurrence and produces the start-up transient quantified in [Table 3.3](#).

```
1  static constexpr int ALPHA_FP    = 64;
2  static constexpr int ADC_MIN_VAL = 50;
3  static constexpr int ADC_MAX_VAL = 4045;
```

```

4
5 inline int updateEMA(int pin, int32_t& emaFP)
6 {
7     int raw = analogRead(pin);
8     emaFP =
9     ((int32_t)ALPHA_FP * raw +
10     (256 - ALPHA_FP) * emaFP) >> 8;
11     return (int)emaFP;
12 }
13
14 inline float adcToDeg(int adc)
15 {
16     float t = constrain(
17     (float)(adc - ADC_MIN_VAL) /
18     (float)(ADC_MAX_VAL - ADC_MIN_VAL),
19     0.0f,
20     1.0f);
21     return t * DEG_MAX;
22 }

```

Listing E.3: Button debounce and toggle. The output is a latched state, so a host that drops frames cannot miss a press.

```

1 static constexpr uint32_t DEBOUNCE_MS = 40;
2
3 inline int updateButtonToggle(
4 int pin, int& state, int& lastRaw, uint32_t& lastChangeMs)
5 {
6     int raw = digitalRead(pin);
7     uint32_t now = millis();
8     if(raw != lastRaw && (now - lastChangeMs) > DEBOUNCE_MS)
9     {
10         lastChangeMs = now;
11         lastRaw = raw;
12         if(raw == LOW)
13         {
14             state = state ? 0 : 1;
15         }
16     }
17     return state;
18 }

```

E.3 Frame construction

Listing E.4: Frame assembly. Inertial values are transmitted raw in LSB, so the scale factors are applied on the host and can be corrected without reflashing.

```

1 for(int j=0;j<JOINTS;j++)
2     p += snprintf(p, end-p, "k1j%d:%.1f,", j+1, degK1[j]);
3 for(int j=0;j<JOINTS;j++)
4     p += snprintf(p, end-p, "k2j%d:%.1f,", j+1, degK2[j]);
5 p += snprintf(p, end-p, "K1IMU:%d,%d,%d,%d,%d,%d,", ax1,ay1,az1,gx1,gy1,gz1);
6 p += snprintf(p, end-p, "K2IMU:%d,%d,%d,%d,%d,%d,", ax2,ay2,az2,gx2,gy2,gz2);

```

```
7 | p += snprintf(p, end-p, "fsr1:%d,fsr2:%d,", fsr1, fsr2);  
8 | p += snprintf(p, end-p, "btn1:%d,btn2:%d", btn1, btn2);
```

Chapter F

Supporting records

The following are maintained in the repository rather than reproduced here, because each is longer than the chapter that cites it and each is generated rather than written.

Table F.1: Supporting records and where they live.

Record	Content
thesis_report/DIVERGENCE.md	Divergence from the planning report, by objective, with the reason and the evidence for each.
thesis_report/EVIDENCE.md	Every numerical claim in this thesis with the file and run that produced it.
recordings/baselines/	Channel-health baselines by date, workspace sweeps, and the five-task verification result.
docs/NEXT_SESSION.md	Current blocking items in priority order.

Chapter G

Evaluation criteria

Each criterion carries a threshold and the justification for that threshold. A criterion with a number and no justification for the number is not a criterion. Two criteria that a reader might expect are deliberately absent and the reason is given at the end.

Table G.1: The nine criteria this work is judged against, with the outcome of each.

	Criterion	Threshold, and its justification	Outcome
1	Convention round trip	Converting a joint angle from the manufacturer's convention to the middleware's and back must return the original to within numerical precision. The failure mode is a near-full revolution of a joint beside a person, so there is no tolerance to spend	Met
2	Directional fidelity	Under 30° per axis. Beyond that an operator must consciously correct rather than move naturally	Partly met: vertical and fore-and-aft pass, lateral does not (Chapter 7)
3	Command tracking in simulation	Under 5 mm RMS. Below that the residual is dominated by the pose deadband rather than by the controller	Met , 0.7 to 4.7 mm
4	Wearer clearance	At least the configured floor at every point of every path, measured geometrically	Not met on three of four task paths; reported per task in Fig. 9.6
5	Safety response completeness	All injected faults detected, responded to and made externally visible. A mechanism handling thirteen of fourteen is not a safety mechanism	Met , 14 of 14
6	Degraded-mode integrity	Under 1 mm of commanded motion contributed by failed channels from a still hand, and a refusal when no positional channel survives	Partly met: right arm 0.014 mm, left arm 54.5 mm; refusal implemented
7	Scenario reachability	Ten repeats of ten over the whole densified path. A pose feasible on 72 to 82 % of calls passes a five-of-five check 19 % of the time, so anything short of N/N is not an established pose	Met , all verified scenarios
8	Control-path latency	Under 200 ms, inherited from the planning report	Not measured end to end; the control-path figures in Chapter 7 are lower bounds
9	Object handling verified in the recording	The fingers must open, close within the holding band and reopen, and the object must track the gripper	Met , all recorded grasp cells

No force or impedance criterion. The mock hardware returns identically zero for every force and effort

field, so any quantity derived from a force reading is *structurally undefined* on this platform rather than merely unmeasured. Force reflection and closed-loop overshoot cannot be defined as criteria until either a dynamic simulator or the physical arm is in the loop.

No angular-tracking criterion. The commanded orientation is pinned to the workspace anchor and never follows the master's wrist, because the wrist channels are dead or incoherent. There is no angular tracking to measure until those channels are repaired.

Chapter H

Equipment

Table H.1: Hardware and software, with the reference for each.

Item	Detail
Manipulators	Two Kinova Gen3, seven degrees of freedom, 8.2 kg each, 0.902 m reach [17]
Grippers	Robotiq two-finger, 85 mm stroke
Wrist cameras	Colour and depth, one per arm
Frame	Backpack harness; arms mounted behind the shoulders with forward tilt
Master arm	Scale model of the same joint sequence: fourteen rotary potentiometers, two inertial units, two force-sensitive resistors, two buttons
Microcontroller	Teensy 4.1 [29], 100 Hz frame rate
Inertial sensors	MPU-6050 at power-on defaults [14, 15]
Controllers	Two tracked hand controllers, headset stationary as the tracking reference (Section 4.4.2)
Host	Laptop, Linux, 4 GiB of video memory
Middleware	ROS 2 [22]
Inverse kinematics	TRAC-IK [1]
Motion generation	Ruckig [2]
Path planning	RRT-Connect [18] in joint space
Plane fitting	RANSAC [8], constrained to the support normal
Detection	YOLO-World [5] with FastSAM [38]
Body tracking	MediaPipe Pose [21]
Prior simulation	MuJoCo [32]; Isaac Lab [24] evaluated and rejected

Chapter I

Where the evidence lives

Every quantity quoted in this thesis is produced by a named program and stored as a record, so that it can be recomputed rather than taken on trust. The mapping below is the index; the records themselves are in the repository.

Table I.1: Principal claims and the programs that produce them.

Claim	Produced by
Channel health verdicts	trajectory_capture/channel_report.py
Cost of each rung of the degradation ladder	scripts/measure_capability_ladder.py
Commanded motion injected by failed channels	scripts/prove_degraded_mode.py
Continuous reachability by direction	scripts/measure_workspace.py
Both-arm overlap and the dead band	scripts/sweep_mount_overlap.py
Mount parameter sweep	scripts/sweep_mount_geometry.py
The centre, asked four ways	scripts/search_centre_on_surface.py, measure_centre_vs_height.py, measure_ centre_vs_wearer_posture.py
Clearance along each task path	scripts/measure_clearance_region.py
Positioning error budget	scripts/measure_control_budget.py
Motion generation and the step sweep	scripts/measure_teleop_motion.py
Simulation-to-hardware terminal error	scripts/measure_sim_to_real_gap.py
Map accuracy	scripts/calibrate_environment.py
Wearer tracking rate and fallbacks	scripts/measure_wearer_tracking_rate.py, verify_wearer_fallbacks.py
Instruction handling	scripts/sweep_t1_instructions.py
Fault injection	srl_teleop/fault_injector.py
Operating window audit	scripts/verify_gui_buttons.py
Test totals	scripts/check_tests.py

Chapter J

Participant documentation

To be attached before submission: the participant information sheet, both consent forms, the pre-session screening questions and the debrief script. Two consent forms rather than one, because the two roles carry materially different risks: one participant carries 17 kg and is within reach of a manipulator moving under another participant's command, and the other is not. Drafts exist in the project's research records and are not reproduced here because they have not been through ethical review.

Bibliography

- [1] Patrick Beeson and Barrett Ames. TRAC-IK: An open-source library for improved solving of generic inverse kinematics. In *Proc. IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids)*, pages 928–935. IEEE, 2015.
- [2] Lars Berscheid and Torsten Kröger. Jerk-limited real-time trajectory generation with arbitrary target states. *Robotics: Science and Systems XVII*, 2021.
- [3] Gunnar A. Borg. Psychophysical bases of perceived exertion. *Medicine and Science in Sports and Exercise*, 14(5):377–381, 1982.
- [4] Matthew Botvinick and Jonathan Cohen. Rubber hands ‘feel’ touch that eyes see. *Nature*, 391(6669):756, 1998.
- [5] Tianheng Cheng, Lin Song, Yixiao Ge, Wenyu Liu, Xinggang Wang, and Ying Shan. YOLO-World: Real-time open-vocabulary object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [6] S. de Wolde, J. Kober, and R. Babuska. Current-based impedance control for interacting with mobile manipulators. *arXiv preprint arXiv:2403.13079*, 2024.
- [7] Anca D. Dragan and Siddhartha S. Srinivasa. A policy-blending formalism for shared control. *The International Journal of Robotics Research*, 32(7):790–805, 2013.
- [8] Martin A. Fischler and Robert C. Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.
- [9] Paul M. Fitts. The information capacity of the human motor system in controlling the amplitude of movement. *Journal of Experimental Psychology*, 47(6):381–391, 1954.
- [10] Jacob W. Guggenheim and H. Harry Asada. Inherent haptic feedback from supernumerary robotic limbs. *IEEE Transactions on Haptics*, 14(1):123–131, 2021.
- [11] Sandra G. Hart and Lowell E. Staveland. Development of NASA-TLX (task load index): Results of empirical and theoretical research. In Peter A. Hancock and Najmedin Meshkati, editors, *Human Mental Workload*, volume 52 of *Advances in Psychology*, pages 139–183. North-Holland, 1988.
- [12] Imperial College London. MUlti-limb Virtual Environment (Dr. Octopus). <https://www.imperial.ac.uk/a-z-research/human-robotics/dr-octopus-/>. Accessed April 2026.
- [13] International Society of Automation. *ANSI/ISA-101.01-2015: Human Machine Interfaces for Process Automation Systems*, 2015.

- [14] InvenSense Inc. *MPU-6000 and MPU-6050 Product Specification, Revision 3.4*, 2013. Document PS-MPU-6000A-00.
- [15] InvenSense Inc. *MPU-6000 and MPU-6050 Register Map and Descriptions, Revision 4.2*, 2013. Document RM-MPU-6000A-00.
- [16] Shervin Javdani, Siddhartha S. Srinivasa, and J. Andrew Bagnell. Shared autonomy via hindsight optimization. In *Robotics: Science and Systems XI*, 2015.
- [17] Kinova Robotics. *Gen3 Ultra Lightweight Robot User Guide*, 2022.
- [18] James J. Kuffner and Steven M. LaValle. RRT-Connect: An efficient approach to single-query path planning. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 995–1001, 2000.
- [19] C. Lee, S. Kwak, and J. Kwak. Development, analysis, and control of series elastic actuator for a wearable robotic leg. *Frontiers in Neurobotics*, 13:17, 2019.
- [20] Matthew R. Longo, Friederike Schüür, Marjolein P. M. Kammers, Manos Tsakiris, and Patrick Haggard. What is embodiment? a psychometric approach. *Cognition*, 107(3):978–998, 2008.
- [21] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Guang Yong, Juhyun Lee, et al. MediaPipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*, 2019.
- [22] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022.
- [23] C. Moon and J. Kim. Assessing the physical impact of supernumerary limbs on a human subject: A simulation study. In *Proc. 46th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, pages 1–4. IEEE, 2024.
- [24] NVIDIA. Isaac lab documentation: Impedance control and motion generators. <https://isaac-sim.github.io/IsaacLab>, 2026. Accessed April 2026.
- [25] Federico Parietti and H. Harry Asada. Dynamic analysis and state estimation for wearable robotic limbs subject to human-induced disturbances. In *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, pages 3880–3887. IEEE, 2013.
- [26] Federico Parietti and H. Harry Asada. Supernumerary robotic limbs for aircraft fuselage assembly: Body stabilization and guidance by bracing. In *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, pages 1176–1183. IEEE, 2014.
- [27] Federico Parietti and H. Harry Asada. Supernumerary robotic limbs for human body support. *IEEE Transactions on Robotics*, 32(2):301–311, 2016.
- [28] Mattia Pinardi, Matthew R. Longo, Domenico Formica, Matija Strbac, Carsten Mehring, Etienne Burdet, and Giovanni Di Pino. Impact of supplementary sensory feedback on the control and embodiment in human movement augmentation. *Communications Engineering*, 2:64, 2023.
- [29] PJRC. Teensy 4.1 development board. <https://www.pjrc.com/store/teensy41.html>. Pinout reference card rev. 4. Accessed August 2026.

- [30] Domenico Prattichizzo et al. Human augmentation by wearable supernumerary robotic limbs: review and perspectives. *Progress in Biomedical Engineering*, 2021.
- [31] MHD Yamen Saraiji, Tomoya Sasaki, Reo Matsumura, Kouta Minamizawa, and Masahiko Inami. Fusion: full body surrogacy for collaborative communication. In *ACM SIGGRAPH 2018 Emerging Technologies*, pages 1–2. ACM, 2018.
- [32] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033. IEEE, 2012.
- [33] Yuchen Tong and Jianbin Liu. Review of research and development of supernumerary robotic limbs. *IEEE/CAA Journal of Automatica Sinica*, 2021.
- [34] H. Toubar, G. Goldsmith, S. A. Tafrishi, M. Innala, and V. Bergstromer. Design, modeling, and control of a series elastic actuator with discretely adjustable stiffness. *Mechanism and Machine Theory*, 170:104720, 2022.
- [35] Bo Yang, Jian Huang, Xu Chen, Caihua Xiong, and Yasuhisa Hasegawa. Supernumerary robotic limbs: A review and future outlook. *IEEE Transactions on Medical Robotics and Bionics*, 3(3):623–639, 2021.
- [36] K. Zhang et al. Review of supernumerary robotic limbs. *Journal of Physics: Conference Series*, 2023.
- [37] Y. Zhang et al. Motion-compensation control of supernumerary robotic arms subject to human-induced disturbances. *Advanced Intelligent Systems*, 2024.
- [38] Xu Zhao, Wenchao Ding, Yongqi An, Yinglong Du, Tao Yu, Min Li, Ming Tang, and Jinqiao Wang. Fast segment anything. *arXiv preprint arXiv:2306.12156*, 2023.
- [39] Y. Zheng et al. Supernumerary robotic limbs: A quantitative analysis and review of design, embodiment, and future challenges. *Journal of Field Robotics*, 2026.
- [40] Haoyuan Zhou, Anzhu Gao, Pierre Ducos, Jonathan Eden, and Etienne Burdet. Spatial guidelines for supernumerary robotic limbs in near-body interactions. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. ACM, 2026.